Java Collections Framework

1. What is Collection?

1.1 Definition

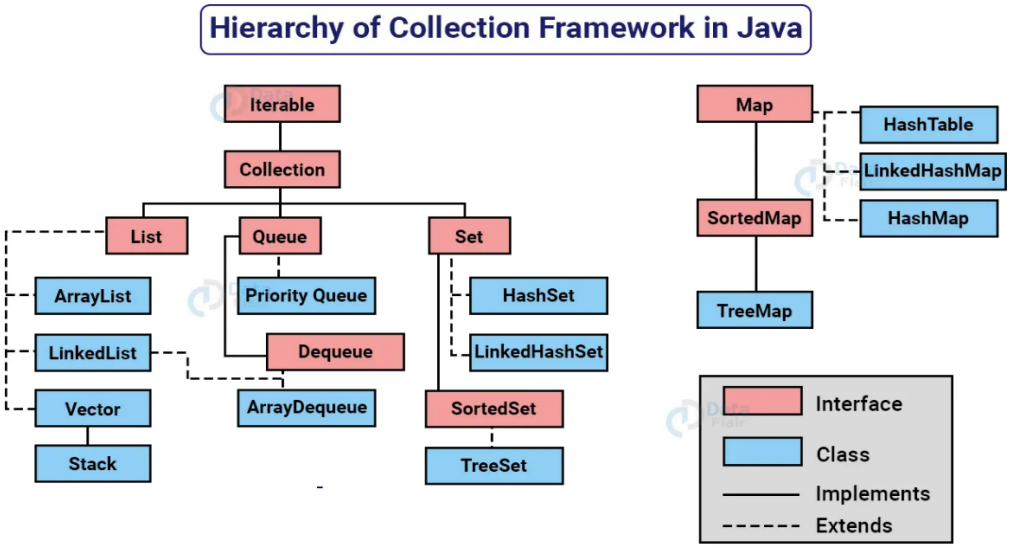
In Java, a **Collection** represents a single unit of objects, known as elements. The **Java Collections Framework (JCF)** is a unified architecture for representing and manipulating these collections. It is a set of interfaces, implementations, and algorithms located primarily in the java.util package.

- Java provides collection interfaces like List, Set, Map, and Queue, with ready-made classes such as ArrayList, HashSet, HashMap, and PriorityQueue, so you don't have to write data-handling code from scratch.
- The Collection Framework improves productivity by making code more reusable, maintainable and faster to develop.

1.2 The Hierarchy

The framework is divided into two distinct trees:

1. **The Collection Interface:** The root for Lists, Sets, and Queues.
2. **The Map Interface:** A separate root for Key-Value pairs (does not extend Collection).



Features of Java Collection Framework

- Provides ready-to-use data structures (e.g., ArrayList, HashSet, HashMap).9
- Offers interfaces (Collection, List, Set, Map, Queue) to define standard behaviors.
- Supports dynamic resizing, unlike arrays with a fixed size.
- Includes algorithms (sorting, searching, iteration) via the Collections utility class.
- Improves code reusability and performance by reducing boilerplate code.

Key Interfaces

- **Collection<E>:** The root interface. Defines basic methods like add, remove, size, iterator.
- **List<E>:** An ordered sequence. Allows duplicates. Accessible by index.
- **Set<E>:** A collection of unique elements. No duplicates.
- **Map<K, V>:** Maps keys to values. Keys are unique; values can be duplicated.
- **Queue<E>:** Holds elements prior to processing. Typically FIFO (First-In-First-Out).

2. Why we use?

2.1 Limitations of Arrays

Before Collections, Java developers used Arrays. Arrays have significant limitations:

1. Fixed Size: Once created, size cannot change (int[] arr = new int[5]).
2. Homogeneous: Typically stores only one data type (unless using Object[]).
3. No Built-in Algorithms: No ready-made methods for sorting, searching, or shuffling.
4. Memory Management: No dynamic memory handling.

2.2 Advantages of Collections

1. Dynamic Size: Grows and shrinks automatically as needed.
2. Heterogeneous: Can store different types of objects (via Object or Generics).
3. Ready-made Algorithms: Built-in sorting, searching, synchronization, and shuffling (Collections class).
4. Interoperability: All collections share a common API, making it easy to switch implementations (e.g., swapping ArrayList for LinkedList).
5. Memory Efficiency: Optimized internal memory management (though some overhead exists).

2.3 Comparison: Array vs. Collection

Feature	Array	Collection
Size	Fixed	Dynamic
Type	Homogeneous (mostly)	Heterogeneous (Objects)
Performance	Faster (Direct Memory)	Slightly slower (Method overhead)
Features	Basic	Rich (Sort, Search, Sync)
Usage	When size is known & fixed	When size varies & features needed

---

3. What we use? (The Classes)

This section lists the concrete implementations you will actually instantiate in code.

3.1 List Implementations

- **ArrayList**: Resizable array. Fast random access.
- **LinkedList:** Doubly-linked list. Fast insertion/deletion.
- **Vector:** Synchronized array (Legacy, avoid).
- **Stack:** Synchronized LIFO (Legacy, use ArrayDeque).

3.2 Set Implementations

- **HashSet:** Unordered, unique elements. Backed by HashMap.
- **LinkedHashSet:** Ordered by insertion. Backed by LinkedHashMap.
- **TreeSet:** Sorted unique elements. Backed by TreeMap.

3.3 Map Implementations

- **HashMap:** Unordered, unique keys. Allows nulls.
- **LinkedHashMap:** Ordered by insertion/access.
- **TreeMap**: Sorted by keys.
- **Hashtable:** Synchronized, no nulls (Legacy).
- **ConcurrentHashMap:** Thread-safe, high performance.

3.4 Queue Implementations

- **PriorityQueue**: Priority heap (Min/Max).
- **ArrayDeque:** Double-ended queue (Stack/Queue).
- **LinkedList:** Also implements Queue.

The Collections Utility Class

In addition to the Collection interface, the java.util package provides a utility class named **Collections** (note the 's' at the end). This class consists exclusively of static methods for performing operations on collections, such as:

- sort(List list)
- binarySearch(List list, Object key)
- reverse(List list)
- synchronizedList(List list) to create a thread-safe (synchronized) list

The Collection Interface & Utility Classes

This module covers the root of the Java Collections Framework. Before diving into specific implementations like ArrayList or HashMap, you must understand the foundational interfaces and the utility classes that manipulate them.

1. Iterable Interface

1.1 What is it?

Definition: The root interface of the entire Collection hierarchy.

Location: java.lang.Iterable

Signature: public interface Iterable<T>

Relationship: Collection extends Iterable.

1.2 Why we use?

Enhanced For-Loop: It enables the "foreach" loop syntax (for (Type var : collection)).

Standardization: It guarantees that any class implementing it can be traversed sequentially.

Separation: It separates the concept of "being traversable" from "being a collection" (e.g., arrays are not Collections, but can be iterated).

1.3 How we use?

```
// You rarely implement Iterable directly unless creating a custom collection

public class MyContainer implements Iterable<String> {

   public Iterator<String> iterator() {

      // return custom iterator

   }

}

// Usage in consumer code

List<String> list = new ArrayList<>();

list.add("A");

// The foreach loop calls iterable.iterator() behind the scenes

for (String s : list) {

   System.out.println(s);

}
```

1.4 Internal Working

Method: Defines a single method: Iterator<T> iterator().

Mechanism: When the JVM encounters a foreach loop, it compiles it into a while loop using the Iterator returned by this method.

Memory: No state is stored in the interface itself; it simply contracts the implementing class to provide a cursor.

Collection Interface

2.1 What is it?

Definition: The root interface for Lists, Sets, and Queues. (Note: Map does not extend Collection).

Location: java.util.Collection

Signature: public interface Collection<E> extends Iterable<E>

Type: It is an Interface, not a class. You cannot instantiate it (new Collection() is illegal).

2.2 Why we use?

Polymorphism: Allows you to write methods that accept any type of collection (void process(Collection<?> c)).

Unified API: Defines standard methods (add, remove, size, isEmpty, contains) that work across ArrayList, HashSet, LinkedList, etc.

Bulk Operations: Provides methods to operate on groups of elements at once (addAll, removeAll, retainAll, containsAll).

2.3 How we use?

```
// Programming to the interface, not the implementation

Collection<String> c = new ArrayList<>();

// OR

Collection<String> c = new HashSet<>();

c.add("Item");

c.remove("Item");

int size = c.size();

boolean exists = c.contains("Item");

c.clear();
```

Internal Working

- **Method Definitions:** Most methods are abstract (no body). Implementations (like ArrayList) provide the logic.
- **Default Methods (Java 8+):** Includes default implementations for Stream API integration (stream(), parallelStream()) and removal based on predicates (removeIf()).
- **Optional Methods:** Some methods are "optional". For example, an immutable collection might throw UnsupportedOperationException when add() is called.
- **Null Handling:** The interface allows null elements, but specific implementations (like TreeSet) may reject them.

3. Iterator Interface

3.1 What is it?

- **Definition:** A cursor that allows you to traverse a collection and remove elements during iteration.
- **Location:** java.util.Iterator
- **Signature:** public interface Iterator<E>

3.2 Why we use?

- **Safe Removal:** You cannot safely remove elements from a collection using a foreach loop (throws ConcurrentModificationException). You **must** use Iterator.remove().
- **Uniform Traversal:** Provides a standard way to loop over List, Set, or Queue without knowing the underlying structure.
- **One-Way:** Unlike ListIterator, this only moves forward.

3.3 How we use?

```
Iterator<String> it = collection.iterator();

while (it.hasNext()) {

  String s = it.next();

  if (s.equals("RemoveMe")) {

    it.remove(); // Safe removal

  }

}
```

3.4 Internal Working (Critical)

- **Cursor:** Maintains an internal index or pointer to the current position.
- **Fail-Fast Mechanism:**
  - Collections maintain a modCount (modification count) variable.
  - The Iterator captures the modCount at creation (expectedModCount).
  - Every time next() or remove() is called, the Iterator checks if modCount == expectedModCount.
  - If the collection was modified externally (e.g., list.add() called while iterating), the counts mismatch, and it throws ConcurrentModificationException.
- **Remove Logic:** When it.remove() is called, it calls the underlying collection's remove method and updates expectedModCount to match the new modCount, preventing the exception.

---

4. Collections Utility Class

4.1 What is it?

- **Definition:** A static utility class consisting exclusively of static methods that operate on or return objects.
- **Location:** java.util.Collections (Note the **'s'** at the end).
- **Constructor:** It has a private constructor; you cannot instantiate it (new Collections() is illegal).

4.2 Why we use?

- **Algorithms:** Provides ready-made algorithms like Sorting, Shuffling, Reversing, and Searching.
- **Wrappers:** Provides methods to wrap collections to make them Synchronized, Unmodifiable, or Checked.
- **Singletons:** Provides empty or singleton list/set/map instances to save memory.

4.3 How we use?

```
List<Integer> list = new ArrayList<>();

list.add(5); list.add(2); list.add(9);

// Sorting

Collections.sort(list);

// Shuffling

Collections.shuffle(list);
```

```
// Binary Search (List must be sorted)
int index = Collections.binarySearch(list, 5);
// Synchronization
List<String> syncList = Collections.synchronizedList(new ArrayList<>());
// Unmodifiable (Read-Only)
List<String> readOnly = Collections.unmodifiableList(list);
// Empty Singleton
List<String> empty = Collections.emptyList();
```

4.4 Internal Working

- **Sorting (sort):**
  - For Objects: Uses **TimSort** (Hybrid of Merge Sort and Insertion Sort). Stable (preserves order of equal elements). O(n log n).
  - For Primitives (via Arrays.sort): Uses **Dual-Pivot Quicksort**. Unstable. O(n log n).
- **Synchronization (synchronizedList):**
  - Returns a wrapper class (e.g., SynchronizedList).
  - Every method call acquires a **mutex lock** (synchronized(mutex)) before delegating to the underlying list.
  - **Warning:** You must manually synchronize around the iterator loop when using these.
- **Unmodifiable (unmodifiableList):**
  - Returns a wrapper class.
  - All mutating methods (add, remove) throw UnsupportedOperationException.
  - **Shallow Immunity:** The list structure cannot change, but the objects *inside* can still be modified.
- **Binary Search:**
  - Requires a sorted list.
  - Uses divide-and-conquer logic. O(log n).

Common Pitfalls

1. **Collection vs. Collections:**
   - Collection is the **interface** (Root of hierarchy).
   - Collections is the **utility class** (Static methods).

- *Mistake:* Trying to new Collections() or calling Collection.sort().

2. **ConcurrentModificationException:**
   - *Cause:* Modifying a collection (add/remove) while iterating using foreach loop.
   - *Fix:* Use Iterator.remove() or RemoveIf() (Java 8).
3. **Synchronized Wrappers:**
   - *Cause:* Thinking Collections.synchronizedList makes iteration thread-safe automatically.
   - *Fix:* You must manually lock during iteration
4. **Comparable Consistency:**
   - *Cause:* `compareTo` says objects are equal (0), but `equals` says false.
   - *Effect:* `TreeSet` might treat them as duplicates even if `HashSet` would not.

Fail-Fast, Fail-Safe & ConcurrentModificationException Deep Dive

This is one of the **most important and misunderstood topics** in Java Collections. Understanding this is critical for writing bug-free multi-threaded code and acing technical interviews.

---

1. What is ConcurrentModificationException?

1.1 Definition

ConcurrentModificationException is a **runtime exception** thrown when a collection is **structurally modified** during iteration (except through the iterator's own remove() or add() methods).

1.2 When Does It Occur?

java

```
// ❌ WRONG - Throws ConcurrentModificationException
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");

for (String s : list) {
    if (s.equals("B")) {
```

```
    list.remove(s);  // ❌ EXCEPTION!
  }
}
```

Output :

```
// ❌ WRONG - Throws ConcurrentModificationException
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
for (String s : list) {
  if (s.equals("B")) {
    list.remove(s);  // ❌ EXCEPTION!
  }
}
```

1.3 Why Does Java Throw This Exception?

1. **Data Integrity:** Prevents unpredictable behavior from concurrent modifications.
2. **Fail-Fast Design:** Detects bugs early rather than allowing silent corruption.
3. **Iterator Consistency:** Ensures iterator sees a consistent view of the collection.

---

2. Fail-Fast vs Fail-Safe: The Core Difference

2.1 Comparison Table

Feature	Fail-Fast	Fail-Safe
Exception	Throws ConcurrentModificationException	**No exception**
Iterator Source	Original collection	**Clone/Snapshot** of collection
Memory Overhead	Low (no copy)	High (creates copy)
Performance	Faster (no copying)	Slower (copy overhead)

Feature	Fail-Fast	Fail-Safe
Data Visibility	Sees latest changes (until exception)	Sees **snapshot** at iterator creation
Thread Safety	❌ Not thread-safe	✅ Thread-safe
Examples	ArrayList, HashMap, HashSet	CopyOnWriteArrayList, ConcurrentHashMap

2

.2 Visual Comparison

```
┌──────────────────────────────────────────────────────────┐
│                    FAIL-FAST ITERATOR                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Collection: [A, B, C, D]                                │
│        │                                                 │
│        ↓ Iterator created                                │
│        │                                                 │
│  Collection: [A, B, C, D]  ← Iterator points HERE        │
│        │                                                 │
│        ↓ External modification (list.remove("B"))        │
│        │                                                 │
│  Collection: [A, C, D]     ← MODIFIED!                   │
│        │                                                 │
│        ↓ Iterator.next() called                          │
│        │                                                 │
│  ❌ ConcurrentModificationException THROWN!              │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│                    FAIL-SAFE ITERATOR                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Collection: [A, B, C, D]                                │
│        │                                                 │
│        ↓ Iterator created                                │
│        │                                                 │
│  Snapshot: [A, B, C, D]    ← Iterator works on COPY      │
│        │                                                 │
│  Collection: [A, B, C, D]                                │
│        │                                                 │
│        ↓ External modification (list.remove("B"))        │
│        │                                                 │
│  Collection: [A, C, D]     ← Original modified           │
│  Snapshot:   [A, B, C, D]  ← Iterator still sees OLD data │
│        │                                                 │
│        ↓ Iterator.next() called                          │
│        │                                                 │
│  ✅ NO EXCEPTION - Iterates over snapshot                │
└──────────────────────────────────────────────────────────┘
```

3. Internal Working: The modCount Mechanism

3.1 How Fail-Fast Detection Works

This is the **most critical internal working** to understand for interviews.

```
// Simplified ArrayList Internal Code

public class ArrayList<E> {

   // Tracks structural modifications

   protected transient int modCount = 0;
```

```
// Add method increments modCount
public boolean add(E e) {
   ensureCapacityInternal(size + 1);
   elementData[size++] = e;
   modCount++;  // ← Incremented on EVERY structural change
   return true;
}
  // Remove method increments modCount
public E remove(int index) {
   modCount++;  // ← Incremented
   // ... removal logic
}
  // Iterator class
private class Itr implements Iterator<E> {
   int cursor;        // Index of next element to return
   int expectedModCount; // Iterator's copy of modCount
         Itr() {
      this.expectedModCount = ArrayList.this.modCount;
      // ← Captures modCount at iterator creation
   }
         public E next() {
      checkForComodification();  // ← Check BEFORE returning element
      // ... return element
   }
         public void remove() {
      // ... removal logic
      expectedModCount = modCount;  // ← Update expectedModCount
   }
```

```
    final void checkForComodification() {
      if (modCount != expectedModCount) {
        throw new ConcurrentModificationException();
        // ← THROWS if collection was modified externally
      }
    }
  }
}
```

3.2 Step-by-Step Execution Flow

```
┌──────────────────────────────────────────────────────────┐
│                FAIL-FAST DETECTION FLOWCHART             │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  1. Create Iterator                                      │
│     expectedModCount = collection.modCount (e.g., 0)     │
│                                                          │
│  2. Modify Collection Externally                         │
│     list.add("X")                                        │
│     collection.modCount++ (now 1)                        │
│                                                          │
│  3. Call iterator.next()                                 │
│     ↓                                                    │
│  4. checkForComodification()                             │
│     if (modCount != expectedModCount)                    │
│        (1 != 0) → TRUE                                   │
│     ↓                                                    │
│  5. Throw ConcurrentModificationException!               │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

3.3 What Counts as Structural Modification?

Operation	Structural Change?	Triggers Exception?
add()	✅ Yes	✅ Yes
remove()	✅ Yes	✅ Yes
clear()	✅ Yes	✅ Yes
addAll()	✅ Yes	✅ Yes
set(index, element)	❌ No (replaces value)	❌ No
iterator.remove()	✅ Yes	❌ No (updates expectedModCount)
iterator.add()	✅ Yes	❌ No (updates expectedModCount)

Key Insight: Changing a value (set()) doesn't change structure, so it's safe during iteration.

4. Which Collections Are Fail-Fast vs Fail-Safe?

4.1 Fail-Fast Collections

Collection	Package	Iterator Type
ArrayList	java.util	Fail-Fast
LinkedList	java.util	Fail-Fast
HashSet	java.util	Fail-Fast
HashMap	java.util	Fail-Fast
TreeSet	java.util	Fail-Fast
TreeMap	java.util	Fail-Fast
Vector	java.util	Fail-Fast
Stack	java.util	Fail-Fast

4

.2 Fail-Safe Collections

Collection	Package	Iterator Type
CopyOnWriteArrayList	java.util.concurrent	Fail-Safe
CopyOnWriteArraySet	java.util.concurrent	Fail-Safe
ConcurrentHashMap	java.util.concurrent	Fail-Safe (Weakly Consistent)
ConcurrentLinkedQueue	java.util.concurrent	Fail-Safe (Weakly Consistent)
ConcurrentSkipListSet	java.util.concurrent	Fail-Safe (Weakly Consistent)
ConcurrentSkipListMap	java.util.concurrent	Fail-Safe (Weakly Consistent)

4.

3 BlockingQueue Collections

Collection	Iterator Type	Notes
ArrayBlockingQueue	Weakly Consistent	May or may not reflect modifications
LinkedBlockingQueue	Weakly Consistent	May or may not reflect modifications
PriorityBlockingQueue	Weakly Consistent	No guaranteed order

---

5. How to Avoid ConcurrentModificationException

5.1 Solution 1: Use Iterator.remove()

```
// ✅ CORRECT - Use Iterator's remove method
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
Iterator<String> it = list.iterator();
while (it.hasNext()) {
  String s = it.next();
  if (s.equals("B")) {
    it.remove();  // ✅ Safe - updates expectedModCount
  }
}
// Result: [A, C]
```

Why It Works: Iterator.remove() updates expectedModCount to match the new modCount.

5.2 Solution 2: Use removeIf() (Java 8+)

```
// ✅ CORRECT - Java 8+ removeIf
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
list.removeIf(s -> s.equals("B"));  // ✅ Safe and clean
// Result: [A, C]
```

Internal Working: removeIf() uses an iterator internally and handles modCount correctly.

5.3 Solution 3: Use Fail-Safe Collection

```
// ✅ CORRECT - Use CopyOnWriteArrayList
List<String> list = new CopyOnWriteArrayList<>();
list.add("A");
list.add("B");
list.add("C");
for (String s : list) {
  if (s.equals("B")) {
    list.remove(s);  // ✅ No exception (fail-safe)
  }
}
// Result: [A, C]
// Note: Iterator still sees [A, B, C] (snapshot)
```

5.4 Solution 4: Collect Then Remove

```
// ✅ CORRECT - Collect items to remove, then remove after iteration
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
List<String> toRemove = new ArrayList<>();
for (String s : list) {
  if (s.equals("B")) {
    toRemove.add(s);
  }
}
list.removeAll(toRemove);  // ✅ Remove after iteration
// Result: [A, C]
```

5.5 Solution 5: Use ListIterator

```
// ✅ CORRECT - ListIterator supports add/remove during iteration
List<String> list = new ArrayList<>();
list.add("A");
list.add("B");
list.add("C");
ListIterator<String> it = list.listIterator();
while (it.hasNext()) {
  String s = it.next();
  if (s.equals("B")) {
    it.remove();  // ✅ Safe
    it.add("B1"); // ✅ Can also add
  }
}
// Result: [A, B1, C]
```

5.6 Solution 6: Use ConcurrentHashMap for Maps

```
// ✅ CORRECT - ConcurrentHashMap is fail-safe
Map<String, Integer> map = new ConcurrentHashMap<>();
map.put("A", 1);
map.put("B", 2);
map.put("C", 3);

for (Map.Entry<String, Integer> entry : map.entrySet()) {
  if (entry.getValue() == 2) {
    map.remove(entry.getKey());  // ✅ No exception
  }
}
```

7. Weakly Consistent Iterators (Advanced)

7.1 What Is Weakly Consistent?

Some concurrent collections (ConcurrentHashMap, ConcurrentLinkedQueue) have iterators that are **neither strictly fail-fast nor fail-safe**.

Property	Description
No Exception	Never throws ConcurrentModificationException
May Reflect Changes	May or may not show modifications made after iterator creation
No Guarantee	No guarantee of seeing all elements
Single Pass	Each element returned at most once

7.3 Comparison

Property	Fail-Fast	Fail-Safe	Weakly Consistent
Exception	Yes	No	No
Snapshot	No	Yes	No
Sees Changes	Until exception	Never	May/May Not
Performance	Fast	Slower (copy)	Fastest
Use Case	Single-threaded	CopyOnWrite	Concurrent collections

Q1: What is ConcurrentModificationException?

Answer: Runtime exception thrown when a collection is structurally modified during iteration (except through iterator's own methods). It's a fail-fast mechanism to detect bugs early.

Q2: How does fail-fast detection work internally?

Answer: Collections maintain a modCount variable incremented on every structural modification. Iterators capture modCount at creation as expectedModCount. On each next() call, they compare the two values. If different, throw exception.

Q3: What's the difference between fail-fast and fail-safe?

Answer:

- **Fail-fast:** Throws exception on concurrent modification, works on original collection, low memory.
- **Fail-safe:** No exception, works on snapshot/clone, higher memory overhead.

Q4: How do you safely remove elements during iteration?

Answer:

1. Use Iterator.remove()
2. Use removeIf() (Java 8+)
3. Use fail-safe collection (CopyOnWriteArrayList)
4. Collect then remove after iteration

Q5: Is ConcurrentHashMap fail-fast or fail-safe?

Answer: Weakly consistent - neither strictly fail-fast nor fail-safe. Iterators don't throw exception but may not reflect all modifications.

Q6: Why doesn't set() trigger ConcurrentModificationException?

Answer: set() replaces a value but doesn't change the **structure** (size) of the collection. modCount is only incremented on structural changes (add/remove/clear).

Q7: Can you get ConcurrentModificationException in single-threaded code?

Answer: Yes! It's not just about threads. Any external modification during iteration triggers it, even in single-threaded code.

Q8: What collections are fail-safe?

Answer: CopyOnWriteArrayList, CopyOnWriteArraySet, ConcurrentHashMap, ConcurrentLinkedQueue, ConcurrentSkipListSet, ConcurrentSkipListMap.

---

10. Best Practices Summary

Scenario	Recommended Approach
Single-threaded removal during iteration	Iterator.remove() or removeIf()
Multi-threaded read-heavy	CopyOnWriteArrayList
Multi-threaded map access	ConcurrentHashMap
High-throughput queue	ConcurrentLinkedQueue
Producer-consumer	BlockingQueue
Need to modify while iterating	ListIterator
Avoid exception at all costs	Fail-safe collections
Detect bugs early	Fail-fast collections (default)

Summary

Concept	Key Takeaway
ConcurrentModificationException	Thrown on external modification during iteration
modCount	Internal counter tracking structural changes
expectedModCount	Iterator's copy of modCount for comparison
Fail-Fast	Throws exception (ArrayList, HashMap)
Fail-Safe	No exception, uses snapshot (CopyOnWriteArrayList)
Weakly Consistent	No exception, may see changes (ConcurrentHashMap)
Safe Removal	Use Iterator.remove(), removeIf(), or fail-safe collections
Structural Change	add/remove/clear trigger exception, set() does not

List Implementations

This module covers the concrete classes that implement the List interface. Lists are ordered collections (sequences) that allow duplicate elements and provide positional access (index-based).

---

1. ArrayList

1.1 What is it?

- **Definition:** A resizable array implementation of the List interface.
- **Package:** java.util
- **Characteristics:** Ordered (insertion order), allows duplicates, allows null, not synchronized (not thread-safe), random access enabled.

1.2 Why we use?

- **Performance:** It offers the best performance for **read-heavy** operations.
- **Simplicity:** It is the most intuitive list implementation (behaves like a dynamic array).
- **Memory:** It is more memory-efficient than LinkedList for large datasets because it doesn't store node pointers.

1.3 When to use?

- ✅ **Frequent Read Access:** When you frequently call get(index).
- ✅ **End Modifications:** When you mostly add/remove elements from the **end** of the list.

- ❌ **Middle Modifications:** Avoid if you frequently insert/remove from the **middle** (shifting overhead).
- ❌ **Multi-threaded Write:** Avoid if multiple threads modify it simultaneously (use CopyOnWriteArrayList or synchronized wrapper).

HOW TO USE:

```
import java.util.ArrayList;
import java.util.List;
public class Main {
  public static void main(String[] args) {
    // Default capacity is 10
    List<String> list = new ArrayList<>();
    // Set initial capacity to avoid resizing overhead
    List<String> optimized = new ArrayList<>(100);
    list.add("A");        // O(1)
    list.add("B");
    list.get(0);          // O(1) - Fast random access
    list.set(0, "Z");     // O(1)
    list.remove(1);       // O(n) - Shifts elements
    list.contains("A");   // O(n)
  }
}
```

1.5 Internal Working

- **Backing Array:** Uses a standard Java array Object[] elementData.
- **Capacity:**
  - Default initial capacity is **10**.
  - When the array is full, it resizes.
-
-

- **Resizing Logic:**
  - New Capacity = oldCapacity + (oldCapacity >> 1) (Approx **1.5x growth**).
  - Creates a new array and copies elements using System.arraycopy() (native method, fast but O(n)).
- **Index Access:** Direct memory calculation (base_address + index * size), making get(i) **O(1)**.
- **Fail-Fast:** Uses modCount. If the list is structurally modified during iteration (except via iterator's remove), it throws ConcurrentModificationException.

1.6 Where to use (Real-World)

- Storing a list of items to display in a UI table.
- Caching recent results where order matters.
- Any scenario where you know the approximate size and read more than write.

2. LinkedList

2.1 What is it?

- **Definition:** A doubly-linked list implementation of the List and Deque interfaces.
- **Package:** java.util
- **Characteristics:** Ordered, allows duplicates, allows null, not synchronized.

2.2 Why we use?

- **Insertion/Deletion:** Efficient manipulation of elements without shifting array contents.
- **Queue/Stack:** Can act as a Queue (FIFO) or Stack (LIFO) due to Deque implementation.
- **No Resizing:** No need to copy arrays when growing.

2.3 When to use?

- ✅ **Frequent Add/Remove:** Especially at the **beginning** or **end** of the list.
- ✅ **Queue/Stack Logic:** When you need addFirst, removeLast, etc.
- ❌ **Random Access:** Avoid if you need get(index) frequently (slow traversal).
- ❌ **Memory Constrained:** Avoid if memory is tight (high overhead per element).

2.4 How to use?

```
import java.util.LinkedList;

import java.util.List;

public class Main {

  public static void main(String[] args) {

    LinkedList<String> list = new LinkedList<>();

    list.add("A");          // Adds to end

    list.addFirst("Start");    // O(1)

    list.addLast("End");      // O(1)

    list.removeFirst();      // O(1)

    list.removeLast();       // O(1)

    list.get(5);            // O(n) - Traverses from head or tail

  }

}
```

2.5 Internal Working

- **Node Structure:** Each element is wrapped in a private static class Node

```
private static class Node<E> {

  E item;

  Node<E> next;

  Node<E> prev;

}
```

- **Pointers:** The LinkedList class holds references to first and last nodes.
- **Traversal:**
  - To get index i, it checks if i < (size >> 1).
  - If yes, traverses from first.
  - If no, traverses backwards from last.
- **Memory Overhead:** Each element consumes memory for the Node object + 2 references (prev, next) + data.
- **Cache Locality:** Poor. Nodes are scattered in heap memory, causing CPU cache misses compared to ArrayList.

2.6 Where to use (Real-World)

- Implementing a Undo/Redo feature (using addFirst/removeFirst).
- Task scheduling where tasks are added/removed from ends frequently.
- LRU Cache implementation (combined with HashMap).

3. Vector

3.1 What is it?

- **Definition:** A legacy synchronized dynamic array.
- **Package:** java.util
- **Characteristics:** Ordered, allows duplicates, **Synchronized** (Thread-safe), legacy.

3.2 Why we use?

- **Legacy Support:** Only used to maintain old code written before Java 2 (1998).
- **Thread Safety:** Provides built-in synchronization (but coarse-grained).

3.3 When to use?

- ❌ **New Code: Do not use** in new development.
- ✅ **Legacy Maintenance:** Only when working with old codebases that depend on it.
- **Alternative:** Use ArrayList + Collections.synchronizedList() or CopyOnWriteArrayList.

3.4 How to use?

```
import java.util.Vector;

public class Main {

  public static void main(String[] args) {

    Vector<String> v = new Vector<>();

    v.add("A"); // Thread-safe (synchronized)

  }

}
```

4. Stack

4.1 What is it?

- **Definition:** A legacy Last-In-First-Out (LIFO) stack of objects.
- **Package:** java.util
- **Characteristics:** Extends Vector, adds push, pop, peek.

4.2 Why we use?

- **Legacy:** Historically used for LIFO operations.
- **Why Avoid:** It extends Vector, inheriting unnecessary methods and synchronization overhead. It violates the "Interface over Implementation" principle.

4.3 When to use?

- ❌ **New Code: Do not use**.
- **Alternative:** Use ArrayDeque (e.g., Deque<String> stack = new ArrayDeque<>();).

4.4 How to use?

```
import java.util.Stack;

public class Main {

  public static void main(String[] args) {

    Stack<String> stack = new Stack<>();

    stack.push("A");   // Add

    stack.pop();      // Remove & Return

    stack.peek();     // Return top without removing

    stack.empty();    // Check if empty

  }

}
```

4.5 Internal Working

- **Inheritance:** Extends Vector.
- **Operations:**
  - push calls addElement (Vector method).
  - pop calls removeElementAt(size - 1).
- **Locking:** Inherits Vector's method-level synchronization.

4.6 Where to use (Real-World)

- Legacy code handling expression evaluation or backtracking algorithms.
- **Modern Replacement:** ArrayDeque is faster and non-synchronized.

5. AbstractList

5.1 What is it?

Definition: An abstract skeletal implementation of the List interface.

Package: java.util

Characteristics: Cannot be instantiated. Designed for inheritance.

5.2 Why we use?

Custom Implementation: To minimize effort when creating a custom List backed by random-access data (like an array).

Boilerplate Reduction: Provides default implementations for add, remove, addAll, etc., based on get and size.

5.3 When to use?

✅ Custom Lists: When you are building your own List implementation (e.g., a List backed by a database or a fixed-size buffer).

❌ General Usage: Never use directly in application code.

5.4 How to use?

Java

```
import java.util.AbstractList;

public class ReadOnlyList extends AbstractList<String> {

  private final String[] data;

  public ReadOnlyList(String[] data) { this.data = data; }

  @Override

  public String get(int index) { return data[index]; }

  @Override

  public int size() { return data.length; }

  // add/remove throw UnsupportedOperationException by default

}
```

5.5 Internal Working

- **Random Access Optimized:** Assumes get(int) is fast (O(1)).
- **Default Methods:** Implements add(int, E) by shifting elements (which might be inefficient for non-array backs).
- **Modification:** Tracks modCount to support fail-fast iterators.

5.6 Where to use (Real-World)

- Creating a view over an existing array without copying data.
- Implementing a list adapter for custom data structures.

---

6. AbstractSequentialList

6.1 What is it?

- **Definition:** An abstract skeletal implementation of the List interface for sequential access.
- **Package:** java.util
- **Characteristics:** Cannot be instantiated. Designed for inheritance.

6.2 Why we use?

- **Custom Linked Lists:** To minimize effort when creating a custom List backed by sequential-access data (like a linked chain).
- **Iterator Based:** Relies on ListIterator for most operations instead of index-based access.

6.3 When to use?

- ✅ **Custom Linked Structures:** When building a custom list where get(int) is O(n).
- ❌ **General Usage:** Never use directly in application code.

6.4 How to use?

java

```
import java.util.AbstractSequentialList;

import java.util.ListIterator;

public class MyLinkedList extends AbstractSequentialList<String> {

  // Must implement listIterator() and size()

  public ListIterator<String> listIterator(int index) { ... }

  public int size() { ... }

}
```

6.5 Internal Working

- **Sequential Access Optimized:** Assumes get(int) is slow.
- **Iterator Reliance:** Implements get(int), set(int, E), add(int, E) by obtaining a ListIterator and traversing to the index.
- **Performance:** Prevents inefficient random-access implementations from pretending to be fast.

6.6 Where to use (Real-World)

- LinkedList class extends AbstractSequentialList.
- Creating custom linked structures where traversal is the primary operation.

Comparison & Differences

Feature	ArrayList	LinkedList	Vector	Stack
Underlying Structure	Dynamic Array	Doubly Linked List	Dynamic Array	Dynamic Array
Random Access (get)	**O(1)** (Fast)	O(n) (Slow)	**O(1)** (Fast)	O(1)
Insert/Delete (End)	O(1)*	O(1)	O(1)*	O(1)
Insert/Delete (Middle)	O(n) (Shift)	O(1)** (Navigate)	O(n) (Shift)	N/A
Memory Overhead	Low (Data only)	High (Nodes + Pointers)	Low	Low
Thread Safety	No	No	**Yes** (Sync)	**Yes** (Sync)
Growth Factor	1.5x	N/A	2x (Default)	2x
Legacy?	No	No	**Yes**	**Yes**
Recommended?	**Yes**	**Yes** (Specific cases)	**No**	**No**

**Amortized constant time (due to resizing). **O(1) if using Iterator at current position.*

Best Practices & Pitfalls

8.1 Initialize Capacity

- **Pitfall:** Letting ArrayList resize repeatedly causes CPU overhead.
- **Fix:** new ArrayList<>(expectedSize).

8.2 Trim Excess Capacity

- **Pitfall:** After adding many elements and removing some, the backing array might be large.
- **Fix:** list.trimToSize() reduces capacity to current size.

8.3 Prefer Interface

- **Pitfall:** ArrayList<String> list = new ArrayList<>();
- **Fix:** List<String> list = new ArrayList<>(); (Allows swapping implementation later).

8.4 Stack Replacement

- **Pitfall:** Using Stack class.
- **Fix:** Use Deque<String> stack = new ArrayDeque<>(); (push, pop, peek).

8.5 SubList Memory Leak

- **Pitfall:** List sub = list.subList(0, 10); keeps reference to original backing array. If original is huge, GC cannot collect it.
- **Fix:** new ArrayList<>(list.subList(0, 10)) to create a copy.

Head-to-Head Comparison

Feature	ArrayList	LinkedList
Underlying Structure	Dynamic Resizable Array (`Object[]`)	Doubly Linked List (`Node`)
Memory Location	**Contiguous** (Next to each other in RAM)	**Scattered** (Nodes anywhere in Heap)
Get (Random Access)	**O(1)** (Direct index calculation)	**O(n)** (Traversal from head/tail)
Add (End)	**O(1)** (Amortized)	**O(1)** (Update pointers)
Add (Middle)	**O(n)** (Shift elements)	**O(n)** (Traverse + Update pointers)
Remove (Middle)	**O(n)** (Shift elements)	**O(n)** (Traverse + Update pointers)
Memory Overhead	**Low** (Only data + capacity waste)	**High** (Data + `prev` + `next` refs per node)
CPU Cache Locality	**High** (Prefetching works well)	**Low** (Cache misses frequent)
Resizing	Yes (Costly copy operation)	No (Dynamic by nature)
Implements	`List`, `RandomAccess`, `Cloneable`, `Serializable`	`List`, `Deque`, `Cloneable`, `Serializable`
Null Values	Allowed	Allowed

2. Internal Working Differences

2.1 Memory Layout

- **ArrayList:**
  - Stores elements in a continuous block of memory.
  - **Formula:** Memory Address = Base Address + (Index * Data Size).
  - **Benefit:** CPU can prefetch adjacent data into cache lines. Iteration is extremely fast.
- **LinkedList:**
  - Stores elements in discrete Node objects scattered across the Heap.
  - **Structure:** [Prev Ref] [Data] [Next Ref].
  - **Drawback:** To access the next element, the CPU must follow the reference, often resulting in a **Cache Miss**.

2.2 Insertion Mechanicsm

- **ArrayList (Middle Insertion):**
  1. Check capacity (resize if needed).
  2. Calculate number of elements to shift (numMoved = size - index).
  3. Call System.arraycopy() (Native C++ method, highly optimized) to shift elements right.
  4. Place new element.
- **LinkedList (Middle Insertion):**
  1. Traverse from head or tail to find the node at index (O(n)).
  2. Create new Node.
  3. Update prev.next and next.prev pointers (O(1)).
  - **Nuance:** If you already have the **Iterator** position, LinkedList insertion is O(1). If you have an **Index**, LinkedList is O(n) due to traversal, making ArrayList potentially faster for small-to-medium lists due to arraycopy optimization.

2.3 Resizing

- **ArrayList:** When size == capacity, a new array is allocated (1.5x size), and all data is copied. This is a heavy operation.
- **LinkedList:** No resizing. Every add allocates a new Node object on the Heap. Heavy on Garbage Collector (GC) pressure if adding millions of items.

Set Implementations

1. Why use Set when we already have List?

Before diving into implementations, we must answer the fundamental question: **Why do we need Sets if we have List.**

What is Set?

1.1 Definition

- Interface: public interface Set<E> extends Collection<E>
- Package: java.util
- Core Contract: A collection that cannot contain duplicate elements.

1.2 The Mathematical Model

Set in Java models the mathematical set abstraction:

- A collection of distinct objects.
- No concept of position/index (unlike List).
- Supports mathematical operations: Union, Intersection, Difference.

2. Why Do We Use Set? (The Fundamental Question)

2.1 Problem: Lists Allow Duplicates

2.2 Solution: Set Enforces Uniqueness Automatically

Feature	List	Set	Why Switch to Set?
Duplicates	**Allowed**	**Not Allowed**	**Data Integrity:** Ensures uniqueness (e.g., User IDs, Email addresses) without manual checks.
Performance (contains)	**O(n)** (Linear Scan)	**O(1)** or **O(log n)**	**Lookup Speed:** Checking if an element exists is vastly faster in a Set.
Ordering	Insertion Order (Guaranteed)	Varies (None, Insertion, Sorted)	**Flexibility:** Sets offer specialized ordering (Sorted) that Lists don't enforce natively.
Mathematical Ops	Manual Implementation	Built-in (addAll, retainAll)	**Set Algebra:** Easy union, intersection, and difference operations.
Nulls	Multiple Nulls Allowed	Single Null (HashSet)	**Constraint:** Sets enforce stricter null policies (TreeSet allows none).

Summary: Use a **List** when order and duplicates matter. Use a **Set** when uniqueness and lookup speed matter.

Set Interface: Core Methods & Contracts

4.1 Inherited from Collection

```
boolean add(E e);         // Returns false if duplicate
boolean remove(Object o);   // Returns true if removed
boolean contains(Object o); // Checks existence
int size();             // Number of elements
boolean isEmpty();        // Checks if empty
void clear();           // Removes all elements
Iterator<E> iterator();    // For iteration
```

. Internal Method Contracts

The Set interface inherits methods from Collection but adds specific behavioral constraints.

4.1 add(E e)

- **Contract:** Adds the element only if not present.
- **Return:** true if added, false if duplicate.
- **Internal Constraint:** Implementations must check existence *before* adding.

4.2 remove(Object o)

- **Contract:** Removes the element if present.
- **Return:** true if removed, false if not found.
- **Internal Constraint:** Must rely on equals() to find the element (no index available).

4.3 iterator()

- **Contract:** Returns an iterator over elements.
- **Internal Constraint:**
    - For Set (general): Order is not guaranteed.
    - For SortedSet: Order must be ascending.
    - **Fail-Fast:** The interface documentation recommends (but doesn't enforce) fail-fast iterators to detect concurrent modification.

4.4 No get(int index)

- **Design Decision:** The Set interface **does not** define get(int index).

- **Internal Reason:** Sets are not ordered by position (unless SortedSet). Positional access contradicts the mathematical definition of a Set.
- **Consequence:** You cannot access elements by index via the Set interface.

2. HashSet

2.1 What is it?

- Definition: The standard implementation of the Set interface.
- Package: java.util
- Characteristics: Unordered, allows one null, not synchronized.

2.2 Why we use? (Purpose)

- Baseline: This is the default choice for a Set.
- Performance: Provides the best time complexity for basic operations (add, remove, contains).
- Uniqueness: Enforces uniqueness without caring about order.

2.3 Internal Working

- Backing Structure: Backed by a HashMap<k,v>.
- **Relationship:** `HashSet` is a thin wrapper over a `HashMap`.
- Storage:
  - The element you add is stored as the Key in the internal HashMap.
  - A static dummy object private static final Object PRESENT = new Object(); is stored as the Value.
- Uniqueness: Guaranteed by the HashMap key uniqueness contract.
- Hashing: Uses hashCode() to determine the bucket. Collisions are handled via Linked List (or Tree if > 8 nodes).
- Nulls: Allows one null key (stored at a specific index in the HashMap).

```
// Inside HashSet.java

private transient HashMap<E, Object> map;

private static final Object PRESENT = new Object();

public boolean add(E e) {

   return map.put(e, PRESENT) == null;

   // put returns null if key was new (added successfully)

   // put returns old value if key existed (duplicate)  }
```

1.2 Insertion Logic (Step-by-Step)

1. **Hash Calculation:** Calls e.hashCode().
2. **Bucket Index:** Calculates index using (n - 1) & hash (where n is table size).
3. **Collision Check:**
    - **Empty Bucket:** Creates a new Node and places it in the array.
    - **Occupied Bucket:**
        - Checks hashCode() and equals().
        - If equal → **Duplicate** → Returns false.
        - If not equal → **Collision** → Adds to Linked List (or Tree).
4. **Treeify (Java 8+):** If a bucket's Linked List exceeds **8 nodes**, it converts to a **Red-Black Tree** for O(log n) lookup within that bucket.

1.3 Memory Layout

- **Array:** HashMap maintains an array of Node<K,V>.

```
static class Node<K,V> implements Map.Entry<K,V> {
  final int hash;
  final K key;   // <-- Your Set Element
  V value;      // <-- PRESENT object
  Node<K,V> next; // <-- Collision chain
}
```

1.4 Null Handling

- **Allowed:** Yes (One null).
- **Internal:** null key is always stored at index 0 of the HashMap table. It bypasses hashCode() calculation.

1.5 Performance

- **Add/Remove/Contains: O(1)** (Average case).
- **Worst Case: O(log n)** (if many collisions trigger Treeify) or **O(n)** (if all elements collide in one bucket and Java 7 logic applies).

2.4 How to use?

```
Set<String> set = new HashSet<>();

set.add("A");      // Returns true

set.add("A");      // Returns false (Duplicate)

set.contains("A");  // O(1) - Very fast
```

. LinkedHashSet

3.1 What is it?

- **Definition:** A HashSet with predictable iteration order.
- **Package:** java.util
- **Characteristics:** Insertion-ordered, allows one null, not synchronized.

3.2 Why we use? (Purpose over HashSet)

- **Problem with HashSet:** Iteration order is unpredictable (depends on hash codes).
- **Solution:** LinkedHashSet maintains the **order in which elements were inserted**.
- **Use Case:** When you need uniqueness + order (e.g., caching, storing history without duplicates).

3.3 Internal Working

- **Backing Structure:** Backed by a **LinkedHashMap**.
- **Structure:** HashMap buckets + **Doubly Linked List** running through all entries.
- **Ordering:** The linked list maintains the insertion order. When iterating, it follows the list, not the buckets.
- **Overhead:** Slightly higher memory than HashSet due to linked list pointers (before, after in Entry class).

  Internal Structure (The Linked List)

- **Enhanced Node:** The internal Entry class extends HashMap.Node and adds two pointers.

  ```
  static class Entry<K,V> extends HashMap.Node<K,V> {

     Entry<K,V> before, after; // <-- Doubly Linked List pointers

  }
  ```

- **Head & Tail:** LinkedHashMap maintains head and tail references to the first and last entries.

  2.3 Insertion Logic

1. **HashMap Logic:** Performs standard HashMap insertion (hashing, bucket placement).

2. **Linked List Logic:**
    - Creates a new Entry.
    - Inserts it at the **end** of the doubly-linked list (tail).
    - Updates tail reference.
3. **Result:** The HashMap buckets handle uniqueness/speed; the Linked List handles iteration order.

2.4 Iteration Logic

- **HashSet:** Iterates over the HashMap bucket array (unordered).
- **LinkedHashSet:** Iterates over the **doubly-linked list** (head → tail).
- **Performance:** Iteration time is proportional to **capacity + size** for HashSet, but just **size** for LinkedHashSet (no empty buckets to skip).

2.5 Memory Overhead

- **Higher than HashSet:** Each entry stores two extra references (before, after).
- **Cost:** Approx. 24-32 bytes extra per element on a 64-bit JVM.

2.4 How to use?

```
Set<String> set = new LinkedHashSet<>();

set.add("C");

set.add("A");

set.add("B");

// Iteration: C, A, B (Insertion Order)
```

3.5 Methods

- Inherits all HashSet methods.
- No additional unique methods, but iteration behavior is different.

---

4. TreeSet

4.1 What is it?

- **Definition:** A Sorted Set implemented with a Tree.
- **Package:** java.util
- **Characteristics: Sorted** (Natural or Comparator), No null, not synchronized.

4.2 Why we use? (Purpose over LinkedHashSet)

- **Problem with LinkedHashSet:** Only maintains insertion order, not sorted order.
- **Solution:** TreeSet keeps elements **sorted** automatically.
- **Use Case:** Leaderboards, range queries (find all IDs between 100 and 200), sorted unique data.

4.3 Internal Working

- **Backing Structure:** Backed by a **TreeMap** (Red-Black Tree).
- **Balancing:** Self-balancing binary search tree. Ensures height remains logarithmic.
- **Comparison:** Elements must implement Comparable or a Comparator must be provided.
- **Nulls: Does NOT allow null**. (Cannot compare null with objects).
- **Performance:** O(log n) for add/remove/contains (slower than HashSet).

  Backing Structure: TreeMap<K, V>

  Relationship: Implements NavigableSet. Backed by a TreeMap.

  Storage: Element = Key, Value = Dummy PRESENT object (similar to HashSet).

  3.2 Internal Structure (Red-Black Tree)

```
static final class Entry<K,V> implements Map.Entry<K,V> {
   K key;
   Entry<K,V> left, right, parent;
   boolean color; // RED or BLACK
}
```

  1. Root is always BLACK.
  2. No two RED nodes can be adjacent.
  3. Every path from a node to its descendant leaves has the same number of BLACK nodes.

- **Balancing:** Ensures tree height remains **O(log n)**.

3.3 Insertion Logic

1. **Comparison:** Compares new element with root using Comparable or Comparator.
    - If < → Go Left.
    - If > → Go Right.
    - If == → **Duplicate** → Return false.
2. **Insert:** Place new node as RED leaf.
3. **Rebalance:** Perform **Rotations** and **Color Flips** to maintain Red-Black properties.
    - *Left Rotate*, *Right Rotate*, *Recolor*.

3.4 Null Handling

- **Allowed: NO**.
- **Reason:** null cannot be compared using compareTo() or Comparator. Throws ClassCastException.
    - *Exception:* If TreeSet contains only one element null and no comparator is used (rare edge case in older Java versions), but generally prohibited.

3.5 Performance

- **Add/Remove/Contains: O(log n)**.
- **Iteration:** O(n) (In-order traversal).
- **Navigation:** floor, ceiling, higher, lower are **O(log n)**.

4.4 How to use?

```
// Natural Ordering

Set<Integer> set = new TreeSet<>();

set.add(5); set.add(1); set.add(9);

// Iteration: 1, 5, 9


// Custom Ordering

Set<String> set = new TreeSet<>((a, b) -> b.compareTo(a)); // Descending
```

Methods (NavigableSet)

- first(): Returns the lowest element.
- last(): Returns the highest element.
- floor(E e): Returns greatest element ≤ e.
- ceiling(E e): Returns least element ≥ e.
- subSet(from, to): Returns view of portion.

---

5. EnumSet

5.1 What is it?

- **Definition:** A specialized Set implementation for use with **Enum** types.
- **Package:** java.util
- **Characteristics:** Extremely fast, compact, ordered (ordinal order), no null.

5.2 Why we use? (Purpose over TreeSet/HashSet)

- **Problem:** Generic Sets (HashSet) have overhead (Objects, Hashing, Nodes).
- **Solution:** EnumSet uses a **Bit Vector**.
- **Use Case:** Storing flags, options, or states defined by an Enum.

5.3 Internal Working

- **Structure:** Backed by a **long array** (Bit Vector).
- **Storage:** Each enum constant has an ordinal() (0, 1, 2...). The bit at that position is set to 1 if present.
    - Example: If Enum has 5 values, a single long (64 bits) can store the entire set.
- **Performance:** Operations are bitwise operations (AND, OR, XOR). Extremely fast (near array speed).
- **Memory:** Minimal. No Node objects created.

  Core Architecture

    - **Backing Structure: Bit Vector** (Array of long).
    - **Abstract Class:** EnumSet is abstract. Factory methods return specific subclasses.
    - **Subclasses:**
        - RegularEnumSet: Uses a single long (for enums ≤ 64 elements).
        - JumboEnumSet: Uses long[] array (for enums > 64 elements).

4.2 Internal Structure

- **Mapping:** Each enum constant has an ordinal() (0, 1, 2...).

```
// Conceptual Internal State
long elements = 0;
// Add MON (ordinal 0)
elements |= (1L << 0); // 0001
// Add WED (ordinal 2)
elements |= (1L << 2); // 0101
```

4.3 Insertion Logic

1. **Check Type:** Ensures element is of the correct Enum type.
2. **Bitwise Operation:** Performs elements |= (1L << e.ordinal()).
3. **No Objects:** No Node objects are created. No hashing. No tree balancing.

4.4 Performance

- **Add/Remove/Contains: O(1)** (Bitwise CPU instructions).
- **Memory:** Extremely compact. A set of 64 enums takes **8 bytes** (one long).
- **Iteration:** Uses Long.numberOfTrailingZeros() to find set bits efficiently.

4.5 Limitations

- **Type Safety:** Can only store one specific Enum type.
- **Nulls:** Not allowed (cannot compute ordinal of null).

5.4 How to use?

```
java
enum Day { MON, TUE, WED }
Set<Day> set = EnumSet.of(Day.MON, Day.TUE);
// Cannot create via new EnumSet(), must use factory methods
```

5.5 Methods

- of(E... elements): Create set with specific elements.
- allOf(Class<E> elementType): Create set with all enum values.
- noneOf(Class<E> elementType): Create empty set.

---

6. SortedSet Interface

6.1 What is it?

- **Definition:** An interface that extends Set to provide total ordering on its elements.
- **Package:** java.util
- **Characteristics:** Interface (cannot instantiate).

6.2 Why we use?

- **Contract:** Defines the contract for sorted sets (like TreeSet).
- **Polymorphism:** Allows you to write code that accepts *any* sorted set, not just TreeSet.

6.3 Methods

- Comparator<?> comparator(): Returns the comparator used for ordering.
- first(), last(): Access endpoints.
- headSet(E toElement), tailSet(E fromElement): Views.

---

7. NavigableSet Interface

7.1 What is it?

- **Definition:** Extends SortedSet with navigation methods.
- **Package:** java.util
- **Characteristics:** Interface (implemented by TreeSet, ConcurrentSkipListSet).

7.2 Why we use? (Purpose over SortedSet)

- **Problem:** SortedSet only gives headSet/tailSet.
- **Solution:** NavigableSet adds methods to find closest matches.
- **Use Case:** Finding the next available slot, rounding values, range searches.

7.3 Methods

- lower(E e): Strictly less than e.
- floor(E e): Less than or equal to e.
- ceiling(E e): Greater than or equal to e.
- higher(E e): Strictly greater than e.
- pollFirst(), pollLast(): Retrieve and remove.

---

8. ConcurrentSkipListSet

8.1 What is it?

- **Definition:** A thread-safe sorted set.
- **Package:** java.util.concurrent
- **Characteristics:** Sorted, Thread-Safe, No null.

8.2 Why we use? (Purpose over TreeSet)

- **Problem:** TreeSet is **not** thread-safe. Wrapping it with Collections.synchronizedSet locks the whole set (slow).
- **Solution:** ConcurrentSkipListSet allows concurrent access without global locking.
- **Use Case:** Multi-threaded environments requiring sorted unique data.

8.3 Internal Working

- **Structure: Skip List** (Probabilistic data structure).
- **Layers:** Multiple layers of linked lists. Top layers skip many elements for fast search.
- **Concurrency:** Lock-free read operations. Fine-grained locking for writes.
- **Performance:** O(log n) expected time complexity. Slightly slower than TreeSet in single-threaded but scales better in multi-threaded.

Core Architecture

- **Backing Structure:** ConcurrentSkipListMap.
- **Data Structure: Skip List** (Probabilistic alternative to Balanced Trees).
- **Purpose:** Thread-safe sorted set without global locking.

5.2 Internal Structure (Skip List)

- **Layers:** Multiple linked lists layered on top of each other.
    - **Bottom Layer:** Contains all elements.
    - **Upper Layers:** Contain subsets of elements (express lanes).

```
static class Node<K,V> {
  K key;
  volatile Object value;
  volatile Node<K,V> next; // Horizontal link
  volatile Node<K,V> down; // Vertical link to lower layer
}
```

5.3 Search Logic

1. Start at the **top-left** header.
2. Move **Right** while next key is less than target.
3. Move **Down** to the next layer.
4. Repeat until bottom layer is reached.
5. **Complexity:** O(log n) expected.

5.4 Concurrency Mechanism

- **Lock-Free Reads:** Traversal uses volatile reads. No locks needed for contains or iteration.
- **CAS Writes:** Insertions use **Compare-And-Swap (CAS)** to link new nodes atomically.
- **Lazy Removal:** Deleted nodes are marked logically deleted first, then physically removed later by garbage collection or subsequent operations.

5.5 Performance vs. TreeSet

- **Single Thread:** TreeSet is usually faster (lower constant overhead).
- **Multi-Thread:** ConcurrentSkipListSet scales better (no contention on a single root node like Tree).

8.4 How to use?

```
Set<Integer> set = new ConcurrentSkipListSet<>();

// Safe to add/remove from multiple threads
```

implementation	Order	Nulls Allowed	Thread Safe	Performance (Add/Contains)	Internal Structure
HashSet	None	One	No	**O(1)**	HashMap (Array + List/Tree)
LinkedHashSet	Insertion	One	No	O(1)	LinkedHashMap + Linked List
TreeSet	**Sorted**	**No**	No	O(log n)	Red-Black Tree
EnumSet	Ordinal	**No**	No	**O(1)** (Bitwise)	Bit Vector (long[])
ConcurrentSkipListSet	Sorted	**No**	**Yes**	O(log n)	Skip List

Set Algebra Methods

Method	Description	Example
addAll(Collection c)	**Union**: Adds all elements from c.	set1.addAll(set2)
retainAll(Collection c)	**Intersection**: Keeps only elements in c.	set1.retainAll(set2)
removeAll(Collection c)	**Difference**: Removes elements in c.	set1.removeAll(set2)

Common Methods Across Sets

Method	Description	Returns
add(E e)	Adds element if absent.	boolean (true if added)
remove(Object o)	Removes element if present.	boolean (true if removed)
contains(Object o)	Checks if element exists.	boolean
size()	Number of elements.	int
isEmpty()	Checks if empty.	boolean
clear()	Removes all elements.	void
iterator()	Returns iterator.	Iterator<E>
toArray()	Converts to array.	Object[]

.1 Hashing (HashSet)

- **Hash Function:** h = key.hashCode() ^ (h >>> 16). (XORs high bits with low bits to reduce collisions).
- **Index Calculation:** index = (table.length - 1) & hash. (Bitwise AND is faster than %).
- **Collision Resolution:**
  - **Java 7:** Linked List only.
  - **Java 8+:** Linked List → **Red-Black Tree** (if size > 8 and capacity > 64).

7.2 Red-Black Balancing (TreeSet)

- **Why?** Prevents the tree from becoming a linked list (skewed tree) which would make operations O(n).
- **How?** Rotations.

    - **Left Rotation:** Pivot right child up.
    - **Right Rotation:** Pivot left child up.
- **Cost:** Extra CPU cycles during insert/delete to maintain balance.

7.3 Bit Vector (EnumSet)

- **Efficiency:** Uses CPU registers for operations.
- **Set Operations:**
    - **Union:** bits1 | bits2
    - **Intersection:** bits1 & bits2
    - **Difference:** bits1 & ~bits2
- **Speed:** Extremely fast compared to object-based sets.

7.4 Skip List Probabilistic Balancing (ConcurrentSkipListSet)

- **Random Level:** When inserting, a random level is generated (like flipping a coin).
    - 50% chance of level 1.
    - 25% chance of level 2.
    - 12.5% chance of level 3.
- **Benefit:** Avoids complex rebalancing logic (like Red-Black trees), making it easier to implement concurrently.

Decision flow chart :

```
Do you need to store unique elements?
│
├─ No → Use List
│
└─ Yes → Do you need sorted order?
   │
   ├─ Yes → Use TreeSet
   └─ No → Do you need insertion order?
      ├─ Yes → Use LinkedHashSet
      │
      └─ No → Use HashSet (default choice)
```

Best Practices & Pitfalls

1. **Null Safety:**
    - HashSet/LinkedHashSet: Allow one null.
    - TreeSet/EnumSet/ConcurrentSkipListSet: **Throw Exception** on null.
2. **Mutable Keys:**
    - Do not store mutable objects in HashSet if their fields affect hashCode(). Changing the object after insertion makes it unretrievable.
3. **Initial Capacity:**
    - HashSet allows setting initial capacity and load factor. new HashSet<>(100, 0.75f) avoids resizing.
4. **Iteration:**
    - HashSet: Order is unpredictable. Do not rely on it.
    - LinkedHashSet: Order is predictable (insertion).
    - TreeSet: Order is sorted.
5. **Choosing the Right Set:**
    - Default: HashSet.
    - Need Order: LinkedHashSet.
    - Need Sorting: TreeSet.
    - Need Enums: EnumSet.
    - Need Concurrent + Sorted: ConcurrentSkipListSet.
6. Set hirerachy :

```
7. java.util.Collection
8.   └── java.util.Set
9.       ├── java.util.HashSet        (Hash table, no order)
10.      ├── java.util.LinkedHashSet   (Hash table + linked list, insertion order)
11.      ├── java.util.TreeSet        (Red-Black tree, sorted order)
12.      └── java.util.EnumSet        (Bit vector, for enums only)
13.
14. java.util.SortedSet (interface)
15.   └── java.util.NavigableSet (interface)
16.      ├── java.util.TreeSet
17.      └── java.util.concurrent.ConcurrentSkipListSet
```

Queue Implementation

1. Why Use Queue When We Already Have List and Set?

Before diving into implementations, we must answer the fundamental architectural question: **Why do we need a Queue interface if we have ArrayList or LinkedList?**

1.1 Semantic Difference (Intent)

- **List:** "I want to **store** a collection of items and access them by position or iterate over them."
- **Set:** "I want to **store** unique items."
- **Queue:** "I want to **process** items in a specific order."
    - Queues imply **workflow**. Elements are typically added at one end and removed from the other as they are processed.
    - Lists/Sets imply **storage**. Elements stay until explicitly removed.

1.2 Method Contract Difference

Queues provide specialized methods that handle failure differently than Lists.

Operation	List (ArrayList)	Queue (LinkedList/ArrayDeque)	Why Queue is Better
Insert	add(e) → Throws exception if fails	offer(e) → Returns false if fails	**Safety:** offer allows graceful handling of capacity limits.
Remove	remove() → Removes by index/object	poll() → Returns null if empty	**Safety:** poll avoids NoSuchElementException on empty collection.
Inspect	get(0) → Throws exception if empty	peek() → Returns null if empty	**Safety:** peek allows checking head without risk.

1.3 Ordering Enforcement

- **List:** You *can* use it as FIFO (remove index 0), but it's not enforced. You might accidentally remove from the end.
- **Queue:** Enforces **FIFO** (First-In-First-Out), **LIFO** (Stack), or **Priority** ordering by design.

1.4 Concurrency & Blocking

- **List:** No built-in support for waiting when empty or full.
- **Queue:** BlockingQueue allows threads to **wait** (block) until an element is available or space opens up. This is critical for Producer-Consumer patterns.

2. Decision Flowchart: Which Queue to Use?

```
Start: Do you need a Queue?
│
├─ Need Priority Ordering (not FIFO)?
│  └─ YES → Use PriorityQueue
│
└─ Need FIFO / LIFO?
   │
   ├─ Multi-threaded Environment?
   │  │
   │  ├─ Need Threads to WAIT (Block) when empty/full?
   │  │  └─ YES → Use BlockingQueue (e.g., ArrayBlockingQueue)
   │  │
   │  └─ NO (Non-blocking, high throughput)?
   │     └─ YES → Use ConcurrentLinkedQueue
   │
   └─ Single-threaded (or external sync)?
      │
      ├─ Need Stack (LIFO) + Queue (FIFO)?
      │  └─ YES → Use ArrayDeque (Preferred)
      │
      └─ Need Null elements?
         ├─ YES → Use LinkedList (ArrayDeque forbids nulls)
         └─ NO → Use ArrayDeque (Faster)
```

3. PriorityQueue

3.1 What is it?

- **Definition:** An unbounded queue based on a priority heap.
- **Package:** java.util
- **Ordering:** Elements are processed based on **priority** (Natural order or Comparator), not FIFO.
- **Nulls: Not allowed**.

3.2 Why we use? (Purpose)

- **Problem:** FIFO is not always fair. Some tasks are more urgent.
- **Solution:** PriorityQueue ensures the "most important" element is always at the head.
- **Use Case:** Task scheduling, Dijkstra's algorithm, Huffman coding, merging sorted streams.

3.3 When to use?

- ✅ **Priority Processing:** When order depends on value, not arrival time.
- ✅ **Finding Top K:** Efficiently keeping track of the largest/smallest K elements.
- ❌ **FIFO Needed:** Do not use if arrival order matters.
- ❌ **Nulls:** Do not use if you need to store null.

3.4 Internal Working

- **Structure: Binary Heap** (stored in an Object array).
- **Logic:**
  - Parent at index i.
  - Left Child at 2i + 1.
  - Right Child at 2i + 2.
- **Heap Property:** Parent is always smaller (Min-Heap) or larger (Max-Heap) than children.
- **Insertion (offer):**
  1. Add element at the end of the array.
  2. **Bubble Up:** Swap with parent until heap property is restored. O(log n).
- **Removal (poll):**
  1. Remove root (head).
  2. Move last element to root.
  3. **Bubble Down:** Swap with smaller child until heap property is restored. O(log n).

- **Resizing:** Similar to ArrayList (grows automatically).

3.5 Methods

- offer(E e): Inserts element.
- poll(): Retrieves and removes the head (highest priority).
- peek(): Retrieves the head without removing.
- size(): Returns count.
- **Note:** Iteration (for-each) does **not** guarantee priority order. Only poll() guarantees order.

3.6 How to use?

```
// Min-Heap (Default)
Queue<Integer> pq = new PriorityQueue<>();
pq.offer(5); pq.offer(1); pq.offer(3);
pq.poll(); // Returns 1

// Max-Heap (Custom Comparator)
Queue<Integer> maxPq = new PriorityQueue<>((a, b) -> b - a);
pq.offer(5); pq.offer(1);
pq.poll(); // Returns 5
```

4. ArrayDeque

4.1 What is it?

- **Definition:** A resizable-array implementation of the Deque (Double Ended Queue) interface.
- **Package:** java.util
- **Ordering:** FIFO (Queue) or LIFO (Stack).
- **Nulls: Not allowed**.

4.2 Why we use? (Purpose over LinkedList)

- **Problem:** LinkedList implements Queue and Deque, but it has high memory overhead (Node objects) and poor cache locality.
- **Solution:** ArrayDeque uses a circular buffer (array).
- **Performance:** Faster than LinkedList for stack/queue operations.
- **Use Case:** Standard Stack or Queue in single-threaded or externally synchronized contexts.

4.3 When to use?

- ✅ **Stack:** Preferred over Stack class.
- ✅ **Queue:** Preferred over LinkedList for FIFO.
- ✅ **Performance:** When memory and CPU cache matter.
- ❌ **Nulls:** Cannot store null.
- ❌ **Thread Safety:** Not thread-safe (use ConcurrentLinkedDeque or wrap).

4.4 Internal Working

- **Structure: Circular Buffer** (Array).
- **Pointers:** head and tail indices.
- **Wrapping:** When tail reaches the end of the array, it wraps around to index 0 (if space exists).
- **Resizing:** When full, creates a new array (2x size) and copies elements linearly (unwrapping the circle).
- **Memory:** Contiguous array. No Node objects.
- **Null Check:** Explicitly checks for null on insert and throws NullPointerException.

4.5 Methods

- **Queue Methods:** offerFirst, offerLast, pollFirst, pollLast.
- **Stack Methods:** push (same as offerFirst), pop (same as pollFirst), peek.
- **Access:** getFirst, getLast.

4.6 How to use?

```
// As Queue (FIFO)
Deque<String> queue = new ArrayDeque<>();
queue.offer("A"); queue.offer("B");
queue.poll(); // Returns "A"

// As Stack (LIFO)
Deque<String> stack = new ArrayDeque<>();
stack.push("A"); stack.push("B");
stack.pop(); // Returns "B"
```

5. BlockingQueue (Interface & Implementations)

5.1 What is it?

- **Definition:** An interface that supports operations that **wait** for the queue to become non-empty when retrieving an element, and wait for space to become available when storing an element.
- **Package:** java.util.concurrent
- **Implementations:** ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue.
- **Nulls: Not allowed**.

5.2 Why we use? (Purpose over ArrayDeque)

- **Problem:** In multi-threading, if a consumer thread polls an empty queue, it wastes CPU cycles (busy-waiting).
- **Solution:** BlockingQueue makes the thread **sleep** (block) until data is available.
- **Use Case: Producer-Consumer Pattern**. Thread-safe handoff without manual wait()/notify().

5.3 When to use?

- ✅ **Producer-Consumer:** One thread produces data, another consumes.
- ✅ **Throttling:** Limit the rate of processing by capping queue size.
- ✅ **Thread Safety:** Built-in synchronization.
- ❌ **Non-Blocking:** If you want immediate failure instead of waiting.

5.4 Internal Working

- **Locking:** Uses a single ReentrantLock (or separate locks for put/take in LinkedBlockingQueue).
- **Conditions:**
  - notEmpty: Consumers wait here if queue is empty.
  - notFull: Producers wait here if queue is full (bounded).
- **Signaling:** When an element is added, notEmpty.signal() wakes up a waiting consumer. When removed, notFull.signal() wakes up a waiting producer.
- **Example (ArrayBlockingQueue):** Backed by a fixed-size array + Lock + Conditions.

5.5 Methods

- put(E e): Inserts, **blocking** if full.
- take(): Retrieves, **blocking** if empty.
- offer(E e, long timeout, TimeUnit unit): Inserts with timeout.
- poll(long timeout, TimeUnit unit): Retrieves with timeout.

5.6 How to use?

```
BlockingQueue<String> queue = new ArrayBlockingQueue<>(10);

// Producer

queue.put("Item"); // Blocks if 10 items already exist

// Consumer

String item = queue.take(); // Blocks if queue is empty
```

6. ConcurrentLinkedQueue

6.1 What is it?

- **Definition:** An unbounded thread-safe queue based on linked nodes.
- **Package:** java.util.concurrent
- **Ordering:** FIFO.
- **Nulls: Not allowed**.

6.2 Why we use? (Purpose over BlockingQueue)

- **Problem:** BlockingQueue uses locks. If contention is high, threads spend time context-switching while waiting for locks.
- **Solution:** ConcurrentLinkedQueue is **lock-free**.
- **Performance:** Higher throughput under high contention if blocking is not required.
- **Use Case:** High-volume logging, passing tasks where dropping or immediate failure is acceptable.

6.3 When to use?

- ✅ **High Concurrency:** Many threads adding/removing simultaneously.
- ✅ **Non-Blocking:** You don't want threads to sleep/wait.
- ✅ **Unbounded:** You don't need to cap the size.
- ❌ **Blocking Needed:** If consumers must wait for data, use BlockingQueue.
- ❌ **Size Check:** size() is O(n) (not constant time) because it traverses nodes.

6.4 Internal Working

- **Structure:** Linked Nodes (similar to LinkedList).
- **Concurrency:** Uses **CAS (Compare-And-Swap)** algorithms (Michael-Scott queue algorithm).
- **Lock-Free:**
    - Enqueue: CAS to update tail pointer.

    - Dequeue: CAS to update head pointer.
- **Lazy Updating:** Head/Tail pointers are not always updated immediately to reduce contention (they are "eventually consistent").
- **Iterators: Weakly Consistent**. They do not throw ConcurrentModificationException but may not reflect recent changes.

6.5 Methods

- offer(E e): Non-blocking insert.
- poll(): Non-blocking remove.
- size(): **O(n)** (Avoid calling frequently in loops).

6.6 How to use?

java

```
Queue<String> queue = new ConcurrentLinkedQueue<>();

queue.offer("A"); // Thread-safe, non-blocking

String s = queue.poll(); // Returns null immediately if empty
```

7. AbstractQueue

7.1 What is it?

- **Definition:** An abstract skeletal implementation of the Queue interface.
- **Package:** java.util
- **Nature:** Cannot be instantiated. Designed for inheritance.

7.2 Why we use? (Purpose)

- **Problem:** Implementing Queue from scratch requires writing many boilerplate methods (addAll, removeAll, contains, etc.).
- **Solution:** Extend AbstractQueue.
- **Requirement:** You must implement offer, poll, peek, iterator, and size. The rest are provided.
- **Use Case:** Creating custom queue implementations (e.g., a queue backed by a database or priority logic not covered by PriorityQueue).

7.3 Internal Working

- **Delegation:** Methods like add(e) call offer(e). If offer returns false, add throws IllegalStateException.
- **Bulk Operations:** Provides default implementations for addAll, etc., using the iterator.
- **Removal:** remove() calls poll(). If poll returns null, remove throws NoSuchElementException.

7.4 How to use?

```
public class MyCustomQueue<E> extends AbstractQueue<E> {
  // Must implement these 5 methods
  public boolean offer(E e) { ... }
  public E poll() { ... }
  public E peek() { ... }
  public Iterator<E> iterator() { ... }
  public int size() { ... }
}
```

Comparison Table

Feature	PriorityQueue	ArrayDeque	BlockingQueue	ConcurrentLinkedQueue
Ordering	Priority (Heap)	FIFO / LIFO	FIFO (usually)	FIFO
Thread Safe	No	No	**Yes** (Locks)	**Yes** (CAS/Lock-Free)
Blocking	No	No	**Yes** (Wait on empty/full)	No
Nulls	No	No	No	No
Internal Structure	Binary Heap (Array)	Circular Buffer (Array)	Array / Linked Nodes	Linked Nodes
Performance	O(log n) insert/remove	O(1) insert/remove	O(1) (with lock overhead)	O(1) (with CAS overhead)
Best Use Case	Task Priority	Stack / Local Queue	Producer-Consumer	High Throughput Logging

9.1 PriorityQueue Iteration

- **Pitfall:** for (E e : priorityQueue) does not return elements in priority order.
- **Fix:** Use poll() repeatedly to retrieve in order.

9.2 ArrayDeque Nulls

- **Pitfall:** arrayDeque.add(null) throws NullPointerException.
- **Fix:** Use LinkedList if nulls are strictly required (though generally discouraged).

9.3 BlockingQueue Deadlocks

- **Pitfall:** Using a bounded BlockingQueue where producers wait for consumers, but consumers are waiting for producers to finish before consuming.
- **Fix:** Ensure consumers run in separate threads or use timeouts (offer with timeout).

9.4 ConcurrentLinkedQueue Size

- **Pitfall:** Calling size() in a loop to check progress.
- **Reason:** size() is O(n) because it traverses the linked list.
- **Fix:** Use isEmpty() which is O(1).

9.5 Stack Choice

- **Pitfall:** Using Stack class.
- **Fix:** Use Deque<String> stack = new ArrayDeque<>();. Stack is legacy and synchronized.

---

10. Summary Checklist

Component	Key Takeaway
Queue Interface	Defines offer, poll, peek. Semantic intent: Processing.
PriorityQueue	Heap-based. Priority order. Not thread-safe.
ArrayDeque	Circular buffer. Fastest FIFO/LIFO. No nulls. Not thread-safe.
BlockingQueue	Locks + Conditions. Blocks on empty/full. Producer-Consumer.
ConcurrentLinkedQueue	CAS-based. Lock-free. High throughput. size() is O(n).
AbstractQueue	Skeleton for custom implementations.
LinkedList	Implements Queue, but ArrayDeque is preferred for performance.
Stack	Legacy. Use ArrayDeque instead.

Summary of Examples

Example	Implementation	Key Method	Use Case
1. Ticket System	ArrayDeque	offer, poll	Standard FIFO processing
2. Undo System	ArrayDeque	push, pop	LIFO Stack behavior
3. Hospital Triage	PriorityQueue	offer, poll	Priority-based processing
4. Log Processor	ArrayBlockingQueue	put, take	Multi-threaded Producer-Consumer
5. Web Logger	ConcurrentLinkedQueue	offer, poll	High-concurrency, non-blocking
6. Safety Check	Queue vs List	poll vs remove	Safe empty-handling
7. Priority Order	PriorityQueue	poll vs foreach	Correct extraction order

Map Implementations Deep Dive

1. Why Use Map When We Already Have List, Set, and Queue?

Before diving into implementations, we must answer the fundamental question: **Why do we need Maps?**

1.1 The Fundamental Difference

Collection Type	Stores	Access Method	Use Case
List	Values only	Index (position)	Ordered sequence, duplicates allowed
Set	Values only	Value equality	Uniqueness, membership testing
Queue	Values only	FIFO/LIFO/Priority	Processing order, workflows
Map	**Key-Value Pairs**	**Key lookup**	**Associations, dictionaries, lookups**

1.2 The Problem Maps Solve

Scenario: Store student records with ID as identifier.

java

```
// ❌ With List - Inefficient
List<Student> students = new ArrayList<>();
// To find student with ID 123:
for (Student s : students) {        // O(n) - Linear scan
   if (s.getId() == 123) {
```

```
        return s;
    }
}

// ❌ With Set - Still inefficient
Set<Student> students = new HashSet<>();
// Still need to iterate or use contains() which is O(1) but doesn't return the object
// ✅ With Map - Direct lookup
Map<Integer, Student> students = new HashMap<>();
Student s = students.get(123);       // O(1) - Direct access!
```

1.3 Key Advantages of Map

1. **Fast Lookup:** O(1) access by key vs O(n) search in List
2. **Association:** Natural mapping between related data (ID→Object, Name→Value)
3. **Uniqueness:** Keys are automatically unique (like Set)
4. **Flexibility:** Values can be duplicated, null, or any type
5. **Real-world Modeling:** Models dictionaries, caches, configurations, relationships

1.4 When NOT to Use Map

- When you only need to store values without keys → Use List/Set
- When you need positional access → Use List
- When you need FIFO/LIFO processing → Use Queue
- When you only care about uniqueness → Use Set

2. Decision Flowchart: Which Map to Use?

```
Start: Do you need Key-Value storage?
│
├─ NO → Use List/Set/Queue
│
└─ YES → Is thread safety required?
    │
    ├─ YES → Do you need high concurrency?
    │   │
    │   ├─ YES → Use ConcurrentHashMap
    │   │
    │   └─ NO → Use Hashtable (Legacy) or synchronizedMap
    │
    └─ NO → Do you need sorted keys?
        │
        ├─ YES → Use TreeMap
        │
        └─ NO → Do you need insertion-order iteration?
            │
            ├─ YES → Use LinkedHashMap
            │
            └─ NO → Use HashMap (Default choice)
                │
                └─ Special Cases:
                    ├─ Keys should be garbage collected? → WeakHashMap
                    └─ Compare by reference (==)? → IdentityHashMap
```

3. HashMap

3.1 What is it?

- **Definition:** Hash table based implementation of the Map interface.
- **Package:** java.util
- **Characteristics:**
  - Unordered (no guarantee of iteration order)
  - Allows one null key and multiple null values
  - Not synchronized (not thread-safe)
  - Default initial capacity: 16, Load factor: 0.75

3.2 Why we use? (Purpose)

- **Performance:** Provides O(1) time complexity for basic operations (get, put).
- **Flexibility:** Most general-purpose Map implementation.
- **Memory Efficiency:** Good balance between speed and memory usage.

3.3 When to use?

- ✅ **Fast Lookup:** When you need O(1) access by key.
- ✅ **No Order Needed:** When iteration order doesn't matter.
- ✅ **General Purpose:** Default choice for most key-value storage.
- ❌ **Thread Safety:** Not safe for concurrent access without wrapping.
- ❌ **Ordering:** Does not maintain insertion or sorted order.

3.4 Internal Working (Pictorial)

```
┌──────────────────────────────────────────────────────┐
│                    HashMap Internal                  │
├──────────────────────────────────────────────────────┤
│                                                      │
│  Table: Node<K,V>[] table (Array of buckets)         │
│                                                      │
│  Index:  0      1      2      3      4      5      6  │
│         ___    ___    ___    ___    ___    ___    ___ │
│         null   null   ──→   ──→   null   ──→   null  null │
│                  |      |            |                 |
│                  |      |            |                 |
│             [K3,V3] [K7,V7]     [K5,V5]                |
│                  |                                    |
│             [K10,V10]                                  |
│                                                      |
│  Key: "John" → hashCode() = 2345 → index = (n-1) & hash │
│  Value: Student object                                 |
│                                                      |
└──────────────────────────────────────────────────────┘

Collision Handling (Java 8+):
┌─────────┐
│ Bucket 2│ ──→ [K1,V1] ──→ [K2,V2] ──→ [K3,V3] (List)
└─────────┘
    |
    ─ If size > 8 → Convert to Red-Black Tree
          ┌─────┐
          │ K2  │
         /       \
    [K1,V1]     [K3,V3]
```

Step-by-Step Insertion:

1. Calculate hash = key.hashCode() ^ (key.hashCode() >>> 16)
2. Calculate index = (table.length - 1) & hash
3. If bucket is empty → Create new Node
4. If bucket has elements → Check for collision
   - If key.equals(existingKey) → Update value
   - If not equal → Add to Linked List (or Tree if > 8 nodes)
5. If size > threshold → Resize (2x) and Rehash

// Basic Operations

V put(K key, V value) // Returns old value or null

V get(Object key) // Returns value or null

V remove(Object key) // Returns removed value or null

boolean containsKey(Object key) // Returns true/false

```
boolean containsValue(V value)  // O(n) - scans all values

// Bulk Operations
void putAll(Map<? extends K, ? extends V> m)
void clear()
int size()
boolean isEmpty()

// View Methods
Set<K> keySet()             // Returns Set of keys
Collection<V> values()        // Returns Collection of values
Set<Map.Entry<K,V>> entrySet()  // Returns Set of key-value pairs

// Java 8+ Methods
V getOrDefault(Object key, V defaultValue)
V putIfAbsent(K key, V value)
boolean remove(Object key, Object value)
boolean replace(K key, V oldValue, V newValue)
V replace(K key, V value)
void forEach(BiConsumer<? super K, ? super V> action)
```

3.6 How to use?

```
import java.util.HashMap;
import java.util.Map;
public class HashMapExample {
   public static void main(String[] args) {
      // Create HashMap
      Map<String, Integer> scores = new HashMap<>();
```

```
        // Set initial capacity to avoid resizing
        Map<String, Integer> optimized = new HashMap<>(100);
            // Put operations
        scores.put("Alice", 95);     // Returns null
        scores.put("Bob", 87);       // Returns null
        scores.put("Alice", 98);     // Returns 95 (old value)

        // Get operations
        Integer aliceScore = scores.get("Alice");  // Returns 98
        Integer charlieScore = scores.get("Charlie"); // Returns null
        int safeScore = scores.getOrDefault("Charlie", 0); // Returns 0

        // Check existence
        boolean hasAlice = scores.containsKey("Alice");   // true
        boolean hasScore100 = scores.containsValue(100);  // false

        // Remove
        Integer removed = scores.remove("Bob");  // Returns 87

        // Iterate
        for (Map.Entry<String, Integer> entry : scores.entrySet()) {
           System.out.println(entry.getKey() + " = " + entry.getValue());
        }

        // Java 8+ operations
        scores.putIfAbsent("David", 90);  // Only puts if key absent
        scores.replaceAll((k, v) -> v + 5); // Add 5 to all scores
    }
}
```

4. LinkedHashMap

4.1 What is it?

- **Definition:** Hash table and linked list implementation of Map with predictable iteration order.
- **Package:** java.util
- **Characteristics:**
  - Maintains **insertion order** (or access order)
  - Allows one null key and multiple null values
  - Not synchronized
  - Slightly slower than HashMap due to linked list overhead

4.2 Why we use? (Purpose over HashMap)

- **Problem with HashMap:** Iteration order is unpredictable (depends on hash codes).
- **Solution:** LinkedHashMap maintains a doubly-linked list running through all entries.
- **Use Case:** When you need fast lookup + predictable iteration order.

4.3 When to use?

- ✅ **Insertion Order:** When you need to iterate in the order elements were added.
- ✅ **LRU Cache:** When configured for access-order, can implement Least Recently Used cache.
- ✅ **Predictable Output:** When debugging or displaying data where order matters.
- ❌ **Performance:** Slightly slower than HashMap (5-10% overhead).
- ❌ **Memory:** Higher memory usage (linked list pointers).

4.5 Methods

- Inherits all HashMap methods
- Additional Constructor:

Java

```
LinkedHashMap(int initialCapacity, float loadFactor, boolean accessOrder)

// accessOrder: true = access-order, false = insertion-order
```

Protected Method (for LRU Cache):

java

```
protected boolean removeEldestEntry(Map.Entry<K,V> eldest)

// Override to return true when cache should remove eldest entry
```

4.4 Internal Working (Pictorial)

```
┌──────────────────────────────────────────────────────────┐
│                    LinkedHashMap Internal                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Combines: HashMap + Doubly Linked List                  │
│                                                          │
│  Hash Buckets:          Doubly Linked List:              │
│  ┌───┬───┬───┐          ┌─────┐    ┌─────┐    ┌─────┐   │
│  │ 0 │ 1 │ 2 │──────→   │ K1  │←→│ K2  │←→│ K3  │   │
│  └───┴───┴───┘          │ V1  │    │ V2  │    │ V3  │   │
│                         └─────┘    └─────┘    └─────┘   │
│                           ↑          ↑          ↑        │
│                        before     before     before      │
│                                                          │
│  Entry Structure:                                        │
│  ┌──────────────────────────────────────┐                │
│  │  int hash        │ K key  │ V value  │               │
│  │  Node next       │ before │ after    │               │
│  └──────────────────────────────────────┘                │
│                                                          │
│  Iteration follows the linked list (K1→K2→K3)            │
│  NOT the hash bucket order                               │
│                                                          │
└──────────────────────────────────────────────────────────┘

Access Order Mode (for LRU Cache):
┌─────┐    ┌─────┐    ┌─────┐
│ K3  │←→│ K1  │←→│ K2  │  (K3 accessed recently, moved to end)
└─────┘    └─────┘    └─────┘
                          ↓
```

4.6 How to use?

```
import java.util.LinkedHashMap;

import java.util.Map;

public class LinkedHashMapExample {

   public static void main(String[] args) {

      // Insertion-order (default)

      Map<String, Integer> insertionOrder = new LinkedHashMap<>();

      insertionOrder.put("C", 3);

      insertionOrder.put("A", 1);

      insertionOrder.put("B", 2);


      // Iteration: C, A, B (insertion order)

      for (String key : insertionOrder.keySet()) {

         System.out.print(key + " ");
```

```
    }

    // Access-order (for LRU Cache)
    Map<String, Integer> accessOrder = new LinkedHashMap<>(16, 0.75f, true);
    accessOrder.put("C", 3);
    accessOrder.put("A", 1);
    accessOrder.put("B", 2);
        accessOrder.get("C"); // Access C
        // Iteration: A, B, C (C moved to end due to access)
    for (String key : accessOrder.keySet()) {
      System.out.print(key + " ");
    }

    // LRU Cache Implementation
    LRUCache<Integer, String> cache = new LRUCache<>(3);
    cache.put(1, "One");
    cache.put(2, "Two");
    cache.put(3, "Three");
    cache.put(4, "Four"); // Removes eldest (1)
}
    // LRU Cache Example
static class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int maxEntries;
        public LRUCache(int maxEntries) {
      super(16, 0.75f, true); // accessOrder = true
      this.maxEntries = maxEntries;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
```

```
        return size() > maxEntries; // Remove when exceeds capacity
    }
  }
}
```

5. TreeMap

5.1 What is it?

- **Definition:** Red-Black tree based implementation of NavigableMap.
- **Package:** java.util
- **Characteristics:**
  - **Sorted** by keys (Natural order or Comparator)
  - Does NOT allow null keys (allows null values)
  - Not synchronized
  - O(log n) performance for basic operations

5.2 Why we use? (Purpose over LinkedHashMap)

- **Problem with LinkedHashMap:** Only maintains insertion order, not sorted order.
- **Solution:** TreeMap keeps keys **sorted** automatically.
- **Use Case:** When you need range queries, sorted iteration, or order statistics.

5.3 When to use?

- ✅ **Sorted Keys:** When you need keys in natural or custom sorted order.
- ✅ **Range Queries:** Finding all keys between a range (e.g., IDs 100-200).
- ✅ **Order Statistics:** Finding firstKey, lastKey, floorKey, ceilingKey.
- ❌ **Performance:** O(log n) vs O(1) for HashMap.
- ❌ **Null Keys:** Cannot store null keys (ClassCastException).

5.4 Internal Working (Pictorial)

5. TreeMap

5.1 What is it?

- **Definition:** Red-Black tree based implementation of NavigableMap.
- **Package:** java.util
- **Characteristics:**

- **Sorted** by keys (Natural order or Comparator)
- Does NOT allow null keys (allows null values)
- Not synchronized
- O(log n) performance for basic operations

5.2 Why we use? (Purpose over LinkedHashMap)

- **Problem with LinkedHashMap:** Only maintains insertion order, not sorted order.
- **Solution:** TreeMap keeps keys **sorted** automatically.
- **Use Case:** When you need range queries, sorted iteration, or order statistics.

5.3 When to use?

- ✅ **Sorted Keys:** When you need keys in natural or custom sorted order.
- ✅ **Range Queries:** Finding all keys between a range (e.g., IDs 100-200).
- ✅ **Order Statistics:** Finding firstKey, lastKey, floorKey, ceilingKey.
- ❌ **Performance:** O(log n) vs O(1) for HashMap.
- ❌ **Null Keys:** Cannot store null keys (ClassCastException).

5.5 Methods

- **Inherits all Map methods**
- **NavigableMap Methods:**

java

```
// Navigation
K firstKey()            // Lowest key
K lastKey()             // Highest key
K lowerKey(K key)       // Strictly less than key
K floorKey(K key)       // Less than or equal to key
K ceilingKey(K key)     // Greater than or equal to key
K higherKey(K key)      // Strictly greater than key

// Range Views
NavigableMap<K,V> subMap(K fromKey, boolean fromInclusive,
                         K toKey, boolean toInclusive)
```

NavigableMap<K,V> headMap(K toKey, boolean inclusive)

NavigableMap<K,V> tailMap(K fromKey, boolean inclusive)

// Reverse Order

NavigableMap<K,V> descendingMap()

NavigableSet<K> descendingKeySet()

Internal working

```
┌──────────────────────────────────────────────────────────┐
│                      TreeMap Internal                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Structure: Red-Black Tree (Self-balancing BST)          │
│                                                          │
│  Properties:                                             │
│  1. Root is BLACK                                        │
│  2. RED nodes cannot have RED children                   │
│  3. All paths from node to leaves have same BLACK nodes  │
│                                                          │
│  Example Tree:                                           │
│                        [50:B]                            │
│                       /      \                           │
│                  [25:R]      [75:B]                      │
│                  /    \       /    \                     │
│             [10:B] [30:B] [60:R] [90:R]                  │
│                                                          │
│  Key: 10, 25, 30, 50, 60, 75, 90 (In-order traversal)    │
│  Color: B=Black, R=Red                                   │
│                                                          │
│  Insertion:                                              │
│  1. Insert as RED node (like BST)                        │
│  2. Rebalance via rotations and recoloring               │
│     - Left Rotate, Right Rotate                          │
│     - Change colors                                      │
│                                                          │
│  Entry Structure:                                        │
│   ┌──────────────────────────────┐                       │
│   │  K key  │  V value  │  color  │                      │
│   │  left   │  right    │  parent │                      │
│   └──────────────────────────────┘                       │
│                                                          │
└──────────────────────────────────────────────────────────┘

Balancing Example:
Before (unbalanced):        After (balanced):
    [10]                        [20]
     \                         /   \
     [20]        →         [10]  [30]
      \
      [30]
```

How to use ?

import java.util.TreeMap;

import java.util.Map;

import java.util.NavigableMap;

```
public class TreeMapExample {
  public static void main(String[] args) {
    // Natural ordering (Integer keys)
    TreeMap<Integer, String> map = new TreeMap<>();
    map.put(50, "Fifty");
    map.put(20, "Twenty");
    map.put(70, "Seventy");
    map.put(10, "Ten");

    // Keys are automatically sorted: 10, 20, 50, 70
    System.out.println("First key: " + map.firstKey());  // 10
    System.out.println("Last key: " + map.lastKey());    // 70
        // Navigation
    System.out.println("Floor of 25: " + map.floorKey(25));   // 20
    System.out.println("Ceiling of 25: " + map.ceilingKey(25)); // 50
    System.out.println("Lower than 20: " + map.lowerKey(20));   // 10
    System.out.println("Higher than 20: " + map.higherKey(20)); // 50

    // Range queries
    NavigableMap<Integer, String> subMap = map.subMap(20, true, 70, false);
    System.out.println("SubMap: " + subMap); // {20=Twenty, 50=Fifty}

    // Custom ordering (Reverse order)
    TreeMap<String, Integer> reverseMap =
      new TreeMap<>((a, b) -> b.compareTo(a));
    reverseMap.put("C", 3);
    reverseMap.put("A", 1);
    reverseMap.put("B", 2);
    // Order: C, B, A (descending)
```

```
    // Descending iteration
    for (Map.Entry<Integer, String> entry : map.descendingMap().entrySet()) {
      System.out.println(entry.getKey() + " = " + entry.getValue());
    }
    // Output: 70=Seventy, 50=Fifty, 20=Twenty, 10=Ten
  }
}
```

6. WeakHashMap

6.1 What is it?

- **Definition:** Hash table based implementation of Map with **weak keys**.
- **Package:** java.util
- **Characteristics:**
  - Keys are stored as **WeakReferences**
  - Entries are automatically removed when keys are garbage collected
  - Not synchronized
  - Allows null keys and values

6.2 Why we use? (Purpose over HashMap)

- **Problem with HashMap:** Holds strong references to keys, preventing garbage collection even if keys are no longer used elsewhere.
- **Solution:** WeakHashMap uses weak references, allowing automatic cleanup.
- **Use Case:** Caching, metadata storage, listener registries where entries should disappear when objects are no longer referenced.

6.3 When to use?

- ✅ **Caching:** When cache entries should be removed if no longer in use.
- ✅ **Metadata:** Storing metadata about objects without preventing their GC.
- ✅ **Listeners:** Registry of listeners that should be cleaned up automatically.
- ❌ **Strong References Needed:** If you need entries to persist regardless of external references.
- ❌ **Performance:** Slight overhead due to weak reference management.

6.4 Internal Working (Pictorial)

```
WeakHashMap Internal

Key Difference from HashMap:
HashMap:    Strong Reference → Object stays in memory
WeakHashMap: Weak Reference  → Object can be GC'd

Structure:
  Entry extends WeakReference<K>
    K key (weak ref)
    V value (strong ref)
    Entry next
    int hash

Garbage Collection Process:
 External
 Reference
      ↓
 Object  ←— WeakReference  WeakHashMap
 "Key"    (in Entry)  ←— Entry
    If external reference is removed:
    ↓
 Object   WeakReference → Referent becomes
 "Key"  ←— (cleared)     null
    ↓ Garbage Collected
 (Memory freed)

Cleanup:
- WeakHashMap has a ReferenceQueue
- GC adds cleared weak references to the queue
- Next operation (get/put) processes the queue
- Removes entries with cleared keys
```

6.5 Methods

- **Same as HashMap** (no additional methods)
- **Behavior Difference:** Entries automatically disappear when keys are GC'd

6.6 How to use?

```
import java.util.WeakHashMap;

import java.util.Map;

public class WeakHashMapExample {

  public static void main(String[] args) throws InterruptedException {

    WeakHashMap<Object, String> weakMap = new WeakHashMap<>();

     // Create key object

    Object key = new Object();
```

```
    weakMap.put(key, "Value");
        System.out.println("Size before GC: " + weakMap.size()); // 1
    System.out.println("Contains key: " + weakMap.containsKey(key)); // true
        // Remove strong reference
    key = null;
        // Suggest garbage collection
    System.gc();
    Thread.sleep(100); // Give GC time to run
        // Entry is automatically removed
    System.out.println("Size after GC: " + weakMap.size()); // 0
        // Practical Example: Caching
    Cache cache = new Cache();
    cache.put(new DataObject("1"), "Result 1");
    cache.put(new DataObject("2"), "Result 2");
        System.out.println("Cache size: " + cache.size()); // 2
    // If DataObjects are no longer referenced elsewhere,
    // they will be automatically removed from cache
}
    // Simple cache that auto-cleans
static class Cache {
    private WeakHashMap<DataObject, String> map = new WeakHashMap<>();
        public void put(DataObject key, String value) {
      map.put(key, value);
    }
        public int size() {
      return map.size();
    }
}
static class DataObject {
    private String id;
```

```
    public DataObject(String id) { this.id = id; }
  }
}
```

7. IdentityHashMap

7.1 What is it?

- **Definition:** Hash table based implementation of Map that uses **reference equality (==)** instead of object equality (equals()).
- **Package:** java.util
- **Characteristics:**
  - Compares keys using == (reference equality) not equals()
  - Uses System.identityHashCode() instead of hashCode()
  - Not synchronized
  - Allows null keys and values

7.2 Why we use? (Purpose over HashMap)

- **Problem with HashMap:** Uses equals() and hashCode() which can be overridden. Two different objects with same content are treated as same key.
- **Solution:** IdentityHashMap uses == to compare keys. Only the exact same object reference is considered equal.
- **Use Case:** Topology preservation, deep copy handling, debugging, proxy patterns, handling mutable keys safely.

7.3 When to use?

- ✅ **Reference Equality:** When you need to distinguish between objects with same content.
- ✅ **Mutable Keys:** When keys might change their hashCode() after insertion.
- ✅ **Deep Copy:** Tracking which objects have been copied to avoid infinite loops.
- ✅ **Proxy/Wrapper:** Distinguishing between original and proxy objects.
- ❌ **Logical Equality:** If you want objects with same content to be treated as same key.
- ❌ **General Purpose:** HashMap is usually what you want.

7.4 Internal Working (Pictorial)

```
┌──────────────────────────────────────────────────────────┐
│                  IdentityHashMap Internal                 │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Key Difference from HashMap:                            │
│  HashMap:    key1.equals(key2) → true if content equal   │
│  IdentityHashMap: key1 == key2 → true ONLY if same ref   │
│                                                          │
│  Example:                                                │
│  String s1 = new String("Hello");                        │
│  String s2 = new String("Hello");                        │
│                                                          │
│  HashMap:                                                │
│  map.put(s1, "Value1");                                  │
│  map.put(s2, "Value2");  // Replaces Value1 (equals=true)│
│  Size = 1                                                │
│                                                          │
│  IdentityHashMap:                                        │
│  map.put(s1, "Value1");                                  │
│  map.put(s2, "Value2");  // New entry (== is false)      │
│  Size = 2                                                │
│                                                          │
│  Hash Calculation:                                       │
│  HashMap:    hash = key.hashCode()                       │
│  IdentityHashMap: hash = System.identityHashCode(key)    │
│                                                          │
│  Structure:                                              │
│  ┌──────────────────────────────────┐                    │
│  │  Uses linear probing (open addressing)  │             │
│  │  Not chained like HashMap               │             │
│  │                                         │             │
│  │  Table: Object[] table                  │             │
│  │  [0] = K1                               │             │
│  │  [1] = V1                               │             │
│  │  [2] = K2                               │             │
│  │  [3] = V2                               │             │
│  │  (Keys and values stored in same array) │             │
│  └──────────────────────────────────┘                    │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

7.5 Methods

- **Same as HashMap** (no additional methods)
- **Behavior Difference:** Uses == and identityHashCode()

How to use ?

```
import java.util.IdentityHashMap;

import java.util.HashMap;

import java.util.Map;

public class IdentityHashMapExample {

   public static void main(String[] args) {

      // HashMap behavior

      Map<String, String> hashMap = new HashMap<>();

      String s1 = new String("Hello");

      String s2 = new String("Hello");

         hashMap.put(s1, "Value1");
```

```
    hashMap.put(s2, "Value2");  // Replaces Value1
    System.out.println("HashMap size: " + hashMap.size()); // 1
    System.out.println("HashMap get(s1): " + hashMap.get(s1)); // Value2
    // IdentityHashMap behavior
    Map<String, String> identityMap = new IdentityHashMap<>();
    identityMap.put(s1, "Value1");
    identityMap.put(s2, "Value2");  // New entry (different reference)
    System.out.println("IdentityHashMap size: " + identityMap.size()); // 2
    System.out.println("IdentityHashMap get(s1): " + identityMap.get(s1)); // Value1
    System.out.println("IdentityHashMap get(s2): " + identityMap.get(s2)); // Value2
        // Practical Use Case: Deep Copy Tracking
    DeepCopyCopier copier = new DeepCopyCopier();
    Object original = new Object();
    Object copy1 = copier.copy(original);
    Object copy2 = copier.copy(original);
        // copy1 and copy2 are the same object (avoided duplicate copy)
    System.out.println("Same copy: " + (copy1 == copy2)); // true
    // Practical Use Case: Mutable Keys
    MutableKey key = new MutableKey(10);
    IdentityHashMap<MutableKey, String> map = new IdentityHashMap<>();
    map.put(key, "Value");
    key.setValue(20);  // Changes hashCode()
    System.out.println("IdentityHashMap still finds: " + map.get(key)); // Value
    // HashMap would lose this entry after hashCode() change
}
// Deep copy helper
static class DeepCopyCopier {
    private IdentityHashMap<Object, Object> copied = new IdentityHashMap<>();
    public Object copy(Object original) {
        if (copied.containsKey(original)) {
```

```
            return copied.get(original);  // Already copied
        }
        Object copy = new Object();  // Simplified
        copied.put(original, copy);
        return copy;
    }
  }
  static class MutableKey {
    private int value;
    public MutableKey(int value) { this.value = value; }
    public void setValue(int value) { this.value = value; }
    @Override
    public int hashCode() {
        return value;  // Changes when value changes
    }
  }
}
```

8. Hashtable

8.1 What is it?

- **Definition:** Legacy synchronized hash table implementation of Map.
- **Package:** java.util
- **Characteristics:**
    - **Synchronized** (thread-safe)
    - Does NOT allow null keys or values
    - Legacy class (from Java 1.0, before Collections Framework)
    - Slower than HashMap due to synchronization

8.2 Why we use? (Purpose)

- **Legacy:** Only for maintaining old code written before Java 2 (1998).
- **Thread Safety:** Provides built-in synchronization (but coarse-grained).

8.3 When to use?

- ❌ **New Code: DO NOT USE** in new development.
- ✅ **Legacy Maintenance:** Only when working with old codebases.
- **Modern Alternatives:**
  - Single-threaded: Use HashMap
  - Multi-threaded: Use ConcurrentHashMap

8.4 Internal Working (Pictorial)

```
                  Hashtable Internal
──────────────────────────────────────────────────────

  Similar to HashMap BUT:
  1. Every public method is synchronized
  2. No null keys or values allowed
  3. Legacy implementation

  Synchronization:
  ┌──────────────────────────────────┐
  │ public synchronized V put(K key, V val) │
  │ public synchronized V get(Object key)   │
  │ public synchronized V remove(Object k)  │
  │ ... ALL methods synchronized            │
  └──────────────────────────────────┘

  Performance Issue:
  Thread 1: put(K1, V1) → Locks entire table
  Thread 2: get(K2) → WAITS (even for different key)
  Thread 3: put(K3, V3) → WAITS

  vs ConcurrentHashMap:
  Thread 1: put(K1, V1) → Locks only bucket 1
  Thread 2: get(K2) → No lock needed (if different bucket)
  Thread 3: put(K3, V3) → Locks only bucket 3

  Null Handling:
  hashtable.put(null, "value"); → NullPointerException
  hashtable.put("key", null);   → NullPointerException
──────────────────────────────────────────────────────
```

8.5 Methods

- **Similar to HashMap** but with some legacy methods:

```
// Legacy methods
Enumeration<K> keys()        // Old-style iteration
Enumeration<V> elements()    // Old-style iteration
boolean contains(Object value)  // Same as containsValue (confusing!)
// Modern methods (same as HashMap)
V put(K key, V value)
V get(Object key)
```

V remove(Object key)

How to use?

```
import java.util.Hashtable;
import java.util.Map;
import java.util.concurrent.ConcurrentHashMap;
public class HashtableExample {
  public static void main(String[] args) {
    // ❌ DON'T DO THIS (Legacy)
    Map<String, String> legacy = new Hashtable<>();
    legacy.put("A", "Value A");
    // legacy.put(null, "Value"); // NullPointerException!

    // ✅ DO THIS (Modern alternatives)
        // Single-threaded
    Map<String, String> singleThreaded = new HashMap<>();
    singleThreaded.put(null, "Null key allowed");
    // Multi-threaded
    Map<String, String> concurrent = new ConcurrentHashMap<>();
    concurrent.put("A", "Value A");
    // Higher performance than Hashtable
  }
}
```

Comparison table

Feature	HashMap	LinkedHashMap	TreeMap	WeakHashMap	IdentityHashMap	Hashtable
Ordering	None	Insertion/Access	Sorted	None	None	None
Null Keys	One allowed	One allowed	**Not allowed**	Allowed	Allowed	**Not allowed**
Null Values	Allowed	Allowed	Allowed	Allowed	Allowed	**Not allowed**
Thread Safe	No	No	No	No	No	**Yes** (Sync)
Performance	**O(1)**	O(1)	O(log n)	O(1)	O(1)	O(1) (slow)
Memory	Low	Medium (list)	High (tree)	Low	Low	Low
Use Case	Default	Order/LRU Cache	Sorted/Range	Auto-cleanup	Reference equality	**Legacy**

10. Common Pitfalls & Best Practices

10.1 Null Keys

```
// ❌ TreeMap doesn't allow null keys
TreeMap<String, Integer> treeMap = new TreeMap<>();
treeMap.put(null, 1); // ClassCastException!

// ✅ HashMap allows one null key
HashMap<String, Integer> hashMap = new HashMap<>();
hashMap.put(null, 1); // Works fine
```

10.2 Mutable Keys

```
// ❌ Don't use mutable objects as keys if they affect hashCode
class MutableKey {
  int value;
  // hashCode() uses value
}
```

```
MutableKey key = new MutableKey(10);
map.put(key, "Value");
key.value = 20;  // Changes hashCode!
map.get(key);    // Returns null (lost entry)

// ✅ Use immutable keys (String, Integer) or ensure hashCode fields are final
```

10.3 Iterating and Modifying

```
// ❌ ConcurrentModificationException
for (String key : map.keySet()) {
  if (key.equals("remove")) {
    map.remove(key); // Exception!
  }
}

// ✅ Use Iterator
Iterator<String> it = map.keySet().iterator();
while (it.hasNext()) {
  if (it.next().equals("remove")) {
    it.remove(); // Safe
  }
}

// ✅ Or use removeIf (Java 8+)
map.keySet().removeIf(key -> key.equals("remove"));
```

10.4 Initial Capacity

```
// ❌ Causes multiple resizes
Map<String, Integer> map = new HashMap<>();
for (int i = 0; i < 1000; i++) {
```

```
    map.put("key" + i, i); // Resizes ~7 times
}

// ✅ Set initial capacity
// Capacity = expectedSize / loadFactor + 1
Map<String, Integer> optimized = new HashMap<>(1334); // 1000/0.75 + 1
```

10.5 Hashtable vs ConcurrentHashMap

```
// ❌ Slow (locks entire map)
Map<String, Integer> legacy = new Hashtable<>();

// ✅ Fast (locks only buckets)
Map<String, Integer> modern = new ConcurrentHashMap<>();
```

Implementation	Choose When...	Avoid When...
HashMap	Default choice, need O(1) performance	Need ordering or thread safety
LinkedHashMap	Need insertion/access order, LRU cache	Performance is critical
TreeMap	Need sorted keys, range queries	Need O(1) performance or null keys
WeakHashMap	Caching, metadata, auto-cleanup	Need strong references
IdentityHashMap	Reference equality, mutable keys, deep copy	Need logical equality
Hashtable	**NEVER** (legacy only)	Always (use ConcurrentHashMap)

1. Why Concurrency Collections When We Have Synchronized Wrappers?

Before diving into implementations, we must answer the fundamental question: **Why do we need concurrent collections if we can use Collections.synchronizedList() or Hashtable?**

1.1 The Problem with Traditional Synchronization

```
// ❌ Traditional Approach - Coarse-Grained Locking
List<String> list = Collections.synchronizedList(new ArrayList<>());
Map<String, Integer> map = new Hashtable<>();
```

// Problem: Every operation locks the ENTIRE collection

Thread 1: list.add("A") → Locks whole list

Thread 2: list.get(0) → WAITS (even though read doesn't conflict)

Thread 3: list.add("B") → WAITS

Issues:

1. **Poor Scalability:** Only one thread can access at a time
2. **High Contention:** Threads block even for non-conflicting operations
3. **Deadlock Risk:** Complex locking patterns can cause deadlocks
4. **Performance:** Lock acquisition/release has overhead

1.2 The Concurrent Collection Solution

java

// ✅ Modern Approach - Fine-Grained/Lock-Free

List<String> list = new CopyOnWriteArrayList<>();

Map<String, Integer> map = new ConcurrentHashMap<>();

// Benefit: Multiple threads can access simultaneously

Thread 1: map.put("A", 1) → Locks only bucket A

Thread 2: map.get("B") → No lock needed (different bucket)

Thread 3: map.put("C", 3) → Locks only bucket C

Advantages:

1. **High Throughput:** Multiple threads operate concurrently
2. **Lock-Free Reads:** Many operations don't need locks at all
3. **No Deadlocks:** Carefully designed to avoid deadlock conditions
4. **Better Scalability:** Performance scales with CPU cores

1.3 Concurrency Concepts

Concept	Description	Example
Thread-Safe	Can be used by multiple threads without external synchronization	ConcurrentHashMap
Lock-Free	Uses CAS instead of locks for some operations	ConcurrentLinkedQueue
Wait-Free	Every operation completes in finite steps (stronger than lock-free)	Rare in Java
Blocking	Threads wait (block) when condition not met	BlockingQueue.take()
Non-Blocking	Operations return immediately (success or failure)	ConcurrentLinkedQueue.offer()
Fail-Safe	Iterator doesn't throw ConcurrentModificationException	ConcurrentHashMap
Fail-Fast	Iterator throws exception on concurrent modification	HashMap, ArrayList

---

2. Decision Flowchart: Which Concurrent Collection to Use?

Advantages:

1. **High Throughput:** Multiple threads operate concurrently
2. **Lock-Free Reads:** Many operations don't need locks at all
3. **No Deadlocks:** Carefully designed to avoid deadlock conditions
4. **Better Scalability:** Performance scales with CPU cores

1.3 Concurrency Concepts

Concept	Description	Example
Thread-Safe	Can be used by multiple threads without external synchronization	ConcurrentHashMap
Lock-Free	Uses CAS instead of locks for some operations	ConcurrentLinkedQueue
Wait-Free	Every operation completes in finite steps (stronger than lock-free)	Rare in Java
Blocking	Threads wait (block) when condition not met	BlockingQueue.take()

Concept	Description	Example
Non-Blocking	Operations return immediately (success or failure)	ConcurrentLinkedQueue.offer()
Fail-Safe	Iterator doesn't throw ConcurrentModificationException	ConcurrentHashMap
Fail-Fast	Iterator throws exception on concurrent modification	HashMap, ArrayList

---

2. Decision Flowchart: Which Concurrent Collection to Use?

```
Start: Need Thread-Safe Collection?
│
├─ NO → Use standard collections (HashMap, ArrayList, etc.)
│
└─ YES → What type of collection?
    │
    ├─ Map (Key-Value)?
    │  │
    │  ├─ Need high concurrency (many reads/writes)?
    │  │  └─ YES → ConcurrentHashMap
    │  │
    │  └─ Need sorted keys?
    │     └─ YES → ConcurrentSkipListMap
    │
    ├─ List?
    │  │
    │  ├─ Read-heavy, write-rare?
    │  │  └─ YES → CopyOnWriteArrayList
    │  │
    │  └─ High write concurrency?
    │     └─ Consider redesign (List not ideal for concurrent writes)
    │
    ├─ Queue?
    │  │
    │  ├─ Need threads to WAIT (block) when empty/full?
    │  │  ├─ YES → BlockingQueue (ArrayBlockingQueue, LinkedBlockingQueue)
    │  │  └─ NO → Continue
    │  │
    │  └─ Need high throughput, non-blocking?
    │     └─ YES → ConcurrentLinkedQueue
    │
    └─ Set?
       │
       ├─ Need sorted + concurrent?
       │  └─ YES → ConcurrentSkipListSet
       │
       └─ Need hash-based + concurrent?
          └─ Use ConcurrentHashMap.keySet() as Set
```

3. ConcurrentHashMap

3.1 What is it?

- **Definition:** Thread-safe implementation of ConcurrentMap interface.
- **Package:** java.util.concurrent
- **Characteristics:**
    - **High concurrency:** Multiple threads can read/write simultaneously
    - **No null keys or values** (unlike HashMap)
    - **Fail-safe iterators** (no ConcurrentModificationException)
    - **Segmented locking** (Java 7) → **CAS + Node locking** (Java 8+)

3.2 Why we use? (Purpose over HashMap/Hashtable)

Feature	HashMap	Hashtable	ConcurrentHashMap
Thread Safe	❌ No	✅ Yes (synchronized)	✅ Yes (fine-grained)
Null Keys	✅ One allowed	❌ Not allowed	❌ Not allowed
Performance	O(1) (fastest)	O(1) (slow due to locks)	O(1) (near HashMap speed)
Locking	None	Entire map	Per-bucket/node
Iteration	Fail-fast	Fail-fast	**Fail-safe**

Use Case: Multi-threaded caching, shared configuration, concurrent counters.

3.3 When to use?

- ✅ **High Concurrency:** Many threads reading/writing simultaneously.
- ✅ **Performance Critical:** Need thread safety without Hashtable overhead.
- ✅ **Atomic Operations:** Need putIfAbsent, compute, merge operations.
- ❌ **Null Keys/Values:** Not allowed (use wrapper or different design).
- ❌ **Ordered Iteration:** No guaranteed order (use ConcurrentSkipListMap).

3.4 Internal Working (Pictorial)

Java 7: Segment Locking

```
┌──────────────────────────────────────────────────────────┐
│          ConcurrentHashMap (Java 7) Internal             │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Structure: Array of Segments (each is a mini HashMap)   │
│                                                          │
│   ┌─────────┐   ┌─────────┐   ┌─────────┐   ┌─────────┐  │
│   │Segment 0│   │Segment 1│   │Segment 2│   │Segment 3│  │
│   │  Lock   │   │  Lock   │   │  Lock   │   │  Lock   │  │
│   ├─────────┤   ├─────────┤   ├─────────┤   ├─────────┤  │
│   │ Bucket 0│   │ Bucket 0│   │ Bucket 0│   │ Bucket 0│  │
│   │ Bucket 1│   │ Bucket 1│   │ Bucket 1│   │ Bucket 1│  │
│   │   ...   │   │   ...   │   │   ...   │   │   ...   │  │
│   └─────────┘   └─────────┘   └─────────┘   └─────────┘  │
│       ↑             ↑             ↑             ↑        │
│  Thread 1      Thread 2      Thread 3      Thread 4      │
│  (Can access    (Can access    (Can access    (Can access  │
│   concurrently) concurrently) concurrently) concurrently)│
│                                                          │
│  Default: 16 segments → 16 concurrent threads max        │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Java 8+: CAS + Node Locking (Current)

```
┌──────────────────────────────────────────────────────────┐
│          ConcurrentHashMap (Java 8+) Internal            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Structure: Array of Nodes (like HashMap) + CAS + Locks  │
│                                                          │
│  Table: Node<K,V>[] table                                │
│                                                          │
│  Index:  0      1      2      3      4      5      6     │
│         ──     ──     ──     ──     ──     ──     ──    │
│         null   ──→    ──→   null   ──→   null   null    │
│                 |      |            |                    │
│            [K1,V1] [K7,V7]      [K5,V5]                  │
│                 |                                        │
│            [K2,V2] (Tree if > 8)                         │
│                                                          │
│  Insertion Logic:                                        │
│  ┌──────────────────────────────────────────────┐        │
│  │ 1. If bucket empty → CAS to insert (lock-free) │      │
│  │ 2. If bucket has elements → synchronized on    │      │
│  │    head node only (not entire map)             │      │
│  │ 3. If collision → Linked List → Tree (if > 8)  │      │
│  └──────────────────────────────────────────────┘        │
│                                                          │
│  CAS Operation (Compare-And-Swap):                       │
│  ┌──────────────────────────────────────────────┐        │
│  │ if (table[index] == null) {                    │      │
│  │     // Try to atomically set new node          │      │
│  │     if (UNSAFE.compareAndSwapObject(...)) {    │      │
│  │         success! (no lock needed)              │      │
│  │     }                                          │      │
│  │ }                                              │      │
│  └──────────────────────────────────────────────┘        │
│                                                          │
│  Read Operations:                                        │
│  - NO locking required (volatile reads)                  │
│  - Multiple threads can read simultaneously              │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

3.5 Methods

// Basic Operations (Thread-Safe)

V put(K key, V value) // Returns old value or null

V get(Object key) // Returns value or null

V remove(Object key) // Returns removed value or null

boolean containsKey(Object key) // Returns true/false

// Atomic Operations (Key Feature!)

V putIfAbsent(K key, V value) // Put only if key absent

boolean remove(Object key, Object value) // Remove only if matches

boolean replace(K key, V oldValue, V newValue) // Replace if matches

```
V replace(K key, V value)        // Replace if key exists

// Compute Operations (Java 8+)
V compute(K key, BiFunction remappingFunction)
V computeIfAbsent(K key, Function mappingFunction)
V computeIfPresent(K key, BiFunction remappingFunction)
V merge(K key, V value, BiFunction remappingFunction)
// Bulk Operations
void forEach(BiConsumer<? super K, ? super V> action)
void search(long parallelismThreshold, BiFunction searcher)
void reduce(long parallelismThreshold, BiFunction reducer)

// View Methods
ConcurrentKeySetView<K,V> keySet()
Collection<V> values()
Set<Map.Entry<K,V>> entrySet()
```

3.6 How to use?

```
import java.util.concurrent.ConcurrentHashMap;
import java.util.Map;
import java.util.concurrent.ConcurrentMap;
public class ConcurrentHashMapExample {
  public static void main(String[] args) {
    ConcurrentMap<String, Integer> map = new ConcurrentHashMap<>();
        // Basic operations
    map.put("A", 1);
    map.put("B", 2);
    map.get("A");  // Thread-safe read

    // Atomic operations (CRITICAL for concurrency)
```

```
    map.putIfAbsent("C", 3);  // Only puts if "C" doesn't exist
        // Safe increment pattern (avoid race conditions)
    // ❌ WRONG (not atomic)
    // Integer count = map.get("counter");
    // map.put("counter", count + 1);  // Race condition!
        // ✅ RIGHT (atomic)
    map.compute("counter", (k, v) -> (v == null) ? 1 : v + 1);
        // Or use merge for counting
    map.merge("word", 1, Integer::sum);  // Increment count
        // Safe removal
    map.remove("A", 1);  // Only removes if value is 1
        // Bulk operations (parallel)
    map.forEach(1, (k, v) -> System.out.println(k + "=" + v));
        // Fail-safe iteration (no ConcurrentModificationException)
    for (Map.Entry<String, Integer> entry : map.entrySet()) {
      System.out.println(entry.getKey() + " = " + entry.getValue());
      // Other threads can modify map during iteration safely
    }
  }
}
```

3.7 Internal Working: Atomic Operations

```
// compute() Internal Logic (Simplified)
public V compute(K key, BiFunction remappingFunction) {
  Node<K,V>[] tab; int n; long sizeCtl; int index; Node<K,V> f;
    // 1. Find the bucket
  // 2. Lock the bucket head (synchronized on node)
  // 3. Apply the remapping function
  // 4. Update the value
  // 5. Unlock
  // All done atomically - no race condition possible
```

```
}
// merge() for Counting Example
map.merge("word", 1, Integer::sum);

// Internal Steps:
// 1. Get current value for "word" (e.g., 5)
// 2. Apply function: 5 + 1 = 6
// 3. Atomically update to 6
// 4. If another thread tries same key, it waits for lock

// Result: Accurate count even with 1000 concurrent threads
```

4. CopyOnWriteArrayList

4.1 What is it?

- **Definition:** Thread-safe variant of ArrayList where all mutative operations (add, set, remove) create a fresh copy of the underlying array.
- **Package:** java.util.concurrent
- **Characteristics:**
  - **Read operations are lock-free** (very fast)
  - **Write operations are expensive** (copy entire array)
  - **Fail-safe iterators** (snapshot of array at creation time)
  - **Best for read-heavy, write-rare scenarios**

4.2 Why we use? (Purpose over synchronizedList)

Feature	synchronizedList	CopyOnWriteArrayList
Read Lock	✅ Yes (every read locks)	❌ No (lock-free reads)
Write Lock	✅ Yes	✅ Yes (copy + replace)
Iterator	Fail-fast (needs sync)	**Fail-safe** (snapshot)
Best For	Balanced read/write	**Read-heavy, write-rare**
Memory	Low	High (copies on write)

Use Case: Listener registries, configuration lists, observer patterns.

4.3 When to use?

- ✅ **Read-Heavy:** 90%+ reads, <10% writes.
- ✅ **Iteration Safety:** Need to iterate without ConcurrentModificationException.
- ✅ **Listener Lists:** Registering/unregistering event listeners.
- ❌ **Write-Heavy:** Copying array on every write is expensive O(n).
- ❌ **Large Lists:** Copying large arrays is memory-intensive.

4.5 Methods

```
// Inherits all ArrayList methods

// Key differences in behavior:

// Add (Creates copy)

boolean add(E e)              // O(n) - copies array

void add(int index, E element)     // O(n) - copies array

boolean addAll(Collection<? extends E> c) // O(n)

// Remove (Creates copy)

E remove(int index)           // O(n) - copies array

boolean remove(Object o)         // O(n) - copies array

// Set (Creates copy)

E set(int index, E element)       // O(n) - copies array


// Read (Lock-free)

E get(int index)              // O(1) - no lock

int size()                    // O(1) - no lock

boolean contains(Object o)        // O(n) - no lock

// Iterator (Fail-safe)

Iterator<E> iterator()          // Returns snapshot iterator

ListIterator<E> listIterator()     // Returns snapshot iterator
```

4.4 Internal Working (Pictorial)

```
┌──────────────────────────────────────────────────────────────┐
│            CopyOnWriteArrayList Internal                     │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Structure: Immutable Array + ReentrantLock                  │
│                                                              │
│  Before Write:                                               │
│  ┌──────────────────────────────────────┐                   │
│  │  Array: [A, B, C, D]                 │                   │
│  │  Lock: UNLOCKED                      │                   │
│  └──────────────────────────────────────┘                   │
│                                                              │
│  Write Operation (add "E"):                                  │
│  ┌──────────────────────────────────────────────┐           │
│  │ 1. Acquire lock                              │           │
│  │ 2. Create NEW array (copy old + new element) │           │
│  │    Old: [A, B, C, D]                         │           │
│  │    New: [A, B, C, D, E]                      │           │
│  │ 3. Replace reference atomically              │           │
│  │ 4. Release lock                              │           │
│  └──────────────────────────────────────────────┘           │
│                                                              │
│  After Write:                                                │
│  ┌──────────────────────────────────────┐                   │
│  │  Array: [A, B, C, D, E]  (new array) │                   │
│  │  Old array: [A, B, C, D] → GC'd      │                   │
│  └──────────────────────────────────────┘                   │
│                                                              │
│  Read Operation:                                             │
│  ┌──────────────────────────────────────────────┐           │
│  │ 1. Get current array reference (volatile read) │         │
│  │ 2. Read from array (NO lock needed)          │           │
│  │ 3. Multiple threads can read simultaneously  │           │
│  └──────────────────────────────────────────────┘           │
│                                                              │
│  Iterator (Fail-Safe):                                       │
│  ┌──────────────────────────────────────────────┐           │
│  │ 1. Capture array reference at iterator creation │        │
│  │ 2. Iterate over that snapshot                │           │
│  │ 3. Even if list changes, iterator sees old data │        │
│  │ 4. NO ConcurrentModificationException        │           │
│  └──────────────────────────────────────────────┘           │
│                                                              │
└──────────────────────────────────────────────────────────────┘

Memory Visualization:
Thread 1 (Reading):  ──► Array v1 [A, B, C]
Thread 2 (Reading):  ──► Array v1 [A, B, C]
Thread 3 (Writing):  ──► Lock → Copy → Array v2 [A, B, C, D]
                         → Replace reference
                         → Old Array v1 → GC (when no readers)
```

4.6 How to use?

```
import java.util.concurrent.CopyOnWriteArrayList;

import java.util.List;

import java.util.Iterator;

public class CopyOnWriteArrayListExample {

    // Common Use Case: Listener Registry

  static class EventManager {

    // Thread-safe listener list

    private List<Listener> listeners = new CopyOnWriteArrayList<>();

        public void addListener(Listener l) {

      listeners.add(l);  // Safe, copies array
```

```
    }
    public void removeListener(Listener l) {
      listeners.remove(l);  // Safe, copies array
    }
        public void fireEvent(String event) {
      // Safe iteration - no ConcurrentModificationException
      // Even if listeners are added/removed during iteration
      for (Listener listener : listeners) {
        listener.onEvent(event);
      }
    }
  }
    public static void main(String[] args) throws InterruptedException {
    CopyOnWriteArrayList<String> list = new CopyOnWriteArrayList<>();
    list.add("A");
    list.add("B");
    list.add("C");
        // Thread 1: Iterate
    Thread reader = new Thread(() -> {
      for (String s : list) {
        System.out.println("Reading: " + s);
        try { Thread.sleep(100); } catch (InterruptedException e) {}
      }
    });
        // Thread 2: Modify (won't affect reader's iteration)
    Thread writer = new Thread(() -> {
      list.add("D");  // Creates new array
      list.add("E");  // Creates new array
      System.out.println("Modified list");
    });
```

```
    reader.start();
    writer.start();
        reader.join();
    writer.join();
        // Final state
    System.out.println("Final list: " + list);  // [A, B, C, D, E]
        // Performance Warning: Don't do this!
    // ❌ WRONG - O(n²) copies
    for (int i = 0; i < 1000; i++) {
      list.add("Item" + i);  // Each add copies entire array!
    }
        // ✅ RIGHT - Batch writes
    List<String> batch = new ArrayList<>();
    for (int i = 0; i < 1000; i++) {
      batch.add("Item" + i);
    }
    list.addAll(batch);  // Only ONE copy
  }
}
```

5. BlockingQueue (Interface & Implementations)

5.1 What is it?

- **Definition:** Interface for queues that support operations that **wait** for the queue to become non-empty when retrieving, and wait for space when storing.
- **Package:** java.util.concurrent
- **Characteristics:**
    - **Blocking:** Threads can wait (block) on empty/full conditions
    - **Thread-Safe:** All operations are atomic
    - **Producer-Consumer:** Designed for multi-threaded handoff

- **Implementations:** ArrayBlockingQueue, LinkedBlockingQueue, PriorityBlockingQueue, DelayQueue, SynchronousQueue

5.2 Why we use? (Purpose over ConcurrentLinkedQueue)

Feature	ConcurrentLinkedQueue	BlockingQueue
Blocking	❌ No (returns immediately)	✅ Yes (waits for condition)
Use Case	High throughput, non-blocking	**Producer-Consumer**
CPU Usage	Low (no waiting)	Low (threads sleep when waiting)
Backpressure	❌ No	✅ Yes (bounded queues)
Timeout Support	❌ No	✅ Yes (poll(timeout))

Use Case: Task queues, work distribution, throttling, pipeline processing.

5.3 When to use?

- ✅ **Producer-Consumer:** One thread produces, another consumes.
- ✅ **Throttling:** Limit processing rate with bounded queue.
- ✅ **Thread Coordination:** Consumers wait for work, producers wait for space.
- ❌ **Non-Blocking Needed:** Use ConcurrentLinkedQueue instead.
- ❌ **Single Thread:** No benefit over regular Queue.

5.5 Methods

```
// Insert Operations
boolean add(E e)          // Throws exception if no space
boolean offer(E e)        // Returns false if no space
void put(E e)             // BLOCKS if no space (wait forever)
boolean offer(E e, long timeout, TimeUnit unit) // Blocks with timeout
// Remove Operations
E remove()                // Throws exception if empty
E poll()                  // Returns null if empty
E take()                  // BLOCKS if empty (wait forever)
E poll(long timeout, TimeUnit unit) // Blocks with timeout
```

// Inspection

E element() // Throws exception if empty

E peek() // Returns null if empty

// Capacity

int remainingCapacity() // How many more elements can be added

int drainTo(Collection<? super E> c) // Remove all to collection

5.4 Internal Working (Pictorial)

```
BlockingQueue Internal Architecture

Core Components:
  1. Queue Storage (Array or Linked Nodes)
  2. ReentrantLock (for mutual exclusion)
  3. Condition: notEmpty (consumers wait here)
  4. Condition: notFull (producers wait here)

Producer Thread:
  1. Acquire lock
  2. While queue is FULL:
     - await(notFull)  ← Thread SLEEPS here
  3. Add element to queue
  4. signal(notEmpty) ← Wake up waiting consumers
  5. Release lock

Consumer Thread:
  1. Acquire lock
  2. While queue is EMPTY:
     - await(notEmpty) ← Thread SLEEPS here
  3. Remove element from queue
  4. signal(notFull) ← Wake up waiting producers
  5. Release lock

State Diagram:
  EMPTY (consumers wait)  --put()-->  PARTIAL  --take()-->  FULL (producers wait)
  EMPTY (consumers wait)  <--take()--  PARTIAL  <--put()--  FULL (producers wait)
```

```
rayBlockingQueue vs LinkedBlockingQueue:

| ArrayBlockingQueue | LinkedBlockingQueue |
|---|---|
| Fixed capacity | Bounded or unbounded |
| Single lock | Two locks (put/take sep-  -te) |
| Better cache | More concurrent |
```

How to use?

import java.util.concurrent.*;

import java.util.concurrent.BlockingQueue;

```
public class BlockingQueueExample {
    // Producer-Consumer Pattern
  static class ProducerConsumer {
    private final BlockingQueue<Integer> queue = new ArrayBlockingQueue<>(10);
        class Producer implements Runnable {
      public void run() {
        try {
          for (int i = 0; i < 20; i++) {
            queue.put(i);  // Blocks if queue is full
            System.out.println("Produced: " + i);
            Thread.sleep(100);
          }
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        }
      }
    }
        class Consumer implements Runnable {
      public void run() {
        try {
          while (true) {
            Integer item = queue.take();  // Blocks if queue is empty
            System.out.println("Consumed: " + item);
            Thread.sleep(200);
          }
        } catch (InterruptedException e) {
          Thread.currentThread().interrupt();
        }
      }
    }   }
```

```
public static void main(String[] args) {
  BlockingQueue<String> queue = new ArrayBlockingQueue<>(5);
      // Producer
  Thread producer = new Thread(() -> {
    try {
      for (int i = 1; i <= 10; i++) {
        queue.put("Task-" + i);  // Blocks after 5 items
        System.out.println("Produced: Task-" + i);
      }
    } catch (InterruptedException e) {
      e.printStackTrace();
    }
  });
      // Consumer
  Thread consumer = new Thread(() -> {
    try {
      while (true) {
        String task = queue.take();  // Blocks when empty
        System.out.println("  Consumed: " + task);
        Thread.sleep(500);  // Simulate work
      }
    } catch (InterruptedException e) {
      e.printStackTrace();
    }
  });

  producer.start();
  consumer.start();
```

```
        // Timeout Example
        try {
            // Wait up to 1 second for an item
            String item = queue.poll(1, TimeUnit.SECONDS);
            if (item == null) {
                System.out.println("Timeout - no item available");
            }
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }
}
```

BlockingQueue Implementations Comparison

Implementation	Capacity	Ordering	Lock Strategy	Use Case
ArrayBlockingQueue	Fixed	FIFO	Single lock	Bounded, predictable memory
LinkedBlockingQueue	Optional (unbounded default)	FIFO	Two locks (put/take)	High throughput, flexible size
PriorityBlockingQueue	Unbounded	Priority	Single lock	Priority-based processing
DelayQueue	Unbounded	Delay-ordered	Single lock	Scheduled tasks, expiration
SynchronousQueue	0 (handoff only)	FIFO/LIFO	No storage	Direct thread handoff

6. ConcurrentLinkedQueue

6.1 What is it?

- **Definition:** An unbounded thread-safe queue based on linked nodes.
- **Package:** java.util.concurrent
- **Characteristics:**
  - **Lock-free:** Uses CAS (Compare-And-Swap) instead of locks
  - **Non-blocking:** Operations return immediately (no waiting)
  - **Unbounded:** No capacity limit
  - **FIFO:** First-In-First-Out ordering
  - **Nulls:** Not allowed

6.2 Why we use? (Purpose over BlockingQueue)

Feature	BlockingQueue	ConcurrentLinkedQueue
Blocking	✅ Yes (waits)	❌ No (immediate)
Locking	✅ Yes (ReentrantLock)	❌ No (CAS only)
Throughput	Medium (lock overhead)	**Very High** (lock-free)
Backpressure	✅ Yes (bounded options)	❌ No (unbounded)
Use Case	Producer-Consumer	High-volume logging, event streams

Use Case: High-throughput event logging, non-blocking task distribution, metrics collection.

6.3 When to use?

- ✅ **High Throughput:** Many threads, need maximum performance.
- ✅ **Non-Blocking:** Don't want threads to wait/sleep.
- ✅ **Unbounded:** Don't need to limit queue size.
- ❌ **Backpressure Needed:** Use BlockingQueue with capacity limit.
- ❌ **Blocking Needed:** Use BlockingQueue if consumers should wait.

6.5 Methods

```
// All methods are non-blocking

// Insert

boolean offer(E e)        // Always returns true (unbounded)
```

boolean add(E e) *// Same as offer*

// Remove

E poll() *// Returns null if empty*

E remove() *// Throws exception if empty*

// Inspection

E peek() *// Returns null if empty*

E element() *// Throws exception if empty*

// Note: NO blocking methods (put, take)

// Note: size() is O(n) - traverses nodes

6.4 Internal Working (Pictorial)

```
┌──────────────────────────────────────────────────────────────┐
│              ConcurrentLinkedQueue Internal                  │
├──────────────────────────────────────────────────────────────┤
│                                                              │
│  Structure: Linked Nodes with CAS Operations                 │
│                                                              │
│  Node Structure:                                             │
│  ┌──────────────────────────────────────┐                    │
│  │  E item                              │                    │
│  │  volatile Node next                  │                    │
│  └──────────────────────────────────────┘                    │
│                                                              │
│  Queue State:                                                │
│  head ──→ [Node1] ──→ [Node2] ──→ [Node3] ──→ null          │
│   ↑                                          ↑               │
│   └──────────────────── tail ────────────────┘               │
│                                                              │
│  Enqueue Operation (CAS-based):                              │
│  ┌──────────────────────────────────────────────────┐        │
│  │ 1. Create new node                               │        │
│  │ 2. Read current tail                             │        │
│  │ 3. Try to CAS tail.next to new node              │        │
│  │    if (UNSAFE.compareAndSwapObject(tail, ...))   │        │
│  │ 4. If CAS fails (another thread won), retry      │        │
│  │ 5. Update tail to new node (can be lazy)         │        │
│  └──────────────────────────────────────────────────┘        │
│                                                              │
│  Dequeue Operation (CAS-based):                              │
│  ┌──────────────────────────────────────────────────┐        │
│  │ 1. Read current head                             │        │
│  │ 2. Get head.next                                 │        │
│  │ 3. Try to CAS head to head.next                  │        │
│  │ 4. If CAS fails, retry                           │        │
│  │ 5. Return head.item                              │        │
│  └──────────────────────────────────────────────────┘        │
│                                                              │
│  CAS (Compare-And-Swap):                                     │
│  ┌──────────────────────────────────────────────────┐        │
│  │ Atomic operation:                                │        │
│  │ if (current_value == expected_value) {           │        │
│  │     current_value = new_value;                   │        │
│  │     return true;  // Success                     │        │
│  │ } else {                                         │        │
│  │     return false; // Another thread modified     │        │
│  │ }                                                │        │
│  └──────────────────────────────────────────────────┘        │
│                                                              │
│  Lazy Updates:                                               │
│  - Head/tail pointers may lag behind actual ends             │
│  - Reduces contention (fewer CAS operations)                 │
│  - Eventually consistent                                     │
└──────────────────────────────────────────────────────────────┘

CAS Visualization:
Thread 1: CAS(tail.next, null, newNode1) → SUCCESS
Thread 2: CAS(tail.next, null, newNode2) → FAIL (tail.next changed)
          → Retry with new tail
```

.6 How to use?

```
import java.util.concurrent.ConcurrentLinkedQueue;
import java.util.Queue;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class ConcurrentLinkedQueueExample {
  public static void main(String[] args) throws InterruptedException {
    Queue<String> queue = new ConcurrentLinkedQueue<>();
        // High-throughput producer
    ExecutorService executor = Executors.newFixedThreadPool(10);
        // 10 producer threads
    for (int i = 0; i < 10; i++) {
      final int threadId = i;
      executor.submit(() -> {
        for (int j = 0; j < 1000; j++) {
          queue.offer("Task-" + threadId + "-" + j);
          // No locking, very fast
        }
      });
    }
        executor.shutdown();
    executor.awaitTermination(10, TimeUnit.SECONDS);

    System.out.println("Total tasks: " + queue.size());  // O(n) operation
        // Consumer (non-blocking)
    String task;
    int processed = 0;
    while ((task = queue.poll()) != null) {
      // Process task
      processed++;
```

```
        // If queue empty, poll() returns null immediately
        // No blocking/waiting
    }
    System.out.println("Processed: " + processed);
        // Important: Don't use size() in loops!
    // ❌ WRONG - O(n²) total
    // for (int i = 0; i < queue.size(); i++) { ... }
        // ✅ RIGHT - O(n) total
    // for (String s : queue) { ... }
  }
}
```

7. Comparison Table

Feature	ConcurrentHashMap	CopyOnWriteArrayList	BlockingQueue	ConcurrentLinkedQueue
Type	Map	List	Queue	Queue
Thread Safe	✅ Yes	✅ Yes	✅ Yes	✅ Yes
Locking	Fine-grained/CAS	Copy-on-write	ReentrantLock	**Lock-free (CAS)**
Read Performance	**O(1) lock-free**	**O(1) lock-free**	O(1) with lock	O(1) lock-free
Write Performance	O(1) with lock	**O(n) copy**	O(1) with lock	O(1) CAS
Best For	Concurrent Map	Read-heavy List	Producer-Consumer	High-throughput Queue
Blocking	❌ No	❌ No	✅ Yes	❌ No
Nulls	❌ Not allowed	✅ Allowed	❌ Not allowed	❌ Not allowed
Iterator	Fail-safe	Fail-safe	Weakly consistent	Weakly consistent
Memory	Medium	High (copies)	Low	Medium

Summary

Collection	Choose When...	Avoid When...
ConcurrentHashMap	Concurrent map access, atomic operations	Need null keys, ordered iteration
CopyOnWriteArrayList	Read-heavy lists, safe iteration	Frequent writes, large lists
BlockingQueue	Producer-consumer, backpressure needed	Non-blocking required
ConcurrentLinkedQueue	High-throughput, lock-free queue	Backpressure, blocking needed

Quick Decision sheet

```
Need to store data?
│
├─ Key-Value pairs? → MAP
│   ├─ Thread-safe + high concurrency? → ConcurrentHashMap
│   ├─ Sorted keys? → TreeMap
│   ├─ Insertion order? → LinkedHashMap
│   └─ Default choice? → HashMap
│
├─ Unique elements only? → SET
│   ├─ Sorted? → TreeSet
│   ├─ Insertion order? → LinkedHashSet
│   └─ Default? → HashSet
│
├─ Ordered + duplicates allowed? → LIST
│   ├─ Fast random access? → ArrayList
│   ├─ Frequent add/remove at ends? → LinkedList
│   └─ Thread-safe read-heavy? → CopyOnWriteArrayList
│
├─ Process in order (FIFO/LIFO/Priority)? → QUEUE
│   ├─ Producer-Consumer? → BlockingQueue
│   ├─ High throughput non-blocking? → ConcurrentLinkedQueue
│   ├─ Priority order? → PriorityQueue
│   └─ Stack/Queue dual? → ArrayDeque
│
└─ Multi-threaded access? → CONCURRENCY COLLECTIONS
    ├─ Map → ConcurrentHashMap
    ├─ List → CopyOnWriteArrayList
    ├─ Queue → BlockingQueue / ConcurrentLinkedQueue
    └─ Set → ConcurrentSkipListSet / ConcurrentHashMap.keySet()
```

Java 8+ Modern Features Deep Dive

Lambda Expressions - The foundation

Functional Interfaces - The contract

Method References - Shorthand syntax

Streams API - Declarative data processing

Optional Class - Null safety

Date/Time API - Modern date handling

Default & Static Methods in Interfaces

Part 1: Lambda Expressions

1.1 What is a Lambda Expression?

Definition: A lambda expression is a concise way to represent an anonymous function (a function without a name) that can be passed around as data.

Before Java 8 (Anonymous Class):

```
// Verbose way
Runnable runnable = new Runnable() {
  @Override
  public void run() {
    System.out.println("Hello");
  }
};
```

After Java 8(Lambda):

```
// Concise way
Runnable runnable = () -> System.out.println("Hello");
```

.2 Why We Use Lambdas?

Problem Before Java 8	Solution with Lambdas
Verbose anonymous classes	Concise syntax
Cannot pass functions as parameters	Functions are first-class citizens
Boilerplate code everywhere	Declarative programming
Hard to parallelize	Easy parallel processing

Real-World Analogy:

- **Before:** Writing a full recipe card for every small task
- **After:** Just saying "boil water" and everyone knows what to do

1.3 Lambda Syntax

```
// Basic syntax
(parameters) -> expression
// OR
(parameters) -> { statements; }
```

Example:

```
// No parameters
() -> System.out.println("Hello")
// One parameter (parentheses optional)
x -> x * 2
// Multiple parameters
(x, y) -> x + y
// With body
(x, y) -> {
   int sum = x + y;
   return sum;
}
// With type declarations
(int x, int y) -> x + y
```

1.4 When to Use Lambdas?

✅ **Use Lambdas When:**

- Passing behavior as a parameter (sorting, filtering)
- Implementing functional interfaces (Runnable, Comparator)
- Working with Streams API, Event handling
- Callbacks

❌ **Don't Use Lambdas When:**

- You need to reuse the logic multiple times (use a method instead)
- The logic is complex (> 3-4 lines)
- You need to access this from enclosing class (lambdas don't capture this)

1.5 Internal Working of Lambdas

Before Java 8: Anonymous classes create **new .class files**

```
// Creates: OuterClass$1.class
new Runnable() {
  public void run() { }
}
```

Java 8+: Uses **invokedynamic** bytecode instruction

```
// Uses LambdaMetafactory at runtime
() -> System.out.println("Hello")
```

Internal Process:

```
┌──────────────────────────────────────────┐
│            Lambda Compilation Process    │
├──────────────────────────────────────────┤
│                                          │
│ 1. Source Code                           │
│    (x, y) -> x + y                       │
│                                          │
│ 2. Compiler creates bootstrap method     │
│    Uses invokedynamic bytecode           │
│                                          │
│ 3. At Runtime:                           │
│    ┌──────────────────────────┐          │
│    │ LambdaMetafactory        │          │
│    │  - Creates implementation class     │
│    │  - Links to functional interface    │
│    └──────────────────────────┘          │
│                                          │
│ 4. Result:                               │
│    - No .class file generated            │
│    - More efficient than anonymous classes
│    - Can be optimized by JVM             │
│                                          │
└──────────────────────────────────────────┘
```

Key Differences:

Feature	Anonymous Class	Lambda
Class File	Creates new .class	No new class (uses invokedynamic)
this reference	Points to anonymous class	Points to enclosing class
Performance	Slower (class loading)	Faster (runtime generation)
Memory	More overhead	Less overhead

1.6 How to Use Lambdas - Common Examples

```
// 1. Runnable (no parameters, no return)
Thread thread = new Thread(() -> {
   System.out.println("Running in thread");
});
// 2. Comparator (sorting)
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
names.sort((a, b) -> a.compareTo(b));
// 3. Consumer (takes parameter, no return)
List<String> list = Arrays.asList("A", "B", "C");
list.forEach(s -> System.out.println(s));
// 4. Function (takes parameter, returns value)
Function<Integer, Integer> square = x -> x * x;
Integer result = square.apply(5);  // 25
// 5. Predicate (takes parameter, returns boolean)
Predicate<Integer> isEven = x -> x % 2 == 0;
boolean check = isEven.test(4);  // true

// 6. Supplier (no parameters, returns value)
Supplier<Double> random = () -> Math.random();
Double value = random.get();
```

Part 2: Functional Interfaces

2.1 What is a Functional Interface?

Definition: An interface with **exactly one abstract method** (can have multiple default/static methods).

A **Functional Interface** is an interface that contains **exactly one abstract method**. It can have multiple default methods, static methods, and methods from Object cla

Key Point: Lambdas can ONLY be used with functional interfaces.

2.2 Why We Use Functional Interfaces?

- **Contract for Lambdas:** Defines what a lambda must implement

- **Type Safety:** Compiler knows what method signature to expect
- **Reusability:** Can be used across the codebase
- **API Design:** Enables fluent, declarative APIs

Why function interface Exist?

```
┌──────────────────────────────────────────────────┐
│              FUNCTIONAL INTERFACE PURPOSE                  │
├──────────────────────────────────────────────────┤
│                                                  │
│  Problem:                                        │
│  - Java didn't support passing functions as parameters  │
│  - Anonymous classes were verbose                │
│  - No way to represent behavior as data          │
│                                                  │
│  Solution:                                       │
│  - Functional Interface = Contract for Lambda    │
│  - Lambda = Implementation of that contract      │
│  - Enables functional programming in Java        │
│                                                  │
│  Analogy:                                        │
│  - Functional Interface = Job Description        │
│  - Lambda = Worker who fulfills that job         │
│                                                  │
└──────────────────────────────────────────────────┘
```

Rules for Functional Interfaces

Rule	Description	Example
One Abstract Method	Exactly one method without implementation	void test(T t);
@FunctionalInterface	Optional annotation (compiler enforcement)	@FunctionalInterface
Default Methods OK	Can have multiple default methods	default void help() {}
Static Methods OK	Can have multiple static methods	static void util() {}
Object Methods OK	Methods from Object class don't count	equals(), toString()

2.3 @FunctionalInterface Annotation

```
@FunctionalInterface
public interface MyFunctionalInterface {
  void doSomething();  // Single abstract method
  // Can have default methods
  default void helper() {
    System.out.println("Helper");
  }
```

```
    // Can have static methods
    static void utility() {
        System.out.println("Utility");
    }
}
```

Compiler Enforcement:

```
@FunctionalInterface
public interface InvalidInterface {
    void method1();
    void method2();  // ❌ Compilation error!
                     // More than one abstract method
}
```

2.5 Internal Working of Functional Interfaces

```
// Source code
Predicate<Integer> isEven = x -> x % 2 == 0;
// Compiled to:
// 1. LambdaMetafactory creates implementation
// 2. Implements Predicate<Integer> interface
// 3. test() method contains lambda logic
// Runtime representation:
class Lambda$1 implements Predicate<Integer> {
    public boolean test(Integer x) {
        return x % 2 == 0;
    }
}
```

2.6 How to Use Built-in Functional Interfaces

```
// 1. Predicate - Filtering
Predicate<String> isEmpty = s -> s.isEmpty();
List<String> nonEmpty = list.stream()
```

```
    .filter(s -> !isEmpty.test(s))
    .collect(Collectors.toList());

// 2. Consumer - Side effects
Consumer<String> printer = s -> System.out.println(s);
list.forEach(printer);

// Chaining consumers
Consumer<String> prefix = s -> System.out.print("Value: ");
Consumer<String> combined = prefix.andThen(printer);
combined.accept("Hello");  // "Value: Hello"

// 3. Function - Transformation
Function<String, Integer> length = String::length;
List<Integer> lengths = names.stream()
    .map(length)
    .collect(Collectors.toList());
// Chaining functions
Function<Integer, Integer> addOne = x -> x + 1;
Function<Integer, Integer> multiplyTwo = x -> x * 2;
Function<Integer, Integer> composed = addOne.andThen(multiplyTwo);
composed.apply(5);  // (5+1)*2 = 12

// 4. Supplier - Lazy creation
Supplier<LocalDate> today = LocalDate::now;
LocalDate date1 = today.get();
LocalDate date2 = today.get();  // Different if called next day

// 5. UnaryOperator - Same type transformation
UnaryOperator<String> toUpper = String::toUpperCase;
```

```
String result = toUpper.apply("hello");  // "HELLO"


// 6. BinaryOperator - Reduction
BinaryOperator<Integer> sum = (a, b) -> a + b;
Integer total = sum.apply(5, 3);  // 8
```

2. Core Functional Interfaces (java.util.function)

2.1 Predicate<T>

Definition: Takes one argument, returns a **boolean**.

Method Signature:

java

```
@FunctionalInterface
public interface Predicate<T> {
  boolean test(T t);


  // Default methods
  default Predicate<T> and(Predicate<? super T> other)
  default Predicate<T> or(Predicate<? super T> other)
  default Predicate<T> negate()
  // Static method
  static <T> Predicate<T> isEqual(Object targetRef)
}
```

When to Use:

- Filtering collections
- Validation logic
- Conditional checks
- Any scenario returning true/false

Internal Working:

```
┌──────────────────────────────────────────────────────┐
│                   PREDICATE INTERNAL                 │
├──────────────────────────────────────────────────────┤
│                                                      │
│  Input: T (any type)                                 │
│     ↓                                                │
│  Process: test(T t)                                  │
│     ↓                                                │
│  Output: boolean (true/false)                        │
│                                                      │
│  Example:                                            │
│  Predicate<Integer> isEven = x -> x % 2 == 0;        │
│                                                      │
│  isEven.test(4)  → true                              │
│  isEven.test(5)  → false                             │
│                                                      │
│  Chaining:                                           │
│  Predicate<Integer> isPositive = x -> x > 0;         │
│  isEven.and(isPositive).test(4)  → true              │
│  isEven.and(isPositive).test(-2) → false             │
│                                                      │
└──────────────────────────────────────────────────────┘
```

Primitive Specializations:

Interface	Method	Input	Output
IntPredicate	test(int)	int	boolean
LongPredicate	test(long)	long	boolean
DoublePredicate	test(double)	double	boolean

\

Example:

```
// Basic
Predicate<String> isEmpty = s -> s.isEmpty();
isEmpty.test("");  // true
// Chaining
Predicate<Integer> isEven = x -> x % 2 == 0;
Predicate<Integer> isPositive = x -> x > 0;
isEven.and(isPositive).test(4);   // true
isEven.or(isPositive).test(3);    // true
isEven.negate().test(4);        // false

// In Streams
list.stream().filter(isEven::test).collect(toList());
```

2.2 Consumer<T>

Definition: Takes one argument, returns **nothing (void)**. Used for side effects.

Method Signature:

```
java
@FunctionalInterface
public interface Consumer<T> {
  void accept(T t);
    // Default methods
  default Consumer<T> andThen(Consumer<? super T> after)
    // Static method
  static <T> Consumer<T> identity()
}
```

When to Use:

- Printing/logging
- Modifying external state
- Performing actions without return value
- forEach operations

Internal Working:

```
┌──────────────────────────────────────────────────────────┐
│                    CONSUMER INTERNAL                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Input: T (any type)                                     │
│     ↓                                                    │
│  Process: accept(T t)                                    │
│     ↓                                                    │
│  Output: void (no return)                                │
│                                                          │
│  Example:                                                │
│  Consumer<String> printer = s -> System.out.println(s);  │
│                                                          │
│  printer.accept("Hello");  // Prints "Hello"             │
│                                                          │
│  Chaining:                                               │
│  Consumer<String> prefix = s -> System.out.print("> ");  │
│  Consumer<String> combined = prefix.andThen(printer);    │
│  combined.accept("Hello");  // "> Hello"                 │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Primitive Specializations:

Interface	Method	Input	Output
IntConsumer	accept(int)	int	void
LongConsumer	accept(long)	long	void
DoubleConsumer	accept(double)	double	void

Example Usage:

```
// Basic
Consumer<String> printer = System.out::println;
printer.accept("Hello");
// Chaining
Consumer<Integer> add = list::add;
Consumer<Integer> log = x -> System.out.println("Added: " + x);
add.andThen(log).accept(5);
// In Streams
list.forEach(System.out::println);
```

2.3 Supplier<T>

Definition: Takes **no arguments**, returns a value. Used for lazy generation.

Method Signature:

2.3 Supplier<T>

Definition: Takes **no arguments**, returns a value. Used for lazy generation.

Method Signature:

```
@FunctionalInterface
public interface Supplier<T> {
  T get();
}
```

When to Use:

- Factory methods
- Lazy initialization

- Generating values on demand
- Providing default values

Internal Working:

```
┌─────────────────────────────────────────────────────┐
│                    SUPPLIER INTERNAL                 │
├─────────────────────────────────────────────────────┤
│                                                     │
│  Input: None                                        │
│     ↓                                               │
│  Process: get()                                     │
│     ↓                                               │
│  Output: T (any type)                               │
│                                                     │
│  Example:                                           │
│  Supplier<Double> random = () -> Math.random();     │
│                                                     │
│  random.get();  // 0.7234...                        │
│  random.get();  // 0.1823... (different each time)  │
│                                                     │
│  Lazy Evaluation:                                   │
│  Supplier<ExpensiveObject> lazy = () -> new ExpensiveObject();
│  // Object NOT created until get() is called        │
│                                                     │
└─────────────────────────────────────────────────────┘
```

Primitive Specializations:

Interface	Method	Input	Output
BooleanSupplier	getAsBoolean()	none	boolean
IntSupplier	getAsInt()	none	int
LongSupplier	getAsLong()	none	long
DoubleSupplier	getAsDouble()	none	double

.4 Built-in Functional Interfaces

Java 8 provides **40+ functional interfaces** in java.util.function package.

Core Interfaces:

Interface	Method	Parameters	Return	Use Case
Predicate<T>	test(T)	1	boolean	Filtering, conditions
Consumer<T>	accept(T)	1	void	Printing, logging
Function<T,R>	apply(T)	1	R	Transformation
Supplier<T>	get()	0	T	Factory, lazy loading
UnaryOperator<T>	apply(T)	1	T	Same type transformation
BinaryOperator<T>	apply(T,T)	2	T	Reduction operations

Example

```
// Basic
Supplier<LocalDate> today = LocalDate::now;
LocalDate date = today.get();
// Factory
Supplier<List<String>> listFactory = ArrayList::new;
List<String> list = listFactory.get();
// Lazy initialization
Supplier<Connection> dbConnection = () -> Database.connect();
// Connection created only when get() called
```

2.4 Function<T, R>

Definition: Takes one argument, returns a **different type** (or same type). Used for transformation.

Method Signature:

```
@FunctionalInterface
public interface Function<T, R> {
  R apply(T t);
    // Default methods
  default <V> Function<V, R> compose(Function<? super V, ? extends T> before)
  default <V> Function<T, V> andThen(Function<? super R, ? extends V> after)
    // Static method
  static <T> Function<T, T> identity()
}
```

When to Use:

- Transforming data
- Mapping operations
- Converting between types
- Any transformation logic

Internal Working:

```
┌──────────────────────────────────────────────────────────┐
│                     FUNCTION INTERNAL                    │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Input: T (input type)                                   │
│     ↓                                                    │
│  Process: apply(T t)                                     │
│     ↓                                                    │
│  Output: R (result type)                                 │
│                                                          │
│  Example:                                                │
│  Function<String, Integer> length = String::length;      │
│                                                          │
│  length.apply("Hello");  // 5                            │
│                                                          │
│  Composition:                                            │
│  Function<Integer, Integer> addOne = x -> x + 1;         │
│  Function<Integer, Integer> multiplyTwo = x -> x * 2;    │
│                                                          │
│  // compose: apply BEFORE function first                 │
│  addOne.compose(multiplyTwo).apply(5);  // (5*2)+1 = 11  │
│                                                          │
│  // andThen: apply AFTER function first                  │
│  addOne.andThen(multiplyTwo).apply(5);  // (5+1)*2 = 12  │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Primitive Specializations:

Interface	Method	Input	Output
IntFunction<R>	apply(int)	int	R
LongFunction<R>	apply(long)	long	R
DoubleFunction<R>	apply(double)	double	R
ToIntFunction<T>	applyAsInt(T)	T	int
ToLongFunction<T>	applyAsLong(T)	T	long
ToDoubleFunction<T>	applyAsDouble(T)	T	double

Example Usage:

```
// Basic transformation

Function<String, Integer> parse = Integer::parseInt;

Integer num = parse.apply("123");  // 123

// Chaining

Function<String, String> trim = String::trim;

Function<String, Integer> length = String::length;

Function<String, Integer> trimThenLength = trim.andThen(length);

trimThenLength.apply("  Hello  ");  // 5

// In Streams

list.stream().map(String::toUpperCase).collect(toList());
```

2.5 UnaryOperator<T>

Definition: Extends Function<T, T>. Takes one argument, returns **same type**.

Method Signature:

```
UnaryOperator<String> toUpper = String::toUpperCase;

String result = toUpper.apply("hello");  // "HELLO"

UnaryOperator<Integer> square = x -> x * x;

Integer squared = square.apply(5);  // 25
```

2.6 BinaryOperator<T>

Definition: Extends BiFunction<T, T, T>. Takes **two arguments of same type**, returns **same type**. Used for reduction.

Method Signature:

```
@FunctionalInterface

public interface BinaryOperator<T> extends BiFunction<T, T, T> {

   // Static methods

   static <T> BinaryOperator<T> minBy(Comparator<? super T> comparator)

   static <T> BinaryOperator<T> maxBy(Comparator<? super T> comparator)

}
```

When to Use:

- Reduction operations (sum, max, min)
- Combining two values
- Aggregate operations
- **Primitive Specializations:**

Interface	Method	Input	Output
`IntBinaryOperator`	`applyAsInt(int, int)`	int, int	int
`LongBinaryOperator`	`applyAsLong(long, long)`	long, long	long
`DoubleBinaryOperator`	`applyAsDouble(double, double)`	double, double	double

Example

```
// Basic

BinaryOperator<Integer> sum = (a, b) -> a + b;

Integer result = sum.apply(5, 3);  // 8
```

```
// Min/Max
BinaryOperator<Integer> min = BinaryOperator.minBy(Integer::compareTo);
min.apply(5, 3);  // 3
// In Streams
Optional<Integer> total = numbers.stream().reduce(sum);
```

2.7 BiFunction<T, U, R>

Definition: Takes **two arguments** (can be different types), returns a value.

Method Signature:

```
@FunctionalInterface
public interface BiFunction<T, U, R> {
  R apply(T t, U u);
    // Default method
  default <V> BiFunction<T, U, V> andThen(Function<? super R, ? extends V> after)
}
```

When to Use:

- Operations requiring two inputs
- Map merge functions
- Complex transformations

Example Usage:

```
BiFunction<String, Integer, String> combine = (s, n) -> s + n;
String result = combine.apply("Hello", 5);  // "Hello5"
// In toMap merge function
Map<String, Integer> map = list.stream()
  .collect(Collectors.toMap(
    key -> key,
    value -> 1,
    (existing, replacement) -> existing + replacement  // BiFunction
  ));
```

2.8 BiConsumer<T, U>

Definition: Takes **two arguments**, returns nothing (void).

Method Signature:

```
@FunctionalInterface
public interface BiConsumer<T, U> {
  void accept(T t, U u);
    // Default method
  default BiConsumer<T, U> andThen(BiConsumer<? super T, ? super U> after)
}
```

When to Use:

- Map forEach operations
- Two-parameter side effects
- Logging with key-value

Example Usage:

```
BiConsumer<String, Integer> printer = (k, v) -> System.out.println(k + "=" + v);
map.forEach(printer);
```

2.9 BiPredicate<T, U>

Definition: Takes **two arguments**, returns boolean.

Method Signature:

```
@FunctionalInterface
public interface BiPredicate<T, U> {
  boolean test(T t, U u);
}
```

When to Use:

- Two-parameter conditions
- Complex filtering
- Validation with multiple inputs

Example Usage:

```
BiPredicate<Integer, Integer> isGreater = (a, b) -> a > b;
isGreater.test(5, 3);  // true
```

Interface	Abstract Method	Parameters	Return	Use Case
Predicate<T>	test(T)	1 (T)	boolean	Filtering, conditions
Consumer<T>	accept(T)	1 (T)	void	Side effects, printing
Supplier<T>	get()	0	T	Factory, lazy loading
Function<T,R>	apply(T)	1 (T)	R	Transformation, mapping
UnaryOperator<T>	apply(T)	1 (T)	T	Same-type transformation
BinaryOperator<T>	apply(T,T)	2 (T,T)	T	Reduction, combining
BiFunction<T,U,R>	apply(T,U)	2 (T,U)	R	Two-input transformation
BiConsumer<T,U>	accept(T,U)	2 (T,U)	void	Two-input side effects
BiPredicate<T,U>	test(T,U)	2 (T,U)	boolean	Two-input conditions

Primitive Specializations

Why? To avoid **autoboxing overhead** (converting int ↔ Integer)

Primitive	Predicate	Consumer	Function	Supplier	UnaryOperator	BinaryOperator
int	IntPredicate	IntConsumer	IntFunction<R>	IntSupplier	IntUnaryOperator	IntBinary Operator
long	LongPredicate	LongConsumer	LongFunction<R>	LongSupplier	LongUnaryOperator	LongBinary Operator
double	DoublePredicate	DoubleConsumer	DoubleFunction<R>	DoubleSupplier	DoubleUnary operator	DoubleBinary Operator

Example:

```
// ❌ With boxing (slower)
Function<Integer, Integer> square = x -> x * x;
Integer result = square.apply(5);  // Boxes int to Integer


// ✅ Without boxing (faster)
```

```
IntUnaryOperator square = x -> x * x;

int result = square.applyAsInt(5);  // No boxing!
```

Part 3: Method References

3.1 What are Method References?

Definition: A **shorthand syntax for lambdas** that call a single method.

Syntax: ClassName::methodName

3.2 Why Use Method References?

- **More readable** than lambdas
- **Less verbose**
- **Clearer intent**

3.3 Types of Method References

```
// 1. Static method reference

// Lambda: (s) -> Integer.parseInt(s)

// Method reference:

Function<String, Integer> parser = Integer::parseInt;


// 2. Instance method reference (particular object)

// Lambda: () -> System.out.println("Hello")

// Method reference:

Runnable runnable = System.out::println;


// 3. Instance method reference (arbitrary object)

// Lambda: (s) -> s.toUpperCase()

// Method reference:

Function<String, String> upper = String::toUpperCase;


// 4. Constructor reference

// Lambda: () -> new ArrayList<>()

// Method reference:
```

```
Supplier<List<String>> listSupplier = ArrayList::new;

// Constructor with parameters
// Lambda: (s) -> new String(s)
// Method reference:
Function<String, String> stringCtor = String::new;
```

.4 When to Use Method References vs Lambdas

```
// ✅ Use method reference when:
list.forEach(System.out::println);  // Clear and concise

// ✅ Use lambda when:
list.forEach(s -> System.out.println("Value: " + s));  // Need customization

// ✅ Use method reference:
Function<String, Integer> parser = Integer::parseInt;

// ✅ Use lambda:
Function<String, Integer> parser = s -> Integer.parseInt(s.trim());  // Preprocessing
```

Part 4: Streams API

4.1 What is a Stream?

Definition: A **sequence of elements** supporting sequential and parallel aggregate operations. It not a data structure

Key Concepts:

- **Not a data structure** - doesn't store elements
- **Functional in nature** - operations don't modify the source
- **Lazy evaluation** - intermediate operations wait until terminal operation
- **Can be traversed only once**
- **Internal iteration :** stream handles iteration, not user

4.2 Why We Use Streams?

Problem with Collections	Solution with Streams
External iteration (for loops)	Internal iteration (declarative)
Sequential processing only	Easy parallelization
Complex nested loops	Readable pipeline
Hard to parallelize	Built-in parallel support

Analogy:

- **Collections:** Like a bucket of water (you carry it around)
- **Streams:** Like a pipeline (water flows through, you process it)

4.3 Stream vs Collection

Feature	Collection	Stream
Storage	Stores data	Doesn't store data
Iteration	External (for loop)	Internal (forEach)
Traversal	Multiple times	**Once only**
Processing	Eager	**Lazy**
Mutability	Can modify	**Read-only** (functional)

4.4 Internal Working of Streams

```
┌──────────────────────────────────────────────────────────┐
│                      STREAM PIPELINE                     │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  ┌─────────┐   ┌──────────────┐   ┌──────────────┐      │
│  │ SOURCE  │ → │ INTERMEDIATE │ → │   TERMINAL   │      │
│  │         │   │ (Lazy, Multiple)│ │ (Eager, 1)   │      │
│  └─────────┘   └──────────────┘   └──────────────┘      │
│       ↓                ↓                  ↓              │
│  Collection       filter, map        collect, count     │
│  Array            sorted, limit      forEach, reduce    │
│  Generator        distinct, skip     anyMatch, find     │
│                                                          │
│  Example:                                                │
│  list.stream()                                           │
│      .filter(x -> x > 10)    ← Intermediate (lazy)       │
│      .map(x -> x * 2)        ← Intermediate (lazy)       │
│      .collect(toList())      ← Terminal (triggers all)   │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

```
┌──────────────────────────────────────────────────────────┐
│                  Stream Pipeline Architecture            │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Source → Intermediate Operations → Terminal Operation   │
│                                                          │
│  Example:                                                │
│  list.stream()                                           │
│      .filter(x -> x > 10)       ← Lazy (not executed)    │
│      .map(x -> x * 2)           ← Lazy (not executed)    │
│      .collect(toList())         ← Eager (triggers all)   │
│                                                          │
│  Internal Process:                                       │
│  ┌──────────────────────────────────────┐                │
│  │ 1. Source (Collection)               │                │
│  │    ↓                                 │                │
│  │ 2. Spliterator (divides data)        │                │
│  │    ↓                                 │                │
│  │ 3. Pipeline (chain of operations)    │                │
│  │    - Each operation wraps previous   │                │
│  │    - Creates pipeline stages         │                │
│  │    ↓                                 │                │
│  │ 4. Terminal Operation triggers:      │                │
│  │    - Pulls elements one by one       │                │
│  │    - Passes through all stages       │                │
│  │    - Applies optimizations (short-circuit)            │
│  │    ↓                                 │                │
│  │ 5. Result returned                   │                │
│  └──────────────────────────────────────┘                │
│                                                          │
│  Lazy Evaluation Example:                                │
│  list.stream()                                           │
│      .filter(x -> {                                      │
│          System.out.println("Filter: " + x);             │
│          return x > 2;                                   │
│      })                                                  │
│      .map(x -> {                                         │
│          System.out.println("Map: " + x);                │
│          return x * 2;                                   │
│      })                                                  │
│      .findFirst();  // Only processes first match!       │
│                                                          │
│  Output (for list [1,2,3,4,5]):                          │
│  Filter: 1                                               │
│  Filter: 2                                               │
│  Filter: 3                                               │
│  Map: 3                                                  │
│  (Stops! findFirst() short-circuits)                     │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

2. Creating Streams (Source Methods)

Method	Description	Example
stream()	Sequential stream from collection	list.stream()
parallelStream()	Parallel stream from collection	list.parallelStream()
Stream.of()	Stream from values	Stream.of(1,2,3)
Arrays.stream()	Stream from array	Arrays.stream(arr)
Stream.generate()	Infinite stream (Supplier)	Stream.generate(Math::random)
Stream.iterate()	Infinite stream (seed + function)	Stream.iterate(0, n -> n+1)
IntStream.range()	Range of ints (exclusive end)	IntStream.range(0, 10)
IntStream.rangeClosed()	Range of ints (inclusive end)	IntStream.rangeClosed(0, 10)

Method	Description	Example
Stream.empty()	Empty stream	Stream.empty()
Files.lines()	Stream from file	Files.lines(path)
Pattern.splitAsStream()	Stream from regex split	pattern.splitAsStream(str)
Collection.stream()	From any collection	set.stream()
Map.keySet().stream()	From map keys	map.keySet().stream()
Map.values().stream()	From map values	map.values().stream()
Map.entrySet().stream()	From map entries	map.entrySet().stream()

---

3. Intermediate Operations (Lazy)

3.1 Filtering Operations

Method	Signature	Description	Returns
filter	filter(Predicate<T>)	Keeps elements matching predicate	Stream<T>
distinct	distinct()	Removes duplicates (uses hashCode/equals)	Stream<T>
limit	limit(long maxSize)	Takes first N elements	Stream<T>
skip	skip(long n)	Skips first N elements	Stream<T>

Example:

```
// filter - Predicate-based filtering
Stream<Integer> filtered = numbers.stream()
    .filter(n -> n % 2 == 0);  // Keep only even numbers


// Internal: Pulls elements one by one, applies predicate,
// passes only matching elements downstream


// distinct - Remove duplicates
Stream<String> unique = words.stream()
    .distinct();  // Uses hashCode() and equals()
```

```
// Internal: Maintains a Set of seen elements,
// filters out duplicates

// limit - Short-circuiting operation
Stream<Integer> limited = numbers.stream()
   .limit(5);  // Take only first 5

// Internal: Counts elements, stops after N
// skip - Skip elements
Stream<Integer> skipped = numbers.stream()
   .skip(3);  // Skip first 3
// Internal: Counts elements, ignores first N
```

3.2 Transformation Operations

Method	Signature	Description	Returns
map	map(Function<T,R>)	Transforms each element	Stream<R>
mapToInt	mapToInt(ToIntFunction<T>)	Maps to IntStream	IntStream
mapToLong	mapToLong(ToLongFunction<T>)	Maps to LongStream	LongStream
mapToDouble	mapToDouble(ToDoubleFunction<T>)	Maps to DoubleStream	DoubleStream
flatMap	flatMap(Function<T,Stream<R>>)	Flattens nested streams	Stream<R>
flatMapToInt	flatMapToInt(Function<T,IntStream>)	Flattens to IntStream	IntStream
flatMapToLong	flatMapToLong(Function<T,LongStream>)	Flattens to LongStream	LongStream
flatMapToDouble	flatMapToDouble(Function<T,DoubleStream>)	Flattens to DoubleStream	DoubleStream

Detailed Explanation:

```
// map - One-to-one transformation
Stream<String> upper = names.stream()
   .map(String::toUpperCase);  // Transform each name

// Internal: Applies function to each element,
// passes result downstream

// mapToInt - Convert to primitive stream
IntStream lengths = names.stream()
   .mapToInt(String::length);  // Avoids boxing
// Internal: Same as map but returns primitive stream
// More efficient (no Integer objects)
// flatMap - One-to-many transformation
List<List<String>> nested = Arrays.asList(
   Arrays.asList("a", "b"),
   Arrays.asList("c", "d")
);
Stream<String> flat = nested.stream()
   .flatMap(List::stream);  // ["a","b","c","d"]

// Internal:
// 1. Applies function (returns Stream for each element)
// 2. Flattens all resulting streams into one
// 3. Passes flattened elements downstream

// Visual:
// Input:  [[a,b], [c,d]]
// After function: [Stream(a,b), Stream(c,d)]
// After flatMap: [a, b, c, d]
```

3.3 Sorting Operations

Method	Signature	Description	Returns
sorted	sorted()	Natural order sorting	Stream<T>
sorted	sorted(Comparator<T>)	Custom order sorting	Stream<T>

```
// Natural order (elements must implement Comparable)
Stream<String> sorted = names.stream()
    .sorted();  // Alphabetical
// Custom order
Stream<Integer> sorted = numbers.stream()
    .sorted((a, b) -> b - a);  // Descending


// Internal:
// 1. Buffers ALL elements (not lazy!)
// 2. Sorts using Arrays.sort (TimSort)
// 3. Emits sorted elements downstream
//
// WARNING: sorted() is NOT lazy - must process all elements
// before emitting any. Can be expensive for large streams.
```

3.4 Peek Operation

Method	Signature	Description	Returns
peek	peek(Consumer<T>)	Performs action on each element (debugging)	Stream<T>

```
// Debugging - see elements at pipeline stage
numbers.stream()
    .peek(n -> System.out.println("Before filter: " + n))
    .filter(n -> n > 5)
    .peek(n -> System.out.println("After filter: " + n))
    .collect(toList());
```

```
// Internal:
// - Passes element through unchanged
// - Executes consumer as side effect
// - Useful for debugging, NOT for modification
//
// WARNING: Don't use peek for state modification!
// It's meant for debugging only.
```

4.5 Stream Operations

Intermediate Operations (Lazy)

```
// 1. filter - Predicate
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
numbers.stream()
   .filter(n -> n % 2 == 0)  // [2, 4]
   .collect(Collectors.toList());

// 2. map - Transformation
List<String> names = Arrays.asList("alice", "bob", "charlie");
names.stream()
   .map(String::toUpperCase)  // ["ALICE", "BOB", "CHARLIE"]
   .collect(Collectors.toList());

// 3. flatMap - Flatten nested structures
List<List<String>> nested = Arrays.asList(
   Arrays.asList("a", "b"),
   Arrays.asList("c", "d")
);
nested.stream()
   .flatMap(List::stream)  // ["a", "b", "c", "d"]
   .collect(Collectors.toList());
```

```
// 4. distinct - Remove duplicates
List<Integer> nums = Arrays.asList(1, 2, 2, 3, 3, 3);
nums.stream()
   .distinct()  // [1, 2, 3]
   .collect(Collectors.toList());



// 5. sorted - Sorting
names.stream()
   .sorted()  // Natural order
   .collect(Collectors.toList());

names.stream()
   .sorted((a, b) -> b.length() - a.length())  // Custom
   .collect(Collectors.toList());
// 6. limit - Take first N
numbers.stream()
   .limit(3)  // [1, 2, 3]
   .collect(Collectors.toList());
// 7. skip - Skip first N
numbers.stream()
   .skip(2)  // [3, 4, 5]
   .collect(Collectors.toList());
// 8. peek - Debugging
numbers.stream()
   .peek(n -> System.out.println("Before: " + n))
   .filter(n -> n > 2)    .peek(n -> System.out.println("After filter: " + n))
   .collect(Collectors.toList());
```

Terminal Operations (Eager)

4.1 Reduction Operations

Method	**Signature**	**Description**	**Returns**
reduce	reduce(T identity, BinaryOperator<T>)	Reduce with identity	T
reduce	reduce(BinaryOperator<T>)	Reduce without identity	Optional<T>
reduce	reduce(U identity, BiFunction<U,T,U>, BinaryOperator<U>)	Reduce with combiner	U
collect	collect(Collector<T,A,R>)	Mutable reduction	R
collect	collect(Supplier<R>, BiConsumer<R,T>, BiConsumer<R,R>)	Custom collection	R

4.2 Matching Operations (Short-Circuiting)

Method	Signature	Description	Returns
anyMatch	anyMatch(Predicate<T>)	True if ANY element matches	boolean
allMatch	allMatch(Predicate<T>)	True if ALL elements match	boolean
noneMatch	noneMatch(Predicate<T>)	True if NO elements match	boolean

Finding operations

Method	**Signature**	**Description**	**Returns**
findFirst	findFirst()	First element (ordered)	Optional<T>
findAny	findAny()	Any element (parallel-friendly)	Optional<T>

4.4 Counting Operation

Method	**Signature**	**Description**	**Returns**
count	count()	Count elements	long

4.5 ForEach Operation

Method	**Signature**	**Description**	**Returns**
forEach	forEach(Consumer<T>)	Perform action on each element	void

Method	Signature	Description	Returns
forEachOrdered	forEachOrdered(Consumer<T>)	Perform action in encounter order	void

4.6 toArray Operations

Method	Signature	Description	Returns
toArray	toArray()	Object array	Object[]
toArray	toArray(IntFunction<A[]>)	Typed array	A[]

4.7 Min/Max Operations

Method	Signature	Description	Returns
min	min(Comparator<T>)	Minimum element	Optional<T>
max	max(Comparator<T>)	Maximum element	Optional<T>

```
// 1. forEach - Side effects
names.forEach(System.out::println);

// 2. collect - Gather results
List<String> result = names.stream()
    .collect(Collectors.toList());

Set<String> set = names.stream()
    .collect(Collectors.toSet());

Map<String, Integer> map = names.stream()
    .collect(Collectors.toMap(
        String::toLowerCase,
        String::length
    ));
```

```
// 3. reduce - Aggregation
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);

// Sum
Integer sum = nums.stream()
    .reduce(0, (a, b) -> a + b);  // 15

// Max
Optional<Integer> max = nums.stream()
    .reduce(Integer::max);  // 5

// Concatenate strings
List<String> words = Arrays.asList("Hello", "World");
String joined = words.stream()
    .reduce("", (a, b) -> a + " " + b);  // "Hello World"

// 4. count
long count = names.stream().count();

// 5. anyMatch, allMatch, noneMatch
boolean hasEven = nums.stream()
    .anyMatch(n -> n % 2 == 0);  // true

boolean allPositive = nums.stream()
    .allMatch(n -> n > 0);  // true

boolean noneNegative = nums.stream()
    .noneMatch(n -> n < 0);  // true

// 6. findFirst, findAny
```

```
Optional<String> first = names.stream()
    .findFirst();

Optional<String> any = names.parallelStream()
    .findAny();

// 7. toArray
String[] array = names.stream()
    .toArray(String[]::new);

// 8. min, max
Optional<Integer> min = nums.stream()
    .min(Integer::compareTo);

Optional<Integer> max2 = nums.stream()
    .max(Integer::compareTo);
```

4.6 Collectors - Advanced

```
// 1. Grouping
List<Person> people = Arrays.asList(
    new Person("Alice", 25, "HR"),
    new Person("Bob", 30, "IT"),
    new Person("Charlie", 25, "HR")
);

// Group by department
Map<String, List<Person>> byDept = people.stream()
    .collect(Collectors.groupingBy(Person::getDepartment));
```

```
// Group by age
Map<Integer, List<Person>> byAge = people.stream()
    .collect(Collectors.groupingBy(Person::getAge));

// 2. Partitioning (true/false split)
Map<Boolean, List<Person>> adults = people.stream()
    .collect(Collectors.partitioningBy(p -> p.getAge() >= 18));

// 3. Mapping collector
Map<String, Set<Integer>> deptAges = people.stream()
    .collect(Collectors.groupingBy(
        Person::getDepartment,
        Collectors.mapping(Person::getAge, Collectors.toSet())
    ));

// 4. Joining
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
String joined = names.stream()
    .collect(Collectors.joining(", "));  // "Alice, Bob, Charlie"

String withPrefix = names.stream()
    .collect(Collectors.joining(", ", "[", "]"));  // "[Alice, Bob, Charlie]"

// 5. Summarizing statistics
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);
IntSummaryStatistics stats = numbers.stream()
    .mapToInt(Integer::intValue)
    .summaryStatistics();

System.out.println("Count: " + stats.getCount());
```

```
System.out.println("Sum: " + stats.getSum());

System.out.println("Min: " + stats.getMin());

System.out.println("Max: " + stats.getMax());

System.out.println("Avg: " + stats.getAverage());


// 6. toMap with merge function

List<String> words = Arrays.asList("a", "b", "a", "c");

Map<String, Integer> wordCount = words.stream()

  .collect(Collectors.toMap(

    word -> word,

    word -> 1,

    (existing, replacement) -> existing + replacement  // Merge function

  ));  // {a=2, b=1, c=1}
```

4.7 Parallel Streams

1. What is a Parallel Stream?

1.1 Definition

A Parallel Stream is a stream that processes elements using multiple threads from the Fork/Join pool, enabling automatic parallelization of operations.

1.2 Sequential vs Parallel

```
┌──────────────────────────────────────────────────────────┐
│            SEQUENTIAL vs PARALLEL STREAMS                 │
├──────────────────────────────────────────────────────────┤
│                                                           │
│  SEQUENTIAL STREAM:                                       │
│  ┌───┐  ┌───┐  ┌───┐  ┌───┐  ┌───┐                      │
│  │ 1 │→│ 2 │→│ 3 │→│ 4 │→│ 5 │   (Single thread)      │
│  └───┘  └───┘  └───┘  └───┘  └───┘                      │
│                                                           │
│  PARALLEL STREAM:                                         │
│  ┌───┐  ┌───┐       ┌───┐  ┌───┐  ┌───┐                 │
│  │ 1 │  │ 2 │       │ 3 │  │ 4 │  │ 5 │   (Multiple     │
│  └───┘  └───┘       └───┘  └───┘  └───┘    threads)      │
│    ↓      ↓           ↓      ↓      ↓                    │
│  ┌──────────┐    ┌──────────────┐                        │
│  │ Thread 1 │    │ Thread 2     │                        │
│  │ Process 1,2│  │ Process 3,4,5 │                       │
│  └──────────┘    └──────────────┘                        │
│          ↓                  ↓                             │
│  ┌──────────────────────────┐                             │
│  │        Combine Results   │                             │
│  └──────────────────────────┘                             │
│                                                           │
```

```
// Sequential stream
List<Integer> numbers = IntStream.rangeClosed(1, 1000)
    .boxed()
    .collect(Collectors.toList());

// Sequential processing
long start = System.currentTimeMillis();
numbers.stream()
    .map(n -> {
        try { Thread.sleep(10); } catch (Exception e) {}
        return n * 2;
    })
    .collect(Collectors.toList());
// Takes ~10 seconds

// Parallel processing
start = System.currentTimeMillis();
numbers.parallelStream()
    .map(n -> {
        try { Thread.sleep(10); } catch (Exception e) {}
        return n * 2;
    })
    .collect(Collectors.toList());
// Takes ~1-2 seconds (uses multiple cores)

// When to use parallel streams:
// ✅ Large datasets (>10,000 elements)
// ✅ CPU-intensive operations
```

// ✅ Independent operations

// ❌ Small datasets (overhead > benefit)

// ❌ I/O operations

// ❌ Order-dependent operations

```
// Convert between sequential and parallel
numbers.stream().parallel();     // Sequential → Parallel
numbers.parallelStream().sequential();  // Parallel → Sequential
```

1.3 How Parallel Streams Work Internally

```
┌──────────────────────────────────────────────┐
│            PARALLEL STREAM INTERNAL WORKING  │
├──────────────────────────────────────────────┤
│                                              │
│ 1. Source Splitting (Spliterator)            │
│    ┌──────────────────────────┐              │
│    │ [1, 2, 3, 4, 5, 6, 7, 8] │              │
│    └──────────────────────────┘              │
│         ↓                  ↓                 │
│    ┌─────────┐        ┌─────────┐           │
│    │ [1,2,3,4] │      │ [5,6,7,8] │          │
│    └─────────┘        └─────────┘           │
│                                              │
│ 2. Parallel Processing (Fork/Join Pool)      │
│    Thread 1: Process [1,2,3,4]               │
│    Thread 2: Process [5,6,7,8]               │
│                                              │
│ 3. Combine Results                           │
│    ┌─────────┐        ┌─────────┐           │
│    │ Result 1 │       │ Result 2 │          │
│    └─────────┘        └─────────┘           │
│         ↓                  ↓                 │
│    ┌──────────────────────────┐              │
│    │       Combined Result    │              │
│    └──────────────────────────┘              │
│                                              │
│ 4. Default Thread Pool                       │
│    - Uses ForkJoinPool.commonPool()          │
│    - Threads = CPU cores - 1                 │
│    - Can be customized                       │
└──────────────────────────────────────────────┘
```

4.8 Common Pitfalls

// ❌ Pitfall 1: Reusing streams

```
Stream<Integer> stream = numbers.stream();
stream.count();  // OK
stream.count();  // ❌ IllegalStateException (stream already operated)
// ✅ Solution: Create new stream
numbers.stream().count();
numbers.stream().count();  // OK
```

```
// ❌ Pitfall 2: Modifying source during stream
List<String> list = new ArrayList<>(Arrays.asList("a", "b", "c"));
list.stream()
  .forEach(s -> list.add(s.toUpperCase()));  // ❌ ConcurrentModificationException

// ✅ Solution: Collect to new list
List<String> result = list.stream()
  .map(String::toUpperCase)
  .collect(Collectors.toList());

// ❌ Pitfall 3: Side effects in intermediate operations
List<Integer> sum = new ArrayList<>();
numbers.stream()
  .map(n -> {
    sum.add(n);  // ❌ Bad practice (stateful lambda)
    return n * 2;
  });

// ✅ Solution: Use terminal operations
List<Integer> sum2 = numbers.stream()
  .map(n -> n * 2)
  .collect(Collectors.toList());

// ❌ Pitfall 4: Forgetting terminal operation
numbers.stream()
  .filter(n -> n > 5);  // ❌ Does nothing (lazy, no terminal op)
```

```
// ✅ Solution: Add terminal operation
numbers.stream()
  .filter(n -> n > 5)
  .collect(Collectors.toList());  // OK

// ❌ Pitfall 5: Using parallel stream incorrectly
List<Integer> nums = Arrays.asList(1, 2, 3, 4, 5);
nums.parallelStream()
  .map(n -> {
    System.out.println(Thread.currentThread().getName());
    return n * 2;
  })
  .collect(Collectors.toList());
// Output order not guaranteed!

// ✅ Solution: Use sequential if order matters
nums.stream()  // Sequential maintains order
  .map(n -> n * 2)
  .collect(Collectors.toList());
```

2. When to Use Parallel Streams

2.1 Good Use Cases

Scenario	Why Parallel Works
Large datasets (>10,000 elements)	Splitting overhead is worth it
CPU-intensive operations	Multiple cores utilized
Independent operations	No shared state between elements
Stateless operations	No side effects
Associative reduction	(a+b)+c = a+(b+c)

2.2 Bad Use Cases

Scenario	Why Parallel Fails
Small datasets (<1,000 elements)	Splitting overhead > benefit
I/O operations	Threads block on I/O
Order-dependent operations	Parallel doesn't guarantee order
Stateful operations	Race conditions
Non-associative reduction	Results may differ

Parallel Stream Methods Reference

Method	Description	Returns
parallel()	Convert to parallel stream	Stream<T>
sequential()	Convert to sequential stream	Stream<T>
isParallel()	Check if parallel	boolean
parallelStream()	Create parallel from collection	Stream<T>
spliterator()	Get spliterator (for splitting)	Spliterator<T>

PART 4: OPTIONAL CLASS - COMPLETE METHOD REFERENCE

1. What is Optional?

1.1 Definition

Optional is a container object that may or may not contain a non-null value. It explicitly represents the absence of a value.

1.2 Purpose

Problem	Solution
NullPointerException	Explicit absence handling
Unclear API (can return null?)	Optional signals "may be absent"
Verbose null checks	Declarative handling
Returning null from methods	Return Optional.empty()

---

2. Creating Optional

Method	Description	Behavior
Optional.empty()	Create empty Optional	Always empty
Optional.of(T)	Create with non-null value	Throws NPE if null
Optional.ofNullable(T)	Create with nullable value	Empty if null, of if not null

Problem Before Java 8:

```
public String getMiddleName() {
   return middleName;  // Could be null
}

// Caller must check
String middle = person.getMiddleName();
if (middle != null) {
   System.out.println(middle);
}
```

Solution of optional

```
public Optional<String> getMiddleName() {
   return Optional.ofNullable(middleName);  // Explicitly says "may be absent"
}
// Caller handles absence
person.getMiddleName()
   .ifPresent(System.out::println);  // Only prints if present
```

Important: Optional is **NOT** meant to be used as:

- ❌ Field/instance variable
- ❌ Method parameter
- ❌ In collections (use List<Optional> sparingly)

5.3 Creating Optional

```
// 1. Empty Optional
Optional<String> empty = Optional.empty();
// 2. Non-null value
Optional<String> present = Optional.of("Hello");
// Optional.of(null) → NullPointerException!

// 3. Nullable value
String value = getNullableValue();
Optional<String> optional = Optional.ofNullable(value);
// If value is null → Optional.empty()
// If value is not null → Optional.of(value)
```

5.4 Optional Methods

```
Optional<String> opt = Optional.of("Hello");

// 1. isPresent - Check if value exists
if (opt.isPresent()) {
   System.out.println(opt.get());
}

// 2. ifPresent - Execute if present
opt.ifPresent(value -> System.out.println(value));
opt.ifPresent(System.out::println);  // Method reference

// 3. ifPresentOrElse - Java 9+
opt.ifPresentOrElse(
   value -> System.out.println("Present: " + value),
   () -> System.out.println("Absent")
);
```

```
// 4. get - Get value (throws if empty)
String value = opt.get();  // "Hello"
Optional.empty().get();  // ❌ NoSuchElementException

// 5. orElse - Default value
String result = opt.orElse("Default");  // "Hello"
Optional.empty().orElse("Default");  // "Default"

// 6. orElseGet - Lazy default
String result2 = opt.orElseGet(() -> "Computed Default");
// Only computes if empty

// 7. orElseThrow - Throw custom exception
String result3 = opt.orElseThrow(() -> new IllegalStateException("Missing value"));

// 8. filter - Filter value
Optional<String> filtered = opt.filter(s -> s.length() > 3);
// Optional.empty() if predicate false

// 9. map - Transform value
Optional<Integer> length = opt.map(String::length);  // Optional[5]

// 10. flatMap - Transform to Optional
Optional<String> upper = opt.flatMap(s -> Optional.of(s.toUpperCase()));

// 11. stream - Java 9+
Stream<String> stream = opt.stream();  // Stream of 0 or 1 element
```

5.5 Optional Best Practices

```
// ❌ WRONG: Using Optional as field
public class Person {
   private Optional<String> name;  // ❌ Don't do this
}
// ✅ RIGHT: Use Optional in return types
public class PersonService {
   public Optional<Person> findById(Long id) {
      return repository.findById(id);
   }
}

// ❌ WRONG: Checking isPresent then get
Optional<String> opt = getValue();
if (opt.isPresent()) {
   process(opt.get());
}

// ✅ RIGHT: Use ifPresent
opt.ifPresent(this::process);

// ✅ BETTER: Use map/flatMap
Optional<Integer> result = opt
   .map(String::length)
   .filter(len -> len > 0);

// ❌ WRONG: Returning null instead of Optional.empty
public Optional<String> findName() {
   if (notFound) {
```

```
        return null;  // ❌
    }
    return Optional.of(name);
}

// ✅ RIGHT
public Optional<String> findName() {
    if (notFound) {
        return Optional.empty();  // ✅
    }
    return Optional.of(name);
}

// ❌ WRONG: Optional in collections
List<Optional<String>> list = new ArrayList<>();  // ❌ Avoid

// ✅ RIGHT: Filter nulls
List<String> list = items.stream()
    .filter(Objects::nonNull)
    .collect(Collectors.toList());
```

5.6 Optional with Streams

```
List<Optional<String>> optionals = Arrays.asList(
    Optional.of("A"),
    Optional.empty(),
    Optional.of("B")
);

// Filter out empty optionals
```

```
List<String> values = optionals.stream()
    .filter(Optional::isPresent)
    .map(Optional::get)
    .collect(Collectors.toList());  // ["A", "B"]

// Better: Use flatMap
List<String> values2 = optionals.stream()
    .flatMap(Optional::stream)  // Java 9+
    .collect(Collectors.toList());  // ["A", "B"]
```

Complete Optional Methods Table

Category	Method	Parameters	Returns	Use Case
Creation	empty()	none	Optional<T>	Create empty
Creation	of(T)	T (non-null)	Optional<T>	Create with value
Creation	ofNullable(T)	T (nullable)	Optional<T>	Create safely
Query	isPresent()	none	boolean	Check existence
Query	isEmpty()	none	boolean	Check empty (Java 11+)
Access	get()	none	T	Get value (throws if empty)
Access	orElse(T)	T default	T	Get or default
Access	orElseGet(Supplier)	Supplier	T	Get or lazy default
Access	orElseThrow()	none	T	Get or throw
Access	orElseThrow(Supplier)	Supplier	T	Get or throw custom
Action	ifPresent(Consumer)	Consumer	void	Execute if present
Action	ifPresentOrElse	Consumer, Runnable	void	Execute or else (Java 9+)
Transform	filter(Predicate)	Predicate	Optional<T>	Filter value
Transform	map(Function)	Function	Optional<U>	Transform value
Transform	flatMap(Function)	Function	Optional<U>	Flatten Optional
Transform	stream()	none	Stream<T>	Convert to Str

Part 6: Date/Time API (java.time)

6.1 Why New Date/Time API?

Problems with old Date/Calendar:

- ❌ Mutable (thread-unsafe)
- ❌ Poor API design (months start from 0)
- ❌ Timezone handling is confusing
- ❌ No clarity between instant, date, time

Solutions with java.time:

- ✅ Immutable (thread-safe)
- ✅ Clear, fluent API
- ✅ Proper timezone support
- ✅ Separation of concerns
-

6.2 Core Classes

```
// 1. LocalDate - Date without time
LocalDate date = LocalDate.now();
LocalDate specific = LocalDate.of(2024, 3, 15);
LocalDate fromString = LocalDate.parse("2024-03-15");

// 2. LocalTime - Time without date
LocalTime time = LocalTime.now();
LocalTime specific = LocalTime.of(14, 30, 0);

// 3. LocalDateTime - Date and time
LocalDateTime dateTime = LocalDateTime.now();
LocalDateTime specific = LocalDateTime.of(2024, 3, 15, 14, 30);

// 4. Instant - Point in time (Unix timestamp)
Instant now = Instant.now();
```

```
Instant fromEpoch = Instant.ofEpochSecond(1000);

// 5. ZonedDateTime - Date/time with timezone
ZonedDateTime zoned = ZonedDateTime.now(ZoneId.of("America/New_York"));

// 6. Duration - Time-based amount
Duration duration = Duration.between(time1, time2);

// 7. Period - Date-based amount
Period period = Period.between(date1, date2);
```

6.3 Common Operations

```
// Adding/subtracting
LocalDate date = LocalDate.of(2024, 3, 15);
date.plusDays(10);     // 2024-03-25
date.minusMonths(2);    // 2024-01-15
date.plus(1, ChronoUnit.YEARS);  // 2025-03-15
// Comparing
LocalDate d1 = LocalDate.of(2024, 1, 1);
LocalDate d2 = LocalDate.of(2024, 12, 31);
d1.isBefore(d2);  // true
d1.isAfter(d2);   // false
d1.isEqual(d2);   // false
// Formatting
DateTimeFormatter formatter = DateTimeFormatter.ofPattern("dd/MM/yyyy");
String formatted = date.format(formatter);  // "15/03/2024"
LocalDate parsed = LocalDate.parse("15/03/2024", formatter);

// Extracting parts
int year = date.getYear();
```

```
Month month = date.getMonth();  // MARCH

int day = date.getDayOfMonth();

DayOfWeek dow = date.getDayOfWeek();  // FRIDAY
```

DATE/TIME API - COMPLETE METHOD REFERENCE

1. Core Classes

Class	Package	Description	Immutable
LocalDate	java.time	Date without time	✅ Yes
LocalTime	java.time	Time without date	✅ Yes
LocalDateTime	java.time	Date and time	✅ Yes
Instant	java.time	Point in time (epoch)	✅ Yes
ZonedDateTime	java.time	Date/time with timezone	✅ Yes
Duration	java.time	Time-based amount	✅ Yes
Period	java.time	Date-based amount	✅ Yes
ZoneId	java.time	Timezone identifier	✅ Yes
DateTimeFormatter	java.time.format	Date/time formatting	✅ Yes

---

2. LocalDate Methods

2.1 Creation Methods

Method	Description	Example
now()	Current date	LocalDate.now()
of(year, month, day)	Specific date	LocalDate.of(2024, 3, 15)
parse(CharSequence)	Parse from string	LocalDate.parse("2024-03-15")
parse(CharSequence, formatter)	Parse with formatter	LocalDate.parse("15/03/2024", formatter)
ofEpochDay(long)	From epoch day	LocalDate.ofEpochDay(0)
ofYearDay(year, dayOfYear)	From year and day	LocalDate.ofYearDay(2024, 100)

.2 Get Methods

Method	Returns	Example
getYear()	int	2024
getMonth()	Month enum	MARCH
getMonthValue()	int (1-12)	3
getDayOfMonth()	int (1-31)	15
getDayOfYear()	int (1-366)	75
getDayOfWeek()	DayOfWeek enum	FRIDAY
getLong(TemporalField)	long	date.getLong(ChronoField.EPOCH_DAY)

2.3 With Methods (Immutable Modifications)

Method	Description	Example
withYear(int)	Change year	date.withYear(2025)
withMonth(int)	Change month	date.withMonth(12)
withDayOfMonth(int)	Change day	date.withDayOfMonth(1)
with(TemporalAdjuster)	Custom adjustment	date.with(nextMonday())
withDayOfWeek(DayOfWeek)	Change day of week	date.withDayOfWeek(MONDAY)

2.4 Plus/Minus Methods

Method	Description	Example
plusDays(long)	Add days	date.plusDays(10)
plusWeeks(long)	Add weeks	date.plusWeeks(2)
plusMonths(long)	Add months	date.plusMonths(1)
plusYears(long)	Add years	date.plusYears(1)
plus(Period)	Add period	date.plus(Period.ofDays(5))
plus(long, TemporalUnit)	Add with unit	date.plus(1, ChronoUnit.YEARS)
minusDays(long)	Subtract days	date.minusDays(10)
minusWeeks(long)	Subtract weeks	date.minusWeeks(2)

Method	Description	Example
minusMonths(long)	Subtract months	date.minusMonths(1)
minusYears(long)	Subtract years	date.minusYears(1)

2.5 Comparison Methods

Method	Returns	Description
isBefore(LocalDate)	boolean	Check if before
isAfter(LocalDate)	boolean	Check if after
isEqual(LocalDate)	boolean	Check if equal
compareTo(LocalDate)	int	Compare (negative/zero/positive)
isLeapYear()	boolean	Check leap year

2.6 Other Methods

Method	Returns	Description
lengthOfMonth()	int	Days in month
lengthOfYear()	int	Days in year (365/366)
atStartOfDay()	LocalDateTime	LocalDateTime at midnight
atTime(LocalTime)	LocalDateTime	Combine with time
until(LocalDate)	Period	Period between dates
toEpochDay()	long	Days since epoch

---

3. LocalTime Methods

3.1 Creation Methods

Method	Description	Example
now()	Current time	LocalTime.now()
of(hour, minute)	Specific time	LocalTime.of(14, 30)
of(hour, minute, second)	With seconds	LocalTime.of(14, 30, 0)
of(hour, minute, second, nano)	With nanoseconds	LocalTime.of(14, 30, 0, 0)
parse(CharSequence)	Parse from string	LocalTime.parse("14:30:00")

3.2 Get Methods

Method	Returns	Example
getHour()	int (0-23)	14
getMinute()	int (0-59)	30
getSecond()	int (0-59)	0
getNano()	int (0-999999999)	0

3.3 Plus/Minus Methods

Method	Description	Example
plusHours(long)	Add hours	time.plusHours(2)
plusMinutes(long)	Add minutes	time.plusMinutes(30)
plusSeconds(long)	Add seconds	time.plusSeconds(10)
plusNanos(long)	Add nanoseconds	time.plusNanos(1000)
minusHours(long)	Subtract hours	time.minusHours(2)
minusMinutes(long)	Subtract minutes	time.minusMinutes(30)

---

4. LocalDateTime Methods

4.1 Creation Methods

Method	Description	Example
now()	Current date-time	LocalDateTime.now()
of(LocalDate, LocalTime)	Combine date and time	LocalDateTime.of(date, time)
of(year, month, day, hour, minute)	Specific date-time	LocalDateTime.of(2024, 3, 15, 14, 30)
parse(CharSequence)	Parse from string	LocalDateTime.parse("2024-03-15T14:30:00")

4.2 Conversion Methods

Method	Returns	Description
toLocalDate()	LocalDate	Extract date
toLocalTime()	LocalTime	Extract time
atZone(ZoneId)	ZonedDateTime	Add timezone
toInstant(ZoneOffset)	Instant	Convert to instant

---

5. Duration and Period

5.1 Duration (Time-based)

Method	Description	Example
between(Temporal, Temporal)	Duration between	Duration.between(time1, time2)
ofHours(long)	Create from hours	Duration.ofHours(2)
ofMinutes(long)	Create from minutes	Duration.ofMinutes(30)
ofSeconds(long)	Create from seconds	Duration.ofSeconds(60)
toHours()	Get hours	duration.toHours()
toMinutes()	Get minutes	duration.toMinutes()
toSeconds()	Get seconds	duration.toSeconds()

5.2 Period (Date-based)

Method	Description	Example
between(LocalDate, LocalDate)	Period between	Period.between(date1, date2)
ofYears(int)	Create from years	Period.ofYears(1)
ofMonths(int)	Create from months	Period.ofMonths(6)
ofDays(int)	Create from days	Period.ofDays(10)
getYears()	Get years	period.getYears()
getMonths()	Get months	period.getMonths()
getDays()	Get days	period.getDays()

---

6. DateTimeFormatter Methods

6.1 Predefined Formatters

Formatter	Pattern	Example
ISO_LOCAL_DATE	yyyy-MM-dd	2024-03-15
ISO_LOCAL_TIME	HH:mm:ss	14:30:00
ISO_LOCAL_DATE_TIME	yyyy-MM-dd'T'HH:mm:ss	2024-03-15T14:30:00
BASIC_ISO_DATE	yyyyMMdd	20240315

. Complete Date/Time Methods Table

Class	Category	Methods
LocalDate	Creation	now(), of(), parse()
	Get	getYear(), getMonth(), getDayOfMonth()
	Modify	withYear(), withMonth(), withDayOfMonth()
	Add/Subtract	plusDays(), plusMonths(), minusDays()
	Compare	isBefore(), isAfter(), isEqual()
LocalTime	Creation	now(), of(), parse()
	Get	getHour(), getMinute(), getSecond()
	Add/Subtract	plusHours(), plusMinutes(), minusHours()
LocalDateTime	Creation	now(), of(), parse()
	Convert	toLocalDate(), toLocalTime(), atZone()
Duration	Creation	between(), ofHours(), ofMinutes()
	Get	toHours(), toMinutes(), toSeconds()
Period	Creation	between(), ofYears(), ofMonths()
	Get	getYears(), getMonths(), getDays()
DateTimeFormatter	Format	ofPattern(), ISO_LOCAL_DATE
	Parse/Format	format(), parse()

Default & Static Methods in Interfaces

7.1 Default Methods

Problem: Adding methods to interfaces breaks existing implementations.

Solution: Default methods provide implementation.

```
public interface List<E> {
  // Existing abstract method
  void add(E element);
    // New method with default implementation
  default void sort(Comparator<? super E> c) {
    Collections.sort((List)this, c);
  }
}
// Existing implementations don't break!
// Can override if needed
public class MyList<E> implements List<E> {
  public void add(E element) { }
    // Optional: Override default
  @Override
  public void sort(Comparator<? super E> c) {
    // Custom sorting
  }
}
```

7.2 Static Methods

```
public interface Utils {
  static boolean isNullOrEmpty(String s) {
    return s == null || s.isEmpty();
  }
}
// Usage
Utils.isNullOrEmpty("test");  // Call on interface, not instance
```

Concept	Key Point
Lambda	Anonymous function, concise syntax
Functional Interface	Single abstract method, @FunctionalInterface
Method Reference	Shorthand Class::method
Stream	Declarative data processing, lazy evaluation
Optional	Explicit null handling, avoid NPE
Date/Time	Immutable, clear API, thread-safe
Default Methods	Interface evolution without breaking

Java Generics - Complete Deep Dive

Generics is one of the **most powerful features** in Java, enabling type-safe, reusable code. This module covers everything from basics to advanced wildcard mechanics with internal working details.

Table of Contents

1. **Introduction to Generics** - What, Why, When, Where
2. **Generic Classes** - Definition, Internal Working, Examples
3. **Generic Methods** - Definition, Internal Working, Examples
4. **Bounded Type Parameters** - Upper & Lower Bounds
5. **Wildcards** - ?, ? extends T, ? super T
6. **Type Erasure** - Internal Working Deep Dive
7. **Common Pitfalls & Best Practices**
8. **Complete Reference Tables**

PART 1: INTRODUCTION TO GENERICS

1. What is Generics?

1.1 Definition

Generics is a Java feature that allows you to write classes, interfaces, and methods that operate on **parameterized types**. Instead of specifying a concrete type, you use a **type parameter** that gets specified when the code is used.

1.2 Before Generics (Java 1.4 and Earlier)

```
// ❌ WITHOUT GENERICS - Type unsafe
List list = new ArrayList();
list.add("Hello");     // String
list.add(123);         // Integer - Compiler allows this!

String s = (String) list.get(0);  // Requires casting
Integer i = (Integer) list.get(1); // Requires casting

// Runtime error! ClassCastException
String wrong = (String) list.get(1);  // ❌ CRASH!
```

After Generics(Java5+)

```
// ✅ WITH GENERICS - Type safe
List<String> list = new ArrayList<>();
list.add("Hello");     // String - OK
list.add(123);         // ❌ Compiler ERROR!

String s = list.get(0);  // No casting needed
// Integer i = list.get(1); // ❌ Compiler ERROR!
```

1.4 Why We Use Generics?

Problem Without Generics	Solution With Generics
Type casting required	**No casting needed**
Runtime ClassCastException	**Compile-time type checking**
Code duplication (different types)	**Reusable code** (one class for all types)
Unclear what type is stored	**Explicit type declaration**
No IDE autocomplete	**Full IDE support**

1

.5 When to Use Generics?

✅ **Use Generics When:**

- Creating collections (List, Map, Set, etc.)
- Writing utility classes that work with multiple types
- Creating APIs that need type safety, Building reusable components

❌ **Don't Use Generics When:**

- Working with primitive types directly (use wrapper classes)
- Creating static fields in generic classes
- Using instanceof with generic types
- Creating arrays of generic types

1.6 Where Generics Are Used?

Location	Example
Classes	class Box<T> { }
Interfaces	interface Comparable<T> { }
Methods	<T> void print(T t) { }
Constructors	public <T> Box(T t) { }
Collections	List<String>, Map<K,V>

---

2. Generic Type Parameters Naming Conventions

Symbol	Meaning	Common Use
T	Type	General type parameter
E	Element	Collections (List, Set)
K	Key	Map keys
V	Value	Map values
N	Number	Numeric types
R	Return	Return type of methods
?	Wildcard	Unknown type

PART 2: GENERIC CLASSES

1. What is a Generic Class?

1.1 Definition

A **Generic Class** is a class that declares one or more type parameters. These parameters are replaced with actual types when the class is instantiated.

1.2 Syntax

```
// Generic class declaration
public class ClassName<T> {
  private T value;
    public T getValue() {
     return value;
  }
  public void setValue(T value) {
    this.value = value;
  }
}
// Usage
ClassName<String> stringBox = new ClassName<>();
ClassName<Integer> integerBox = new ClassName<>();
```

1.3 Internal Working of Generic Classes

```
┌──────────────────────────────────────────────────────────┐
│              GENERIC CLASS INTERNAL WORKING              │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Source Code:                                            │
│  public class Box<T> {                                   │
│      private T value;                                    │
│      public T get() { return value; }                    │
│      public void set(T value) { this.value = value; }    │
│  }                                                       │
│                                                          │
│  After Compilation (Type Erasure):                       │
│  public class Box {                                      │
│      private Object value;        // T → Object         │
│      public Object get() {        // T → Object         │
│          return value;                                   │
│      }                                                   │
│      public void set(Object value) { // T → Object       │
│          this.value = value;                             │
│      }                                                   │
│  }                                                       │
│                                                          │
│  At Runtime:                                             │
│  Box<String> box = new Box<>();                          │
│  // JVM sees: Box box = new Box();                       │
│  // Type information is NOT available at runtime         │
│  // This is called "Type Erasure"                        │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

Why Generics Classes exist?

```
┌──────────────────────────────────────────────────────────┐
│                WHY GENERIC CLASSES?                      │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Problem Without Generics:                               │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐              │
│  │ StringBox │  │ IntegerBox │  │ DoubleBox │           │
│  │ (duplicate │  │ (duplicate │  │ (duplicate │          │
│  │  code)    │  │  code)    │  │  code)    │           │
│  └──────────┘  └──────────┘  └──────────┘              │
│                                                          │
│  Solution With Generics:                                 │
│  ┌──────────────────────────────────────┐               │
│  │              Box<T>                  │               │
│  │  (ONE class works for ALL types)     │               │
│  └──────────────────────────────────────┘               │
│        ↓            ↓            ↓                       │
│  Box<String>  Box<Integer>  Box<Double>                  │
│                                                          │
│  Benefits:                                               │
│  - Code reusability                                      │
│  - Type safety                                           │
│  - No casting                                            │
│  - Clear API                                             │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.5 How to Use Generic Classes - Examples

```
// Example 1: Simple Generic Class
public class Box<T> {
   private T content;
      public void set(T content) {
      this.content = content;
   }
      public T get() {
      return content;
   }
}

// Usage
Box<String> stringBox = new Box<>();
stringBox.set("Hello");
String value = stringBox.get();  // No casting!
Box<Integer> intBox = new Box<>();
intBox.set(123);
Integer num = intBox.get();  // No casting!
```

```
// Example 2: Multiple Type Parameters
public class Pair<K, V> {
   private K key;
   private V value;
    public Pair(K key, V value) {
      this.key = key;  this.value = value;
   }
      public K getKey() { return key; }
   public V getValue() { return value; }
}
// Usage
Pair<String, Integer> pair = new Pair<>("Age", 25);
String key = pair.getKey();     // "Age"
Integer val = pair.getValue();   // 25

// Example 3: Generic Class with Constraints
public class NumberBox<T extends Number> {
   private T number;
      public void set(T number) {
      this.number = number;
   }
    public double getDoubleValue() {
      return number.doubleValue();  // Safe - Number has this method
   }
}
// Usage
NumberBox<Integer> intBox = new NumberBox<>();
intBox.set(100);
double d = intBox.getDoubleValue();  // 100.0
// NumberBox<String> box = new NumberBox<>(); // ❌ Compiler ERROR!
```

1.6 Generic Class Methods Reference

Method Type	Example	Description
Setter	void set(T value)	Store value of type T
Getter	T get()	Retrieve value of type T
Constructor	Box(T value)	Initialize with type T
Utility	boolean contains(T value)	Check if contains value
Transformation	<R> R map(Function<T,R> f)	Transform to type R

PART 3: GENERIC METHODS

1. What is a Generic Method?

1.1 Definition

A **Generic Method** is a method that introduces its own type parameters. The type parameter scope is limited to the method itself.

1.2 Syntax

```
// Generic method declaration

// <T> comes BEFORE return type

public <T> void methodName(T parameter) {

   // method body

}// Generic method with return type

public <T> T methodName(T parameter) {

   return parameter;

}
```

1.3 Generic Method vs Generic Class

Feature	Generic Class	Generic Method
Type Parameter Scope	Entire class	Single method only
Declaration	class Box<T>	<T> void method(T t)
Type Inference	At instantiation	At method call
Static Methods	Cannot use class type param	Can have own type param

1.4 Internal Working of Generic Methods

```
┌──────────────────────────────────────────────────────────┐
│                GENERIC METHOD INTERNAL WORKING                │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Source Code:                                            │
│  public static <T> T identity(T value) {                 │
│      return value;                                       │
│  }                                                       │
│                                                          │
│  After Compilation (Type Erasure):                       │
│  public static Object identity(Object value) {           │
│      return value;                                       │
│  }                                                       │
│                                                          │
│  At Call Site:                                           │
│  String s = identity("Hello");                           │
│                                                          │
│  Compiled to:                                            │
│  String s = (String) identity((Object)"Hello");          │
│  // Compiler inserts cast automatically                  │
│                                                          │
│  Type Inference:                                         │
│  - Compiler analyzes arguments to determine T            │
│  - No need to specify <String> explicitly                │
│  identity("Hello");  // T inferred as String             │
│  identity(123);      // T inferred as Integer            │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.5 Why Generic Methods?

```
┌──────────────────────────────────────────────────────────┐
│                WHY GENERIC METHODS?                          │
├──────────────────────────────────────────────────────────┤
│                                                          │
│  Scenario: You need a utility method that works with     │
│  any type, but you don't want to make the entire class   │
│  generic.                                                │
│                                                          │
│  Without Generic Method:                                 │
│  public static Object identity(Object value) {           │
│      return value;  // Loses type information            │
│  }                                                       │
│  String s = (String) identity("Hello");  // Casting needed │
│                                                          │
│  With Generic Method:                                    │
│  public static <T> T identity(T value) {                 │
│      return value;  // Preserves type                    │
│  }                                                       │
│  String s = identity("Hello");  // No casting!           │
│                                                          │
│  Benefits:                                               │
│  - Type safety at compile time                           │
│  - No casting required                                   │
│  - Works with any type                                   │
│  - Can be static (unlike generic class methods)          │
│                                                          │
└──────────────────────────────────────────────────────────┘
```

1.6 How to Use Generic Methods - Examples

```
// Example 1: Simple Generic Method
public class Utils {
    // <T> before return type
    public static <T> T identity(T value) {
        return value;
    }
```